Theoretical Foundations of `tpeanuts`

Fast Simulation of Neutrino Oscillations in Matter

MSc Thesis (Trabajo Fin de Máster)

Juan Ramon Diaz Santos
diazjuan@alumni.uv.es

Supervisors: Roberto Ruiz de Austri Bazan & Michele Lucente

July 2026

This document accompanies the `tpeanuts` code repository and aims to connect, package by package and module by module, the physics of neutrino oscillations with its concrete implementation. Blue boxes (**Theoretical Foundation**) contain the derivations; green boxes (**Implementation**) link that theory to actual functions and classes in the code; orange boxes collect approximations and ranges of validity; text highlighted in yellow marks key results. Sections flagged as **pending** are scaffolding ready to be completed incrementally.

Contents

PART I

General Framework

Chapter 1

General Theoretical Framework and Conventions

1.1 Purpose and how to read this document

`tpeanuts` is a `PyTorch` rewrite (GPU-first, with batching support) of the original `peanuts` code [11], extended with BSM scenarios (NSI and a 3+1 sterile neutrino), a proprietary perturbative evolution scheme, and interface layers to external reference engines (`nuSQuIDS`, `MCEq`, Honda/HKKM tables, `pymsis`).

Every section in the chapters that follow corresponds to *one source file* in the repository and always follows the same structure:

- `package/module.py`

  one line describing its purpose

  identifies the exact file.
- The blue box **Theoretical Foundation** contains the physics and mathematical derivations that justify what the module computes.
- The green box **Implementation in `tpeanuts`** translates that theory into the actual functions/classes, with their essential signature and the sign, unit, or tensor-shape conventions that must be respected to use them correctly.
- The orange box **Approximations and Limitations** (when applicable) documents the range of validity, limiting cases, and numerical simplifications.
- The grey box **Key References** links the specific bibliography for that piece of physics.
- The teal box **Usage Example** (from Chapter 11 onwards) gives a complete, runnable Python snippet showing how the module's public entry points are actually called together, with prose walking through each step.
- Central results are highlighted; sections that do not yet have a complete derivation are explicitly flagged as pending and are listed in the pending-tasks index right after the table of contents.

1.2 The physics problem: neutrino oscillation in matter

A neutrino is produced and detected in a flavour eigenstate $|\nu_\alpha\rangle$, $\alpha \in \{e, \mu, \tau\}$ (plus $|\nu_s\rangle$ in the sterile sector, Chapter 4), but propagates as a superposition of mass eigenstates $|\nu_i\rangle$. The PMNS mixing matrix [17, 22] connects the two bases,

$$|\nu_\alpha\rangle = \sum_i U_{\alpha i}\ |\nu_i\rangle\,. \tag{1.1}$$

- In vacuum each mass eigenstate evolves with a kinetic phase e^{-iE_iL}.

- When crossing matter an additional effective potential from coherent forward interaction appears (the MSW effect,[18, 24]).

In both cases the evolution is summarised by an effective Hamiltonian $H(x)$ acting on the vector of flavour amplitudes,

$$i\frac{d}{dx}\,\psi(x) = H(x)\,\psi(x), \qquad \psi = \begin{pmatrix} \psi_e \\ \psi_\mu \\ \psi_\tau \end{pmatrix}, \tag{1.2}$$

where x is the dimensionless propagation coordinate defined in Section 1.3. This document organises *all* propagation code around three objects that appear systematically in every module:

1. the **Hamiltonian** $H = H_{\rm kin} + H_{\rm mat}$ (Chapters 2 and 4),
2. the **evolution operator** (evolutor) S that formally solves (1.2), $\psi(x_2) = S(x_2, x_1)\,\psi(x_1)$, obtained either exactly (constant density, Chapter 6) or numerically via segmented exponentiation (Chapter 5),
3. the **transition probability** $P(\nu_\alpha \to \nu_\beta) = |S_{\beta\alpha}|^2$ (Chapter 2), the final observable quantity.

The *Physical Media* chapters (Chapters 7 to 10) instantiate these three objects for each concrete propagation environment (vacuum, solar interior, Earth, atmosphere), and the *External Engines* chapters document how those results are cross-checked against independent reference codes.

1.3 Notation and global conventions

Theoretical Foundation

Physics is worked out in natural units $\hbar = c = 1$ unless explicitly converted to laboratory units. Public input quantities always use:

- neutrino energy E in MeV,
- squared mass differences Δm^2 in eV^2,
- path lengths L in metres,
- electron density n_e in $\mathrm{mol\,cm^{-3}}$ (molar electron density, not mass density).

The Hamiltonian is made dimensionless by introducing an *evolution scale* $L_{\rm scale}$ (by default the Earth radius $R_\oplus$) and the propagation coordinate

$$x \equiv \frac{L}{L_{\rm scale}}, \tag{1.3}$$

so that $H(x)$ in (1.2) is unitless. The two Hamiltonian terms are made dimensionless as

$$k_i = L_{\rm scale}\,\frac{\Delta m^2_{i1}}{2E}\,\frac{1}{\hbar c} \qquad \text{(kinetic term, see Chapter 2)}, \tag{1.4}$$

$$V = \pm\sqrt{2}\,G_F\,n_e\,L_{\rm scale} \qquad \text{(MSW matter potential, see Chapter 2)}, \tag{1.5}$$

with sign $+$ for neutrinos and $-$ for antineutrinos (the standard sign convention for the charged-current term).

Implementation in tpeanuts

- `util/constant.py`

  single source of truth for physical constants: G_F in MeV^{-2}, $\hbar$, c, $\hbar c$, N_A, Earth and Sun radii, the astronomical unit, the proton mass, and the mass-to-nucleon-density conversion factor.

- `util/context.py`

  `RuntimeContext` bundles `(device, dtype)` and is passed explicitly throughout the stack instead of duplicating those two arguments in every function.

- `util/type.py`

  tensor conversion and validation: resolving the complex dtype paired with a real dtype (`cdtype_from_real`), building/validating 3- or 4-flavour state vectors (`state_tensor`), and broadcasting flavour vectors to batched shapes (`broadcast_last3`).

- `util/torch_util.py`

  default device/dtype resolution, generic tensor broadcasting, batch flattening, and the convention `enforce_identity_for_zero_length` (a zero-length evolutor must collapse exactly to the identity, which numerical exponentials do not automatically guarantee in the $x \to 0$ limit).

- `core/common/neutrino.py`

  canonical flavour-index convention: $e \to 0$, $\mu \to 1$, $\tau \to 2$ (plus $s \to 3$ in the sterile sector), used throughout the code to index tensors.

Probability matrix convention

Theoretical Foundation

Given the flavour evolutor S, the final amplitude is $\psi_\beta(x_2) = \sum_\alpha S_{\beta\alpha}\,\psi_\alpha(x_1)$, so

$$P(\nu_\alpha \to \nu_\beta) = |S_{\beta\alpha}|^2\,, \tag{1.6}$$

with the *initial flavour* index in the column and the *final flavour* index in the row. This is the convention fixed, once and for the whole project, by `core.common.probability`.

Implementation in tpeanuts

`core/common/probability.py`
full derivation in Chapter 2. Every `*_probability_state(...)` function in the medium modules (`vacuum_probability_state`, `solar_probability_state`, `earth_probability_state`, `atmosphere_probability_state`) delegates to the utilities in this module to keep the index convention identical across the whole project.

1.4 Coherent states versus incoherent mixtures

Theoretical Foundation

The code systematically distinguishes two representations of the state being propagated, selected with the `massbasis` parameter:

Coherent propagation (`massbasis=False`). The state is a flavour-amplitude vector ψ_α carrying physical relative phases. The evolutor is applied directly, $\psi_{\text{final}} = S\,\psi_{\text{inicial}}$, and $P_\alpha = |\psi_{\text{final},\alpha}|^2$. This is the correct regime when trajectories are short compared to the coherence length (vacuum, atmosphere, Earth in a single leg).

Incoherent mass mixture (`massbasis=True`). The state is a vector of real weights $w_i \geq 0$, $\sum_i w_i = 1$, over mass eigenstates *without* relative-phase information. The final flavour probability is

$$P_\alpha = \sum_i \left| \left(S\,U_{\text{PMNS}}\right)_{\alpha i} \right|^2 w_i. \tag{1.7}$$

This is the physically correct regime for solar neutrinos: the spread at the production point and the enormous Sun–Earth distance completely decohere the mass superposition before it reaches Earth (Chapter 8), so only the populations w_i survive, not the phases.

Approximations and Limitations

The choice `massbasis=True/False` is not a numerical detail: it is a physical statement about whether the neutrino wave-packet's coherence length is larger or smaller than the scale of the problem. Using the incoherent mode on a short leg (e.g. only inside the atmosphere) would discard real interference; using the coherent mode for Sun–Earth propagation would overstate coherence and produce spurious interference patterns that are not observed.

1.5 Named presets and parameter traceability

Implementation in `tpeanuts`

`core/common/presets.py`
single registry (`register_preset`/`get_preset`/`list_presets`) for every named parameter set in the project: `OSCILLATION_PRESETS` (mixing angles and Δm^2, for both the standard 3-flavour sector and the 3+1 sterile extension) and `NSI_PRESETS` (non-standard couplings). Each preset carries its physical justification and bibliographic citation (e.g. [7] for standard best-fit oscillation parameters, [6] for the *LMA-Dark* preset, and [15] for the IceCube DeepCore NSI bounds), so any numerical result can be traced back to its origin in the literature.

PART II

Propagation Core — `core/`

Chapter 2

`core.common` — shared Hamiltonian, mixing, and probabilities

This chapter covers the physics infrastructure shared by *every* flavour scenario (the 3-flavour SM and its BSM extensions) and by *every* propagation medium. It is the conceptual core of the whole project: this is where the factorisation trick that makes matter propagation cheap to compute is defined.

2.1 `potential.py` — kinetic and matter potentials

`core/common/potential.py`

dimensionless (matter + kinetic) potentials, GPU-first.

Theoretical Foundation

The propagation Hamiltonian in natural units splits into a kinetic part (vacuum oscillation) and a coherent matter-interaction part (the MSW effect),

$$H = H_{\text{kin}} + H_{\text{mat}}. \tag{2.1}$$

Kinetic term. For a mass eigenstate i with energy $E \gg m_i$, the phase accumulated relative to the lightest mode after travelling a distance L is $\Delta m^2_{i1} L/(2E)$ (in natural units, then divided by $\hbar c$ to convert to laboratory units). Introducing the evolution scale L_{scale} (by default the Earth radius $R_\oplus$) and the dimensionless coordinate $x = L/L_{\text{scale}}$, the dimensionless kinetic eigenvalue entering the Hamiltonian is

$$k_i = L_{\text{scale}} \frac{\Delta m^2_i}{2E} \frac{1}{\hbar c}. \tag{2.2}$$

Charged- versus neutral-current matter potentials. Coherent forward scattering off the background medium proceeds through two distinct weak-interaction channels, each contributing an independent term to H_{mat} and computed by its own dedicated function:

- **Charged current (CC).** Only ν_e ($\bar{\nu}_e$) can exchange a $W^\pm$ with the ambient electrons (there are no positrons, muons, or taus to scatter off in ordinary matter), so this channel singles out the electron flavour alone. It adds an effective Wolfenstein-type potential [24],
$$V_{CC} = \sqrt{2}\, G_F\, n_e, \tag{2.3}$$
which only affects the ee entry of the flavour-basis Hamiltonian. Made dimensionless with the scale L_{scale} and computed by `matter_potential_cc`,
$$V_{CC} = \pm\sqrt{2}\, G_F\, n_e\, L_{\text{scale}}, \qquad (+ \text{ for } \nu,\ - \text{ for } \bar{\nu}). \tag{2.4}$$
- **Neutral current (NC).** A Z^0 is exchanged with electrons, protons, *and* neutrons alike, and identically across all three *active* flavours (lepton universality). In

electrically neutral matter the proton and electron NC contributions cancel exactly, leaving

$$V_{NC} = -\frac{1}{\sqrt{2}}\, G_F\, n_n, \tag{2.5}$$

where n_n is the neutron density. Made dimensionless the same way and computed by `matter_potential_nc`,

$$V_{NC} = \mp\frac{1}{\sqrt{2}}\, G_F\, n_n\, L_{\text{scale}}\,, \qquad (- \text{ for } \nu,\ + \text{ for } \bar{\nu}). \tag{2.6}$$

For antineutrinos both potentials flip sign, $V \to -V$ (the weak interaction has opposite sign for particle/antiparticle), which is why both functions take the same `antinu` flag. In the pure 3-flavour Standard Model, $V_{NC}\,\mathbb{I}_3$ is a common phase across all three active flavours and therefore has *no effect on any oscillation probability*: this is why V_{NC} never appears at all in the SM matter Hamiltonian, and why n_n is nowhere plumbed through the SM code path.
The 3+1 sterile extension changes this qualitatively, since the sterile flavour is a gauge singlet and *does not* receive V_{NC}: subtracting the same global phase $V_{NC}\,\mathbb{I}_4$ from $\mathrm{diag}(V_{CC} + V_{NC}, V_{NC}, V_{NC}, 0)$ leaves a genuine relative term on the sterile diagonal entry,

$$\mathrm{diag}(V_{CC} + V_{NC}, V_{NC}, V_{NC}, 0) - V_{NC}\,\mathbb{I}_4 = \mathrm{diag}(V_{CC}, 0, 0, -V_{NC}), \tag{2.7}$$

which *does* affect sterile-active oscillation probabilities, with magnitude comparable to V_{CC} itself: for isoscalar matter (equal proton and neutron density, e.g. Earth's mantle) $V_{NC}/V_{CC} = n_n/(2n_e) \approx 1/2$, independent of the absolute density. See `hamiltonian.py` below for how this term enters the reduced Hamiltonian, and Chapters 9 and 10 for how n_n is exposed from the Earth and atmosphere density profiles.

Implementation in `tpeanuts`

- `kinetic_potential(DeltamSq, E, evolution_scale_m, ...)` implements $k = L_{\text{scale}}\,\Delta m^2/(2E\hbar c)$, accepting scalar or tensor inputs with automatic `device`/`dtype` inference.
- `matter_potential_cc(n_e, antinu, evolution_scale_m, ...)` implements $V_{CC} = \pm\sqrt{2} G_F n_e L_{\text{scale}}$; the sign is set by the boolean flag `antinu`.
- `matter_potential_nc(n_n, antinu, evolution_scale_m, ...)` implements $V_{NC} = \mp\frac{1}{\sqrt{2}} G_F n_n L_{\text{scale}}$ (same `antinu` sign convention as the CC potential). It has no `legacy_precision` counterpart (see below), since the legacy `peanuts` reference never modelled the sterile extension.
- Public API input units: n_e, n_n in $\mathrm{mol\,cm^{-3}}$, Δm^2 in eV^2, E in MeV, `evolution_scale_m` in metres (default $R_\oplus$, see `util.constant.R_E`).

Approximations and Limitations

The `legacy_precision` flag. Both `matter_potential_cc` and `kinetic_potential` accept a boolean `legacy_precision` keyword, but only `matter_potential_cc` does anything

with it: it selects which numerical value of the matter-potential prefactor $\sqrt{2}\,G_F\,n_e$ is used.

- `legacy_precision=False` (the default): the prefactor $\sqrt{2}\,G_F\,N_A \times 10^6\,(\hbar c)^2$ is computed once, at import time, from the full double-precision constants in `util/constant.py` (see the table below).

- `legacy_precision=True`: the prefactor is replaced by the single hardcoded value 3.868×10^{-7} MeV m^2, reproduced from Eq. 4.17 of the legacy `peanuts` reference implementation [11], itself only quoted to 4 significant digits. This mode exists exclusively to reproduce `peanuts` output bit-for-bit in the legacy validation tests (`medium/*/test/*legacy*`); it is **not** intended for physics production use, since it discards precision that is otherwise available at no cost.

- `kinetic_potential` accepts the same keyword purely for call-site symmetry with `matter_potential` (so both can be driven by one shared `legacy_precision` switch); no legacy-rounded kinetic prefactor exists, so the flag is a no-op there.

Symbol	Description	Value	Units
G_F	Fermi coupling constant	$1.1663787 \times 10^{-11}$	MeV^{-2}
N_A	Avogadro constant	$6.02214076 \times 10^{23}$	mol^{-1}
$\hbar$	Reduced Planck constant	$6.582119569 \times 10^{-22}$	MeV s
c	Speed of light (exact)	2.99792458×10^{8}	m s^{-1}
$\hbar c$	derived from $\hbar$, c	$1.9732698 \times 10^{-13}$	MeV m
Matter-potential prefactor $\sqrt{2}\,G_F\,N_A \times 10^6 (\hbar c)^2$:			
full precision	derived from the constants above	3.86793×10^{-7}	MeV m^2
legacy (Eq. 4.17, [11])	hardcoded, 4 sig. digits	3.868×10^{-7}	MeV m^2

Precision difference. The two prefactors differ by a relative 1.85×10^{-5} (~ 15 parts per million) — five orders of magnitude larger than double-precision machine epsilon ($\approx 2.2 \times 10^{-16}$). This confirms the discrepancy is not a floating-point rounding artefact but a deliberate consequence of the legacy reference value carrying only 4 significant digits; `legacy_precision` therefore trades a small, well-quantified accuracy loss for exact reproducibility of the legacy `peanuts` numbers.

Approximations and Limitations

- **Ultrarelativistic kinetic term.** *Formulation:* the mass-eigenstate energy is linearised as $E_i \approx E + \Delta m_i^2/(2E)$ to obtain $k_i = L_{\text{scale}}\,\Delta m_i^2/(2E\hbar c)$ (`kinetic_potential`). *Justification and validity:* requires $E \gg m_i$ for every propagating mass eigenstate; for the solar/reactor/atmospheric energies simulated here ($E \gtrsim$ MeV) and sub-eV neutrino masses this holds to better than one part in 10^{12}, so the linearisation is exact for every scenario in this project.

- **Coherent forward elastic scattering.** *Formulation:* only the forward ($\theta = 0$)

elastic-scattering amplitude contributes to V_{CC}/V_{NC} — the standard Wolfenstein index-of-refraction argument [24]; incoherent (large-angle or inelastic) scattering is neglected entirely. *Justification and validity:* holds whenever the neutrino's mean free path for incoherent scattering is far longer than the propagation baseline, true throughout the Sun, Earth, and atmosphere at the MeV–GeV energies simulated here.

- **Electrically neutral, unpolarised matter for V_{NC}.** *Formulation:* $n_p = n_e$ is assumed when deriving $V_{NC} = -\frac{1}{\sqrt{2}} G_F n_n$, so the proton and electron NC contributions cancel exactly and only the neutron density survives; the medium is further assumed unpolarised and at rest, so no macroscopic spin- or bulk-velocity-dependent term is added to H_{mat}. *Justification and validity:* exact for ordinary, non-magnetised astrophysical matter — the Sun, Earth, and atmosphere all satisfy local charge neutrality to extremely high precision, and carry no coherent spin polarisation on the scales relevant here.

- **Local, instantaneous density.** *Formulation:* V_{CC} and V_{NC} are evaluated from the density $n_e(x)$, $n_n(x)$ at the neutrino's instantaneous position, treating the medium as locally static and homogeneous on the scale of the local oscillation length. *Justification and validity:* requires the density profile to vary slowly compared to that oscillation wavelength; the segmented-evolutor machinery (`evolutor.py` below, and the numerical and perturbative chapters, Chapters 5 and 6) exists precisely to control the residual error from this approximation by refining the segmentation where the density changes fastest.

2.2 pmns.py — shared mixing infrastructure

`core/common/pmns.py`

abstract class ***PMNS*** *and container* ***PMNSParams****, common to every flavour scenario.*

Theoretical Foundation

Mixing angles and the CP-violating phase

The active-flavour sector is parametrised by three mixing angles and one CP phase, all real (angles in radians) [21]:

- θ_{12} (**solar** angle) sets the (e, μ)-plane rotation; the mixing angle measured by solar and by KamLAND reactor data, and the largest of the three.
- θ_{13} (**reactor** angle) sets the (e, τ)-plane rotation; the smallest angle, measured by short-baseline reactor experiments (Daya Bay, RENO, Double Chooz). Its non-zero value is what makes CP violation observable at all in the lepton sector.
- θ_{23} (**atmospheric** angle) sets the (μ, τ)-plane rotation; measured by atmospheric and long-baseline accelerator experiments, close to (possibly exactly) maximal, $\theta_{23} \approx \pi/4$.
- δ (**Dirac CP phase**) is the only source of CP violation carried by this parametrisation (Majorana phases, relevant only if neutrinos are Majorana particles, do not affect oscillation probabilities and are absent from it). δ has no physical, rephasing-

invariant effect unless all three angles are non-zero; it enters oscillation probabilities through the Jarlskog invariant (PMNS.jarlskog_invariant, see below).

These four scalars, together with a RuntimeContext fixing the target device/dtype, are exactly the contents of PMNSParams — nothing else: the mass-squared splittings do not enter the mixing matrix, so they are kept out of this container and live in MassSpectrum instead (next section).

Elementary rotation and phase generators

Every mixing matrix in this project, SM or BSM, is assembled from two elementary building blocks that PMNS implements once, sized generically by self.n_flavours so the same formulas serve the 3×3 active-only case and any larger active+sterile case (BSM chapter). Restricted to the base 3×3 active sector, with the fixed flavour order $(e, \mu, \tau) = (0, 1, 2)$, they read explicitly:

$$R_{12}(\theta_{12}) = \begin{pmatrix} \cos\theta_{12} & \sin\theta_{12} & 0 \\ -\sin\theta_{12} & \cos\theta_{12} & 0 \\ 0 & 0 & 1 \end{pmatrix} \qquad \text{rotation in the } (e,\mu) \text{ plane;} \tag{2.8}$$

$$R_{13}(\theta_{13}) = \begin{pmatrix} \cos\theta_{13} & 0 & \sin\theta_{13} \\ 0 & 1 & 0 \\ -\sin\theta_{13} & 0 & \cos\theta_{13} \end{pmatrix} \qquad \text{rotation in the } (e,\tau) \text{ plane;} \tag{2.9}$$

$$R_{23}(\theta_{23}) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos\theta_{23} & \sin\theta_{23} \\ 0 & -\sin\theta_{23} & \cos\theta_{23} \end{pmatrix} \qquad \text{rotation in the } (\mu,\tau) \text{ plane;} \tag{2.10}$$

$$\Delta(\delta) = \mathrm{diag}(1,\, 1,\, e^{i\delta}) \qquad \text{CP phase, carried at the } \tau \text{ index.} \tag{2.11}$$

R_{12}, R_{13}, R_{23} are always **real**: PMNS never attaches a phase to them (they call the shared generator with phase=None); only Δ carries δ, and only at position (τ, τ). This is a modelling choice of the parametrisation, not a computational shortcut: the same underlying generator (PMNS._rot) accepts an arbitrary rotation plane (i, j) and an arbitrary phase, which is exactly what lets the 3+1 sterile extension reuse it unchanged for the active-sterile rotations R_{14}, R_{24}, R_{34} (each carrying its own CP phase) without modifying this base class at all (BSM chapter).
How R_{12}, R_{13}, R_{23}, Δ combine into a full mixing matrix — the product order, and which rotations sit inside versus outside the block that commutes with the matter potential — is scenario-specific and is given in the SM and BSM chapters.

Flavour, mass, and reduced bases

Three bases appear throughout this project:

- the **flavour basis** $\alpha = e, \mu, \tau(, s)$: the physical basis in which neutrinos are produced and detected, and the basis in which the matter potential is always diagonal. In the plain 3-flavour Standard Model, or the 3+1 sterile extension without its optional neutral-current term, only the electron entry is populated, $H_{\mathrm{mat}} = \mathrm{diag}(V_{CC}, 0, \ldots, 0)$; once that neutral-current term is active, $H_{\mathrm{mat}} = \mathrm{diag}(V_{CC}, 0, 0, -V_{NC})$ carries a second, non-zero entry on the sterile diagonal

(potential.py section above, hamiltonian.py section below) — diagonal in every case, but not necessarily supported on the electron entry alone;

- the **mass basis** $i = 1, \ldots, n$: the eigenbasis of the free-propagation (kinetic-only) part of the Hamiltonian, in which the propagation phases $e^{-ik_i x}$ are defined;
- an intermediate **reduced basis**, introduced purely for computational convenience (hamiltonian.py section below).

Every concrete PMNS scenario provides a unitary U (pmns_matrix) relating the flavour and mass bases: an operator A_{flavour} and its mass-basis counterpart A_{mass} are related by $A_{\text{flavour}} = U\, A_{\text{mass}}\, U^\dagger$. In addition, it factorises this U as

$$U = O\, U_{\text{red}}, \tag{2.12}$$

for a second unitary U_{red} (reduced, the *reduced mixing matrix*) and a third, scenario-specific unitary O (outer_block, the *outer block*). This factorisation defines the reduced basis: it is the basis reached from the flavour basis by applying O *alone*,

$$A_{\text{flavour}} = O\, A_{\text{reduced}}\, O^\dagger, \qquad A_{\text{reduced}} = O^\dagger\, A_{\text{flavour}}\, O, \tag{2.13}$$

implemented generically, for any scenario's O, by flavour_basis and reduced_basis (see the implementation box below). U_{red} is precisely the remaining rotation that carries a reduced-basis operator the rest of the way to the true mass basis: combining $A_{\text{flavour}} = U\, A_{\text{mass}}\, U^\dagger = O\, A_{\text{reduced}}\, O^\dagger$ with $U = O\, U_{\text{red}}$ gives the reduced-to-mass-basis transformation and its inverse,

$$A_{\text{reduced}} = U_{\text{red}}\, A_{\text{mass}}\, U_{\text{red}}^\dagger, \qquad A_{\text{mass}} = U_{\text{red}}^\dagger\, A_{\text{reduced}}\, U_{\text{red}}, \tag{2.14}$$

so the reduced basis coincides with the mass basis exactly when O is the identity (equivalently $U_{\text{red}} = U$). That U factorises this way, and that flavour_basis/reduced_basis implement exactly the O-dressing above, is the core.common contract every concrete PMNS subclass must satisfy; *which* concrete U, O, U_{red} a given scenario provides is scenario-specific and is given in the SM and BSM chapters.

Implementation in tpeanuts

- PMNSParams: immutable container for $\theta_{12}, \theta_{13}, \theta_{23}, \delta$ and the RuntimeContext.
- PMNS (abstract torch.nn.Module): builds the shared generators R12, R13, R23, Delta above, sized generically by self.n_flavours (3 for the SM, 4 for the 3+1 sterile case), and implements the part of the public interface that is *independent of the number of flavours and of the product structure*: refresh, flavour_basis, reduced_basis, select_antinu (see box below), conjugate, transpose, dagger, the reduced-matrix variants reduced_conjugate/reduced_transpose/reduced_dagger, and the rephasing-invariant jarlskog_invariant.
- Concrete subclasses only need to supply pmns_matrix, reduced, and outer_block: how the four generators above combine into the full and reduced mixing matrices is genuinely different per scenario, so those three methods remain abstract here.
- flavour_basis / reduced_basis apply the generic dressing $O(\cdot)O^\dagger$ / its inverse $O^\dagger(\cdot)O$, for *any* scenario-specific outer block O (outer_block) and any reduced-

basis operator (not only the Hamiltonian); why this reproduces the full-flavour result exactly is shown in the `hamiltonian.py` section below.

Implementation in tpeanuts

`select_antinu(U, antinu)`. Applies the standard antineutrino prescription to a full or reduced mixing matrix: returns U unchanged for neutrinos and its complex conjugate U^* for antineutrinos (the sign flip of the matter potential, $V \to -V$, is handled separately by `potential.matter_potential`, not here). `antinu` may be a plain Python `bool` (applies globally) or a boolean tensor broadcast against the batch dimensions of U via `torch.where`, so a single batched call can mix neutrinos and antineutrinos within the same tensor — e.g. when propagating an atmospheric flux that contains both. Every mixing matrix exposed by `PMNS` (`pmns_matrix`, `reduced`, `outer_block`) routes through this single function, so the neutrino/antineutrino convention is enforced in exactly one place.

2.3 mass_spectrum.py — mass-squared splittings

`core/common/mass_spectrum.py`

abstract class `MassSpectrum`: the two measurable mass-squared splittings shared by every oscillation model.

Theoretical Foundation

Oscillation probabilities depend on mass-squared *differences*, never on absolute masses, so the mass sector is parametrised by two independent, directly measurable splittings [21]:

- $\Delta m^2_{21} \equiv m^2_2 - m^2_1$ (**solar** splitting): by convention always positive ($m_2 > m_1$); measured by solar and KamLAND reactor data, $\Delta m^2_{21} \approx 7.4 \times 10^{-5}$ eV^2. Sets the "slow" oscillation frequency.
- $\Delta m^2_{3\ell}$ (**atmospheric** splitting): $\Delta m^2_{31} \equiv m^2_3 - m^2_1$ if m_3 is the heaviest state (**normal ordering**, $\ell = 1$), or $\Delta m^2_{32} \equiv m^2_3 - m^2_2$ if m_3 is the lightest (**inverted ordering**, $\ell = 2$); measured by atmospheric and long-baseline accelerator data, $|\Delta m^2_{3\ell}| \approx 2.5 \times 10^{-3}$ eV^2. Sets the "fast" oscillation frequency.

These are exactly the two fields stored by `MassSpectrum`: no absolute mass scale, and no third independent splitting (since $\Delta m^2_{32} = \Delta m^2_{31} - \Delta m^2_{21}$ by construction, only two of the three pairwise splittings are independent).

Normal and Inverted Ordering

Whether m_3 is the heaviest or the lightest mass eigenstate — the **neutrino mass ordering** — is not yet known experimentally, and is not fixed by any mixing angle: it is encoded entirely in the *sign* of $\Delta m^2_{3\ell}$, which `MassSpectrum` takes as a single signed input rather than as a separate ordering flag. Concretely, `difference_vector_base()` builds

the reduced (global-phase-removed) three-component vector

$$(\Delta m^2_1,\ \Delta m^2_2,\ \Delta m^2_3) = \begin{cases} (0,\ \Delta m^2_{21},\ \Delta m^2_{3\ell}) & \Delta m^2_{3\ell} > 0 \text{ (normal ordering, NO)}, \\ (-\Delta m^2_{21},\ 0,\ \Delta m^2_{3\ell}) & \Delta m^2_{3\ell} < 0 \text{ (inverted ordering, IO)}. \end{cases} \tag{2.15}$$

Both branches use the same stored $\Delta m^2_{21} > 0$ and the same $|\Delta m^2_{3\ell}|$: only the sign of the single scalar $\Delta m^2_{3\ell}$ passed in switches the ordering, there is no separate boolean or string argument anywhere in the API. This is a common source of confusion when porting values from sources that specify ordering explicitly: the sign must be set on $\Delta m^2_{3\ell}$ itself before it reaches `MassSpectrum`. The batched form evaluates both branches and selects per-element with `torch.where`, so a single tensor of $\Delta m^2_{3\ell}$ values with mixed signs is handled correctly in one call.

Implementation in tpeanuts

- `MassSpectrum` (abstract `dataclass`): stores Δm^2_{21}, $\Delta m^2_{3\ell}$ as its only fields.
- `difference_vector_base()`: concrete, shared by every scenario; builds the 3-component vector above.
- `difference_vector()`: abstract; sizes the base vector to the concrete scenario's flavour count — returned unchanged for the plain 3-flavour case (`MassSpectrum_SM`, SM chapter), with a fourth entry Δm^2_{41} appended for the 3+1 sterile case (`MassSpectrum_BSM`, BSM chapter). This split exists because sizing is scenario-specific while the active-sector vector itself is not.
- Consumed by `kinetic_eigenvalue_vector` (`hamiltonian.py`, next section) to build the dimensionless kinetic phases k_i, and bundled into `OscillationParameters` as the `mass_spectrum` field (next section).

2.4 oscillation.py — bundled oscillation state

core/common/oscillation.py

`OscillationParameters`: an immutable container grouping `pmns`, `mass_spectrum`, `antinu`, and the optional `nsi` coupling.

Implementation in tpeanuts

This introduces no new physics: it bundles `pmns`, the mass-squared splittings, the antineutrino selector, and the optional NSI coupling that the propagation stack needs into a single immutable object, rather than threading them as independent arguments through every function. `OscillationParameters` is a passive domain container: it does not build itself, and it never imports the concrete SM/BSM/preset implementations it bundles. Building a fully assembled `OscillationParameters` from a named preset — choosing the concrete PMNS/mass-spectrum implementation and reading the parameter values — is done by `PropagationConfig.oscillation_parameters_from_preset` in `tpeanuts/config/propagation.py`, from the preset registry in `tpeanuts/config/presets.py`: that composition layer is intentionally kept outside this module, which stays a plain container.

Attribute	Type	Default	Role
pmns	PMNS	required	Built mixing object (PMNS_SM or PMNS_sterile).
mass_spectrum	MassSpectrum	required	Built splitting object (MassSpectrum_SM or MassSpectrum_BSM).
antinu	bool/Tensor	False	Antineutrino selector ($U \to U^*$, $V \to -V$).
nsi	NSIConfig or None	None	Optional NSI coupling; None means the plain SM matter potential.
preset_name	str	"_SM_NUFIT52_NO"	Name of the preset used to build this object; "custom" for the bare dataclass default (explicit construction, no preset).
ordering	str	"NO"	Mass-ordering label carried by the preset ("NO"/"IO"); "" for the bare dataclass default.
label	str	""	Short identifier string carried by the preset.
description	str	""	Human-readable description/reference carried by the preset.
Derived BSM-extension flags (read-only @property, not stored fields):			
BSM_extension_sterile	bool	from pmns	True iff pmns.n_flavours == 4 (a 3+1 sterile object).
BSM_extension_NSI	bool	from nsi	True iff nsi is not None.
BSM_extension	bool	OR of the two above	True iff *any* BSM extension is active; the routing flag checked by core.BSM, core.numerical, and core.perturbative to pick the SM-only fast path or the general BSM path.

Approximations and Limitations

The three BSM_extension* flags are *derived*, never stored: they are recomputed on every access from pmns/nsi rather than set at construction time, so they can never drift out of sync with the objects they describe (there is no way to end up with BSM_extension_sterile=True while pmns is actually a 3-flavour PMNS_SM). The design rationale for keeping mass_spectrum separate from pmns is that the PMNS mixing matrix does not depend on $\Delta m^2_{21}/\Delta m^2_{3\ell}$ (those parameters only enter through the kinetic term, previous sections), so storing them on PMNSParams as well would duplicate the same values.

Default Configuration

Unless a different preset is requested, the reference configuration used throughout this project's examples and tests is the NuFIT 5.2 (2022) Standard Model best fit with normal ordering, registered as `"_SM_NUFIT52_NO"` in `tpeanuts.config.presets.OSCILLATION_PRESETS` [7, 19]. Built via `PropagationConfig.oscillation_parameters_from_preset("_SM_NUFIT52_NO")`, it populates `OscillationParameters` as follows:

Attribute	Default value	Note
θ_{12}	$33.41°$	solar angle
θ_{13}	$8.58°$	reactor angle
θ_{23}	$49.0°$	atmospheric angle
δ	$197.0°$	Dirac CP phase
Δm^2_{21}	7.41×10^{-5} eV^2	solar splitting
$\Delta m^2_{3\ell}$	$+2.511 \times 10^{-3}$ eV^2	atmospheric splitting (positive)
`antinu`	`False`	neutrino, not antineutrino
`ordering`	`"NO"`	normal ordering, set by the positive sign above
`BSM_extension_sterile`	`False`	`pmns` is 3-flavour, not sterile
`BSM_extension_NSI`	`False`	`nsi = None`
`BSM_extension`	`False`	no BSM extension active

2.5 `hamiltonian.py` — the propagation Hamiltonian

`core/common/hamiltonian.py`

the single Hamiltonian-assembly module in the project: correct and complete for the 3-flavour SM, the 3+1 sterile extension, Non-Standard Interactions, or any combination, with no separate SM-only implementation.

Theoretical Foundation

General construction of the flavour Hamiltonian

In the flavour basis (`pmns.py` section above), the propagation Hamiltonian is, in full generality, the mass-basis kinetic term rotated into the flavour basis by the full mixing matrix U, plus whatever matter term $H_{\rm mat}$ the active scenario contributes:

$$H_{\rm flavour} = U\ \mathrm{diag}(k_1, \ldots, k_n)\, U^\dagger + H_{\rm mat}. \tag{2.16}$$

This decomposition is always correct, for *any* unitary U and *any* Hermitian $H_{\rm mat}$, with no assumption on how U factorises or on what $H_{\rm mat}$ concretely is: it is simply the Schrödinger equation written in the flavour basis. What *does* depend on the scenario is the concrete content of $H_{\rm mat}$ (`potential.py` section above): plain 3-flavour Standard Model, or 3+1 sterile *without* its optional neutral-current term, $H_{\rm mat} = \mathrm{diag}(V_{CC}, 0, \ldots, 0)$, populated on the electron entry alone; with Non-Standard Interactions and/or the sterile neutral-current term active, $H_{\rm mat}$ acquires extra diagonal and/or off-diagonal entries beyond the

electron one (full expression below, in "The 3+1 sterile neutral-current term" subsection). Taken at face value, evolving H_{flavour} is expensive in every case: since U and H_{mat} do not commute in general, H_{flavour} must be re-diagonalised as a genuinely complex $n \times n$ matrix at every point along the path where the electron (and, when the NC term is active, neutron) density changes.

The reduction trick: factorising out the outer block

Every concrete `PMNS` scenario additionally factorises its mixing matrix as $U = O\,U_{\text{red}}$ (`pmns.py` section above), where the scenario-specific **outer block** O (`outer_block`) is constructed, in every scenario implemented so far, to leave the electron flavour entry fixed, and therefore to commute exactly with the *plain charged-current* matter term:

$$O\ \mathrm{diag}(V_{CC}, 0, \ldots, 0)\, O^\dagger = \mathrm{diag}(V_{CC}, 0, \ldots, 0). \tag{2.17}$$

This commutation is a structural property of O alone, and holds for *any* scenario's O and U_{red} — but, as stressed below, only for this plain CC-only piece of H_{mat}, not for H_{mat} in general. Whenever H_{mat} reduces to this plain CC-only form (i.e. neither NSI nor the sterile neutral-current term is active), the general expression above simplifies to

$$\begin{aligned} H_{\text{flavour}} &= U\,\mathrm{diag}(k_1, \ldots, k_n)\, U^\dagger + \mathrm{diag}(V_{CC}, 0, \ldots, 0) \\ &= O\,U_{\text{red}}\,\mathrm{diag}(k)\,U_{\text{red}}^\dagger\,O^\dagger + O\ \mathrm{diag}(V_{CC}, 0, \ldots, 0)\,O^\dagger \\ &= O\,\underbrace{\left[U_{\text{red}}\,\mathrm{diag}(k)\,U_{\text{red}}^\dagger + \mathrm{diag}(V_{CC}, 0, \ldots, 0)\right]}_{H_{\text{red}}}\,O^\dagger \;=\; O\,H_{\text{red}}\,O^\dagger. \end{aligned} \tag{2.18}$$

Computational consequence (the peanuts "reduction trick"). The n-flavour evolution problem reduces to:

1. solving the evolution of the smaller, cheaper reduced Hamiltonian H_{red}, obtaining $S_{\text{red}}(x)$;
2. *dressing* the result once with the constant matrix O (which depends on neither density nor x), $S_{\text{flavour}}(x) = O\,S_{\text{red}}(x)\,O^\dagger$.

This is exactly what `pmns.flavour_basis` does: the spectral calculations that diagonalise the propagation Hamiltonian only ever need to act on H_{red}, never on H_{flavour}. This shortcut is *not* available once H_{mat} carries an NSI or sterile neutral-current contribution, since neither commutes with O in general (see below, and the dedicated implementation box on the explicit BSM H_{mat} construction).

Assembling the reduced Hamiltonian

Given the reduced mass-squared vector $(\Delta m_1^2, \ldots, \Delta m_n^2)$ from `MassSpectrum` (`mass_spectrum.py` section above) and the reduced mixing matrix U_{red} of the active `PMNS` scenario, the reduced Hamiltonian is

$$H_{\text{red}} = U_{\text{red}}\ \mathrm{diag}(k_1, \ldots, k_n)\, U_{\text{red}}^\dagger + \mathrm{diag}(V_{CC}, 0, \ldots, 0)\,. \tag{2.19}$$

Each dimensionless kinetic eigenvalue k_i entering $\mathrm{diag}(k_1, \ldots, k_n)$ is obtained by feeding the corresponding component of that mass-squared vector, together with the neutrino energy E, through `kinetic_potential` (`potential.py` section above); applying this conversion to the whole vector at once, after first sizing `MassSpectrum.difference_vector()`

to `oscillation.pmns`'s flavour count, is exactly what `kinetic_eigenvalue_vector` (below) does. The matter term populates only the electron-flavour entry, by construction of H_{mat} (`potential.py` section above): this holds for any flavour count n, so no special handling is needed as n grows — every additional flavour beyond the first simply carries a zero matter coupling.
When Non-Standard Interactions are active, the matter term generalises to $V_{CC}\,(\text{diag}(1,0,\ldots,0)+\varepsilon)$ for a Hermitian coupling matrix ε, built in the flavour basis and rotated back into the reduced basis with $O^\dagger(\cdot)O$.

The 3+1 sterile neutral-current term, and why it breaks the reduction trick

Optionally supplying a neutron density n_n for a 4-flavour `PMNS` scenario (`potential.py` section above) adds the relative neutral-current term derived there to the flavour-basis matter Hamiltonian,

$$H_{\text{mat}} = \text{diag}(V_{CC},0,0,0) + V_{CC}\,\varepsilon - \text{diag}(0,0,0,V_{NC}), \tag{2.20}$$

(the NSI term present only when ε is set). Crucially, the outer block O of the 3+1 sterile scenario mixes the $\{\mu,\tau,s\}$ block (Chapter 4) and therefore does *not* commute with a term that singles out the sterile diagonal entry:

$$O\ \text{diag}(0,0,0,V_{NC})\,O^\dagger \neq \text{diag}(0,0,0,V_{NC}). \tag{2.21}$$

This means the reduction-trick shortcut of the previous subsection — skipping the $O^\dagger(\cdot)O$ rotation entirely when H_{mat} is already diagonal in the reduced basis — is only valid when *neither* NSI *nor* the neutral-current term is active. Whenever either is present, `hamiltonian_matter_reduced` always takes the general path: build H_{mat} in the flavour basis (as above) and rotate it, $H^{\text{red}}_{\text{mat}} = O^\dagger\,H_{\text{mat}}\,O$. Omitting n_n (the default) recovers the CC-only sterile matter term used throughout the rest of this thesis, unchanged; the term has no effect at all for a 3-flavour `PMNS` (there V_{NC} is the unobservable global phase discussed in `potential.py`), so passing n_n to a 3-flavour scenario is silently ignored rather than raising.

Implementation in `tpeanuts`

- `kinetic_eigenvalue_vector(oscillation, E, ...)` (`core/common/hamiltonian.py`) converts the mass spectrum's `difference_vector()` (previous section) into the dimensionless kinetic phases k_i via `kinetic_potential` (`potential.py`, previous sections).
- `hamiltonian_kinetic_reduced(...)` implements $U_{\text{red}}\,\text{diag}(k)\,U_{\text{red}}^\dagger$ for any flavour count, always using the Hermitian conjugate.
- `hamiltonian_matter_reduced(oscillation, n_e_mol_cm3, *, n_n_mol_cm3=None, ...)` implements $\text{diag}(V_{CC},0,\ldots,0)$ (no rotation) when neither NSI nor a genuine sterile NC term is requested, or the general rotated construction above otherwise. The `n_n_mol_cm3` keyword is optional and only meaningful for a 4-flavour `oscillation.pmns`.
- `_matter_direction_reduced(oscillation, *, antinu, include_nc, context)` (module-private): factors the reduced matter Hamiltonian into

$H^{\text{red}}_{\text{mat}} = V_{CC}\,P_{CC} + V_{NC}\,P_{NC}$, returning the two density-independent "direction" matrices P_{CC}, P_{NC} on their own. Shared by `hamiltonian_matter_reduced` and by the perturbative first-order correction (Chapter 6), which needs the direction matrices in isolation to sandwich the spectral projectors.

- `hamiltonian_reduced(...)` / `hamiltonian_flavour(...)` sum both terms directly from n_e (and optional n_n) and the splittings; `hamiltonian_flavour` additionally applies the $O(\cdot)O^\dagger$ dressing via `pmns.flavour_basis`.

Explicit construction of H_{mat} under BSM extensions

`hamiltonian_matter_reduced` never rebuilds H_{mat} term by term at every call; it factorises it into position-*independent* "direction" matrices and position-*dependent* scalar potentials, via `_matter_direction_reduced`:

$$H^{\text{red}}_{\text{mat}} = V_{CC}\,P_{CC}\;[+\,V_{NC}\,P_{NC}]. \tag{2.22}$$

Concretely, in code order:

- **Neither NSI nor the sterile NC term requested** (the common case). Fast path: $P_{CC} = \text{diag}(1, 0, \ldots, 0)$ directly in the reduced basis, with *no* rotation, since this direction is exactly O-invariant ((2.18) above); P_{NC} is not built at all.
- **NSI and/or the sterile NC term requested.** General path: build the flavour-basis direction matrices $D_{\text{flavour}} = \text{diag}(1, 0, \ldots, 0) + \varepsilon$ (with $\varepsilon = 0$ if NSI is off) and, only if the NC term is requested, $D_{n,\text{flavour}} = -\,\text{diag}(0, \ldots, 0, 1)$ (the sterile entry); rotate both into the reduced basis with the scenario's outer block O,

$$P_{CC} = O^\dagger\,D_{\text{flavour}}\,O, \qquad P_{NC} = O^\dagger\,D_{n,\text{flavour}}\,O. \tag{2.23}$$

- `hamiltonian_matter_reduced` then evaluates the position-dependent scalar potentials — V_{CC} always, via `matter_potential_cc`, and, only when `n_n_mol_cm3` is supplied to a 4-flavour scenario, V_{NC} via `matter_potential_nc` — and assembles $H^{\text{red}}_{\text{mat}} = V_{CC}\,P_{CC}\,[+\,V_{NC}\,P_{NC}]$ above.

This split is exactly what lets `evolutor_first_order` (Chapter 6) reuse P_{CC}, P_{NC} directly, sandwiching them against the spectral projectors without recomputing the rotation at every energy or density sample.

Hermitian Conjugate vs. Plain Transpose

The reduced kinetic term is always built as $U_{\text{red}}\,\text{diag}(k)\,U^\dagger_{\text{red}}$ (the Hermitian conjugate, never the plain transpose): this is the only form for which `pmns.flavour_basis` applied to it reproduces $U_{\text{full}}\,\text{diag}(k)\,U^\dagger_{\text{full}}$ exactly, since $U_{\text{full}} = O\,U_{\text{red}}$ per (2.18).

Whenever U_{red} happens to be real for a given scenario, the Hermitian conjugate and the transpose coincide there numerically; the distinction only becomes observable once U_{red} is genuinely complex.

This is, in fact, exactly what happened in the original `peanuts` implementation [11]: since it only ever considered the plain 3-flavour Standard Model, its reduced mixing matrix was always real, so using the plain transpose there was correct, not merely convenient –

the two conventions were indistinguishable in that restricted setting.

It is only once `tpeanuts` generalises to a genuinely complex $U_{\rm red}$ (the 3+1 sterile extension with $\delta_{14} \neq 0$, Chapter 4) that the plain transpose silently becomes wrong, which is why the code always uses the Hermitian conjugate unconditionally rather than branching on whether $U_{\rm red}$ happens to be real.

2.6 `evolutor.py` (generic) — operator composition

`core/common/evolutor.py`

applying an evolutor to a state and composing ordered segment evolutors.

The concrete construction of a single segment's evolutor $S(x_{k+1}, x_k)$ — closed-form, perturbative, or fully numerical — is model-dependent and is given in the corresponding SM and BSM chapters (Chapters 3 to 6); this module only composes and applies whatever evolutors those chapters produce.

Theoretical Foundation

Composing segment evolutors

If the propagation path is split into n consecutive legs $x_0 \to x_1 \to \cdots \to x_n$, the composition property of the position-dependent Schrödinger equation gives

$$S(x_n, x_0) = S(x_n, x_{n-1}) \cdots S(x_2, x_1)\, S(x_1, x_0)\,, \tag{2.24}$$

i.e. the total evolution operator is the *ordered* product (most recent leg on the left) of the per-leg operators. This identity is what allows any continuous density profile to be treated as a sequence of locally solvable segments, each with its own closed-form or numerically integrated evolutor.

Applying the evolutor to a state

Once the total evolutor $S(x_n, x_0)$ is known, it is applied to an initial state as a plain matrix–vector product,

$$\psi(x_n) = S(x_n, x_0)\, \psi(x_0)\,, \tag{2.25}$$

implemented directly by `apply_evolutor_to_state`. What "the state" ψ represents, and how a transition probability is read off from the result, splits into two cases:

- **Coherent propagation.** $\psi(x_0)$ is a genuine flavour amplitude vector (a quantum superposition carrying relative phase, e.g. a single produced flavour, $\psi(x_0) = e_\alpha$). The transition amplitude is obtained by applying the formula above directly, and the transition probability to flavour β follows immediately as

$$P_\beta = |\psi_\beta(x_n)|^2 = \big|(S\,\psi(x_0))_\beta\big|^2. \tag{2.26}$$

- **Incoherent propagation.** The initial ensemble is a classical mixture of *mass* eigenstates with weights $w_i \geq 0$, $\sum_i w_i = 1$, carrying no relative phase between them (e.g. neutrinos that lost coherence over a very long baseline). There is no single amplitude vector $\psi(x_0)$ to apply S to; instead each mass eigenstate is propagated and projected onto the final flavour separately, and the resulting *probabilities* — not amplitudes — are summed with the classical weights,

$$P_\beta = \sum_i \big|(S\,U_{\rm PMNS})_{\beta i}\big|^2\, w_i. \tag{2.27}$$

Both formulas, together with the unitarity check that verifies them, are derived and implemented in full in the `probability.py` section below.

Implementation in tpeanuts

- `apply_evolutor_to_state(S, psi)`: applies a scalar or batched evolutor to a state vector, with automatic broadcasting.
- `compose_segment_evolutors(segments, *, segment_dim=-3, multiply="left")`: performs the ordered product above; internally uses a binary-tree reduction (`util.math.tree_reduce_matmul`) to minimise the depth of the matrix-multiplication chain on GPU. The default `multiply="left"` accumulates each new segment on the left ($U_{\text{total}} \to U_{\text{seg}}\, U_{\text{total}}$), reproducing the "most-recent-leg-on-the-left" convention above; `multiply="right"` accumulates in the opposite (left-to-right) order, for callers that need the reversed product directly instead of composing and reversing afterwards.

2.7 probability.py — from evolutors to observable probabilities

`core/common/probability.py`

the single transition-probability convention for the whole project.

Theoretical Foundation

Given the flavour evolutor S (row = final flavour, column = initial flavour),

$$P(\nu_\alpha \to \nu_\beta) = |S_{\beta\alpha}|^2. \tag{2.28}$$

- For a coherent initial state ψ_α (flavour superposition with physical phase), the final flavour probability is
$$P_\beta = \Big|\sum_\alpha S_{\beta\alpha}\psi_\alpha\Big|^2 = |(S\psi)_\beta|^2. \tag{2.29}$$
- For an incoherent mass mixture with weights $w_i \geq 0$, $\sum_i w_i = 1$ (no relative-phase information between mass eigenstates, the regime selected by `massbasis=True` throughout the code),
$$P_\beta = \sum_i |(S\,U_{\text{PMNS}})_{\beta i}|^2\, w_i. \tag{2.30}$$

Probability conservation: since S is unitary, $\sum_\beta P_{\beta\alpha} = 1$ for every column α; this is used as a numerical consistency check, not as an additional physics ingredient.

Implementation in tpeanuts

- `probability_transition(S, ...)`: the full probability matrix, or a selected channel (transition or survival).
- `probability_coherent_state(psi)` / `probability_coherent(S, psi)`: implement the coherent formula above.

- `probability_incoherent(...)`: implements the incoherent formula, either from a precomputed probability matrix or by projecting mass weights through S and U_{PMNS}.
- `probability_state(...)`: dispatches between both modes depending on the type of initial state received. Every medium's `[medium]_probability_state` (Chapters 7 to 10) delegates to this function.
- `normalize_probability_columns(P, eps=1e-15)`: rescales each column of a probability matrix to sum exactly to one, for callers that need a strictly conserved matrix after a numerically imperfect construction (e.g. a truncated spectral sum); `eps` guards the division against a vanishing column sum.
- `check_probability_conservation` / `check_probability_matrix`: numerical unitarity checks, used extensively in the tests (`core/BSM/test`, `core/common/test`).

Theoretical Foundation

Averaging a probability over a coordinate

A probability is dimensionless and normalised, so averaging it over any coordinate (energy, an arbitrary production variable, or the full zenith range) must *preserve* that normalisation: the natural reduction is a **weighted average**, not a plain integral. Given a weight w over a grid x (an explicit production spectrum for energy, an arbitrary caller-supplied weight for any other coordinate, or the geometric solid-angle element for angle),

$$\langle P \rangle = \frac{\int P(x)\, w(x)\, \mathrm{d}x}{\int w(x)\, \mathrm{d}x}, \tag{2.31}$$

evaluated by the trapezoidal rule, so $\langle P \rangle$ stays in $[0, 1]$ regardless of how w is normalised. This is the defining difference from `flux.py` below: a flux is not itself normalised, so its energy reduction is a plain (unnormalised) integral, not a weighted average. Like every other function in this module, all three reductions below accept a final flavour dimension of 3 or 4, the latter for the 3+1 sterile extension.

Implementation in `tpeanuts`

Three functions implement the weighted-average pattern above, one generic and two coordinate-specific:

Function	Weight w	Formula implemented
`probability_weighted_average(P, coordinate, weight, dim=-2)`	arbitrary, caller-supplied	$\langle P\rangle = \frac{\int P(x)\,w(x)\,dx}{\int w(x)\,dx}$ — the generic primitive, used e.g. for production-height averages (Chapter 10)
`probability_integrated(P, E_grid_MeV, spectrum, energy_dim=-2)`	production spectrum $w(E)$, **required** (no default)	$\langle P\rangle = \frac{\int P(E)\,w(E)\,dE}{\int w(E)\,dE}$
`probability_integrated_angular(P, theta_deg, angular_dim=-2)`	$w(\theta) = \sin\theta$, fixed and geometric	$\langle P\rangle = \frac{\int P(\theta)\sin\theta\,d\theta}{\int \sin\theta\,d\theta}$

`probability_integrated` has no default `spectrum`: a flat weight over an arbitrary energy grid is a modelling choice, not a safe default, so every caller must pass one explicitly (an explicit flat tensor if that is genuinely intended). `probability_integrated_angular` takes nothing in its place, since its weight is purely geometric. Both energy- and angle-specific functions are self-contained implementations (their own trapezoidal-rule code), not thin wrappers around `probability_weighted_average`; the latter exists as the shared pattern for coordinates — such as production height — that do not warrant a dedicated function of their own.

2.8 `flux.py` — from probabilities to fluxes

`core/common/flux.py`

multiplying final flavour probabilities by flux normalisations and optional spectra.

Theoretical Foundation

From probabilities to a differential flux

If $\Phi_\alpha^{\text{prod}}$ is the flux (or its normalised spectral shape) of the produced flavour α, the detected flux in flavour β is

$$\Phi_\beta^{\text{det}}(E) = \Phi_\alpha^{\text{norm}} \cdot f(E) \cdot P(\nu_\alpha \to \nu_\beta; E), \tag{2.32}$$

where $f(E)$ is an optional normalised spectrum. This module knows nothing about the physical origin of Φ^{prod} or the medium traversed: it is purely the final algebraic step, shared by `medium.vacuum.flux`, `medium.solar.flux`, `medium.earth.flux`, and `medium.atmosphere.flux`. When the full transition matrix $P_{\beta\alpha}$ is available rather than an already-selected final-flavour probability, the equivalent matrix form $\Phi_\beta^{\text{det}} = \sum_\alpha P_{\beta\alpha}\,\Phi_\alpha^{\text{prod}}$ is provided directly by `flux_transition` (see below), mirroring `probability_incoherent`'s plain-matrix branch (`probability.py` above).

From differential flux to event rate

$\Phi_\beta^{\text{det}}(E)$ above is a *differential* flux: a density in energy (and, when angularly resolved, in zenith/nadir angle), not yet a rate. Obtaining a neutrino **rate** — the quantity that, multiplied by an effective area and an exposure time, gives an expected event count —

requires integrating the energy dependence out:

$$\Phi_\beta^{\text{rate}} = \int \Phi_\beta^{\text{det}}(E)\, dE\,, \tag{2.33}$$

evaluated numerically by the trapezoidal rule over a finite energy grid. When Φ_β^{det} additionally carries an angular dependence (e.g. the atmospheric or Earth-crossing zenith angle), integrating only over energy keeps that angular resolution, giving a rate resolved by both angle and flavour. A further integration over the full zenith range is provided by this module too (`flux_integrated_angular` below), as the geometric counterpart to energy integration; unlike `probability.py` above, both reductions here are **unnormalised** totals – a flux is a physical rate density, not a probability, so nothing should be divided out. Every function in this module, like every function in `probability.py`, accepts a final flavour dimension of 3 or 4 (the 3+1 sterile extension).

Implementation in `tpeanuts`

- `flux_state(probability, flux, spectrum=None)`: multiplies and preserves the final-flavour dimension; if `spectrum` is given, it is applied as an element-wise multiplicative factor before reduction. Every medium's `[medium]_flux_state` (Chapters 7 to 10) delegates to this function.
- `flux_transition(probability_matrix, initial_flux)`: the matrix form $\Phi_\beta = \sum_\alpha P_{\beta\alpha}\Phi_\alpha$ described above, for callers that hold the full transition matrix rather than a final-flavour probability vector.

Three functions implement the unnormalised-integral pattern above, one generic and two coordinate-specific:

Function	Coordinate	Formula implemented
`flux_integrated_coordinate(flux, coordinate, dim=-2)`	arbitrary	$\Phi_{\text{out}} = \int \Phi(x)\, dx$ — the generic, unnormalised primitive
`flux_integrated(flux, E_grid_MeV, energy_dim=-2)`	energy E	$\Phi^{\text{rate}} = \int \Phi(E)\, dE$ — delegates directly to `flux_integrated_coordinate`
`flux_integrated_angular(flux, theta_deg, angular_dim=-2)`	zenith angle θ	$\Phi_{\text{total}} = 2\pi \int \frac{\mathrm{d}\Phi}{\mathrm{d}\Omega} \sin\theta\, \mathrm{d}\theta$ — its own implementation (extra 2π factor), not a wrapper

`flux_integrated`'s default `energy_dim=-2` matches a plain `(..., E, N)` flux ($N = 3$ or 4, the latter for the 3+1 sterile extension); an angle-resolved `(..., E, angle, N)` flux is integrated with `energy_dim=-3`, preserving the angle axis in the result. `flux_integrated_angular` assumes azimuthal symmetry and is evaluated by the trapezoidal rule with an explicit 2π azimuthal factor – unlike `probability_integrated_angular` (`probability.py` above), none of the three reductions here normalise: a flux is a physical rate density, not a probability, so nothing should be divided out.

2.9 neutrino.py — flavour index convention

core/common/neutrino.py

FLAVOUR_TO_INDEX dictionary, flavour_index validator, and flavour_state builder.

Implementation in tpeanuts

Fixes, once and for all, the flavour index convention used in every tensor across the project: $e \to 0$, $\mu \to 1$, $\tau \to 2$, $s \to 3$. The sterile index has no charged-lepton counterpart, but the 3+1 extension (PMNS_sterile, Chapter 4) fixes its position at index 3, so it is included under a small set of aliases ("s", "sterile", "nus", "nu_s") for callers that want to select or build it by name instead of hand-indexing a length-4 tensor. flavour_index accepts these text aliases and validates out-of-range integer indices, preventing silent indexing errors when building initial states; flavour_state returns the corresponding unit coherent-state vector, sized to n_flavours (3 by default, or 4 to select the sterile flavour).

Chapter 3

core.SM — Standard Model specialisation (3 flavours)

3.1 pmns.py — the Standard Model mixing matrix

core/SM/sm_pmns.py

PMNS_SM: the concrete 3-flavour implementation of the abstract class core.common.pmns.PMNS.

Theoretical Foundation

The Standard Model product structure

This is the only module that explicitly fixes a product structure for the generic factors defined in Chapter 2. The full 3-flavour PMNS matrix is

$$U = R_{23}(\theta_{23})\,\Delta(\delta)\,R_{13}(\theta_{13})\,\Delta(\delta)^\dagger\,R_{12}(\theta_{12}), \qquad \Delta(\delta) = \mathrm{diag}(1, 1, e^{i\delta}), \tag{3.1}$$

built from the generators R_{12}, R_{13}, R_{23}, Δ defined in Chapter 2. `PMNS_SM.outer_block` and `PMNS_SM.reduced` identify

$$O \equiv R_{23}(\theta_{23})\,\Delta(\delta), \qquad U_{\mathrm{red}} \equiv R_{13}(\theta_{13})\,\Delta(\delta)^\dagger\,R_{12}(\theta_{12}), \qquad U = O\,U_{\mathrm{red}}, \tag{3.2}$$

so that this single factorisation is the concrete instance, for the SM, of the abstract $U = O\,U_{\mathrm{red}}$ contract from Chapter 2.

Why the outer block commutes with the matter Hamiltonian

O does not mix the electron-flavour index: R_{23} acts only on the (μ, τ) block and Δ is diagonal, so O has the block form $O = \left(\begin{smallmatrix} 1 & 0 \\ 0 & O_{2\times 2} \end{smallmatrix}\right)$ in the flavour basis. Since the matter potential only has a non-zero ee entry, $H_{\mathrm{mat}} = \mathrm{diag}(V, 0, 0)$, it follows that

$$O\,H_{\mathrm{mat}}\,O^\dagger = H_{\mathrm{mat}} \qquad \text{(they commute exactly)}. \tag{3.3}$$

This is precisely the property Chapter 2 assumes, without proof, to derive the general factorisation Eq. (2.18); here it is verified explicitly for the concrete SM O.

Transforming the Hamiltonian to the reduced basis

Given the flavour-basis Hamiltonian $H_{\mathrm{flavour}} = H_{\mathrm{kin}} + H_{\mathrm{mat}} = U\,\mathrm{diag}(k_1, k_2, k_3)\,U^\dagger + H_{\mathrm{mat}}$ (`hamiltonian.py` section, Chapter 2), substituting $U = O\,U_{\mathrm{red}}$ and using $O\,H_{\mathrm{mat}}\,O^\dagger = H_{\mathrm{mat}}$ gives

$$H_{\mathrm{flavour}} = O\,\underbrace{\left[U_{\mathrm{red}}\,\mathrm{diag}(k)\,U_{\mathrm{red}}^\dagger + H_{\mathrm{mat}}\right]}_{H_{\mathrm{red}}}\,O^\dagger = O\,H_{\mathrm{red}}\,O^\dagger, \tag{3.4}$$

matching Eq. (2.18): `PMNS_SM.reduced_basis` applied to H_{flavour} (inherited from `PMNS`, computing $O^\dagger(\cdot)O$) yields exactly H_{red} above.

The CP phase drops out of the reduced kinetic term

R_{12} does not mix the τ index (third index) and Δ acts non-trivially only on that same index, so the product $R_{12}\,\mathrm{diag}(k)R_{12}^T$ has no non-zero entries connecting to index 3; conjugation by Δ therefore acts trivially on it:

$$\Delta^\dagger(R_{12}\,\mathrm{diag}(k)R_{12}^T)\Delta = R_{12}\,\mathrm{diag}(k)R_{12}^T, \tag{3.5}$$

so that **the CP phase δ drops out completely from the reduced kinetic term**, which reduces to the real matrix $U_{\mathrm{red}}^{(0)} \equiv R_{13}R_{12}$:

$$U_{\mathrm{red}}\,\mathrm{diag}(k)\,U_{\mathrm{red}}^\dagger = (R_{13}R_{12})\;\mathrm{diag}(k)\;(R_{13}R_{12})^T. \tag{3.6}$$

This is why `PMNS_SM.reduced` returns $R_{13}R_{12}$ rather than the formally-defined $R_{13}\Delta^\dagger R_{12}$: the two coincide once contracted with the (real, diagonal) kinetic term, but $R_{13}R_{12}$ is real, cheaper to evolve, and avoids carrying a phase that always cancels.

Recovering the flavour basis

Once H_{red} (and its evolutor S_{red}) has been obtained, the flavour-basis result is recovered by dressing once with O (`PMNS_SM.flavour_basis`, inherited from `PMNS`, computing $O(\cdot)O^\dagger$):

$$H_{\mathrm{flavour}} = O\,H_{\mathrm{red}}\,O^\dagger, \qquad S_{\mathrm{flavour}}(x) = O\,S_{\mathrm{red}}(x)\,O^\dagger. \tag{3.7}$$

Coherent propagation scheme. In vacuum ($H_{\mathrm{mat}} = 0$), a coherent flavour state is most simply understood, and computed, entirely in the mass basis:

1. **Flavour → mass**: project onto mass eigenstates, $\psi_{\mathrm{mass}}(x_0) = U^\dagger\,\psi_{\mathrm{flavour}}(x_0)$;
2. **Coherent propagation**: each mass eigenstate accumulates its own phase independently, since the mass basis diagonalises the kinetic term by construction, $\psi_{\mathrm{mass}}(x) = \mathrm{diag}(e^{-ik_1x},\dots,e^{-ik_3x})\,\psi_{\mathrm{mass}}(x_0)$;
3. **Mass → flavour**: project back, $\psi_{\mathrm{flavour}}(x) = U\,\psi_{\mathrm{mass}}(x)$.

Composing the three steps gives the familiar vacuum evolutor $S_{\mathrm{vacuum}}(x) = U\,\mathrm{diag}(e^{-ik_ix})U^\dagger$ (`medium.vacuum.evolutor`). In matter, H_{mat} does not commute with the full U, so this three-step picture does not apply directly at the full-U level; the reduction trick derived above is exactly what restores an analogous, computationally cheap three-step scheme – flavour → reduced → propagate → flavour – using O and U_{red} in place of U.

Implementation in `tpeanuts`

`PMNS_SM(params: PMNSParams)` implements only the three abstract methods that depend on the product structure: `pmns_matrix` (the full 3×3 U), `reduced` ($U_{\mathrm{red}} = R_{13}R_{12}$), and `outer_block` ($O = R_{23}\Delta$). Everything else (rotation builders, `flavour_basis`/`reduced_basis`, antineutrino selection via `select_antinu`) is inherited unchanged from the base class, without duplicating code. It is the default scenario for the whole project: any function in `medium.*` that receives a generic `PMNS` object works unmodified with either `PMNS_SM` or `PMNS_sterile` (Chapter 4), thanks to the inheritance design of Chapter 2.

Key References

Standard PMNS parametrisation: [21]. Reference numerical values (global best fits) used in the default presets: [7, 19].

3.2 mass_spectrum.py — Standard Model mass spectrum

core/SM/sm_mass_spectrum.py

MassSpectrum_SM, the concrete 3-flavour implementation of MassSpectrum (Chapter 2).

Theoretical Foundation

The Standard Model scenario has no sterile state to accommodate, so difference_vector() simply returns difference_vector_base() unchanged, sized to exactly 3 components:

$$(\Delta m^2_1,\ \Delta m^2_2,\ \Delta m^2_3) = \begin{cases} (0,\ \Delta m^2_{21},\ \Delta m^2_{3\ell}) & \Delta m^2_{3\ell} > 0 \text{ (NO)}, \\ (-\Delta m^2_{21},\ 0,\ \Delta m^2_{3\ell}) & \Delta m^2_{3\ell} < 0 \text{ (IO)}. \end{cases} \tag{3.8}$$

The ordering-dependent construction itself – selecting between the two branches above from the sign of $\Delta m^2_{3\ell}$ – is entirely inherited from MassSpectrum (Chapter 2, mass_spectrum.py section): MassSpectrum_SM adds no logic of its own beyond fixing the output size to 3, in contrast with MassSpectrum_BSM (Chapter 4), which appends a fourth, sterile entry Δm^2_{41} to the same base vector.

Implementation in tpeanuts

MassSpectrum_SM(DeltamSq21, DeltamSq31): adds no fields beyond the base MassSpectrum; difference_vector() is a single-line override, return self.difference_vector_base(context=context).

Chapter 4

`core.BSM` — Beyond-the-Standard-Model extensions

This chapter covers the two supported BSM extensions: non-standard interactions (NSI) in the 3-flavour sector, and a fourth light sterile neutrino (the "3+1" scheme). Both reuse as much as possible of the infrastructure from Chapter 2: NSI only modifies H_{mat}, reusing whichever `PMNS` object is otherwise in use, and the sterile sector extends the factorisation $U = O\,U_{\text{red}}$ to 4×4. There is no separate `core.BSM` Hamiltonian-assembly module: `hamiltonian_reduced` and `hamiltonian_flavour` (Chapter 2, `hamiltonian.py` section) already handle NSI, the sterile extension, or both simultaneously.

4.1 `bsm_nsi.py` — non-standard interactions

`core/BSM/bsm_nsi.py`

parametric configuration of NSI couplings and construction of the ϵ matrix.

Theoretical Foundation

NSI are effective four-fermion operators that modify coherent forward scattering beyond the standard MSW potential [12]. The matter Hamiltonian in the flavour basis generalises to

$$H_{\text{mat}}^{\text{NSI}} = V \left(\begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} + \epsilon \right), \qquad V = \pm\sqrt{2}\, G_F\, n_e\, L_{\text{scale}}, \tag{4.1}$$

with ϵ a dimensionless 3×3 Hermitian matrix,

$$\epsilon = \begin{pmatrix} \epsilon_{ee} & \epsilon_{e\mu} & \epsilon_{e\tau} \\ \epsilon_{e\mu}^{*} & \epsilon_{\mu\mu} & \epsilon_{\mu\tau} \\ \epsilon_{e\tau}^{*} & \epsilon_{\mu\tau}^{*} & \epsilon_{\tau\tau} \end{pmatrix}. \tag{4.2}$$

Diagonal entries are real; off-diagonal entries are generally complex. The Standard Model limit is $\epsilon = 0$. Only the traceless part of ϵ matters for oscillations (an overall trace is an irrelevant global phase), so fixing $\epsilon_{\mu\mu} = 0$ is a valid convention for the diagonal sector without loss of physical generality.

No NSI-specific mixing matrix. `NSIConfig` touches only H_{mat}: it does not define, or require, any dedicated `PMNS` subclass. If the sterile extension is not also active, NSI propagation reuses the plain 3-flavour `PMNS_SM` object from Chapter 3 completely unchanged – the same U, O, U_{red} as the pure Standard Model case. `OscillationParameters.nsi` is simply an optional, orthogonal attribute alongside `pmns` (Chapter 2, `oscillation.py` section): which mixing scenario is active and whether NSI is active are two independent choices, and every combination (SM+NSI, sterile alone, sterile+NSI) is valid.

Theoretical Foundation

Transforming the matter Hamiltonian: Standard Model versus NSI

In the plain Standard Model, the reduced-basis matter term needs no rotation at all: $H^{\text{red}}_{\text{mat}} = H^{\text{flavour}}_{\text{mat}} = \text{diag}(V, 0, \ldots, 0)$, because $O\, H_{\text{mat}}\, O^\dagger = H_{\text{mat}}$ exactly (Chapter 3) – the matter term is identical in the flavour and reduced bases, so it is simply added to the reduced kinetic term as-is.
When NSI are active this shortcut disappears: as shown below, $O\, H^{\text{NSI}}_{\text{mat}}\, O^\dagger \neq H^{\text{NSI}}_{\text{mat}}$ in general. `hamiltonian_matter_reduced` (Chapter 2) therefore takes the opposite route: it builds $H^{\text{NSI}}_{\text{mat}}$ directly in the *flavour* basis as above, and then rotates it explicitly *down* into the reduced basis,

$$H^{\text{NSI, red}}_{\text{mat}} = O^\dagger\, H^{\text{NSI, flavour}}_{\text{mat}}\, O\,, \tag{4.3}$$

the exact inverse of the flavour-basis dressing $O(\cdot)O^\dagger$ (Chapter 2). Because the transformation is performed explicitly here – rather than assumed to be a no-op, as in the SM case above – the identity $H_{\text{flavour}} = \texttt{pmns.flavour\_basis}(H_{\text{red}})$ holds exactly for *arbitrary* ϵ, not only for the diagonal-uniform Standard Model limit.

Approximations and Limitations

Adjoint, not transpose. O is generally complex (Δ carries $e^{i\delta}$), so $O^\dagger = (O^*)^T$ and the plain transpose O^T differ whenever $\delta \neq 0$. The rotation above **must** use the conjugate transpose (`O.conj().transpose(-2,-1)` in `hamiltonian_matter_reduced`) to preserve Hermiticity: for Hermitian ϵ (and hence Hermitian $H^{\text{NSI}}_{\text{mat}}$), $O^\dagger\, H\, O$ is again Hermitian for any unitary O, whereas $O^T\, H\, O$ is not, in general, even Hermitian – let alone the physically correct transformed operator. This mirrors the identical requirement already established for the kinetic term (Chapter 2, `pmns.py/hamiltonian.py` sections): the Hermitian conjugate is the only convention consistent with a unitary change of basis for an operator; the plain transpose coincides with it only by accident, when the matrix involved happens to be real.

Approximations and Limitations

Since $H^{\text{NSI}}_{\text{mat}}$ is no longer proportional to $\text{diag}(V, 0, 0)$, the commutation property $O\, H_{\text{mat}}\, O^\dagger = H_{\text{mat}}$ assumed by the general factorisation Eq. (2.18) **no longer holds in general** once ϵ has entries outside the ee block. The code does not rely on it being true – as shown above, `hamiltonian_matter_reduced` builds $H^{\text{NSI}}_{\text{mat}}$ in the flavour basis and rotates it explicitly, so the result is exact for any ϵ. What *is* lost is the Standard Model *simplification* (Chapter 3) that H_{red} needs no reference to θ_{23}/δ and can be built with a real U_{red}: for generic NSI, H_{red} is genuinely complex and depends on every mixing parameter, so the closed-form spectral formulas used elsewhere in the project no longer apply, and the relevant diagonalisation (e.g. in `medium.solar.probability`) is performed numerically instead, via `torch.linalg.eigh`.

Implementation in `tpeanuts`

- `NSIConfig` (frozen dataclass): stores the six independent parameters of ϵ (3 real diagonal + 3 complex off-diagonal) and exposes `epsilon_tensor_base()` (the raw complex 3×3 tensor), `epsilon_tensor(n_flavours=...)` (embedded for any

flavour count, zero-padded for the sterile rows/columns), and `is_sm_limit`.

- `NSIConfig.from_preset(name)` builds named configurations from `tpeanuts.config.presets.NSI_PRESETS` (Section 4.2), each with its physical justification documented.
- The ϵ tensor is read via `oscillation.nsi.epsilon_tensor(...)`, passed through `pmns.select_antinu` ($\epsilon \to \epsilon^*$ for antineutrinos, mirroring the $U \to U^*$ convention), and injected directly into `hamiltonian_matter_reduced` (Chapter 2) – the single Hamiltonian-assembly function handling this case; there is no separate `core.BSM` builder.

Key References

Original NSI proposal: [12]. Model-independent bounds: [1]. LMA-Dark degeneracy in the global fit: [6]. IceCube DeepCore bounds: [15].

4.2 NSI presets

`tpeanuts/config/presets.py`

`NSI_PRESETS`: named, literature-justified ϵ configurations, built via `NSIConfig.from_preset`.

Implementation in `tpeanuts`

Every preset below sets a handful of ϵ parameters to a representative, literature-motivated value (all other components default to zero) rather than to a unique global best fit: current data leaves large NSI degeneracies, so no single preferred point exists. Their purpose is threefold: (1) give the validation notebooks and tests physically motivated ϵ values instead of arbitrary numbers, covering diagonal-only, off-diagonal, single- and multi-parameter regimes; (2) reproduce, and stay traceable to, specific published sensitivity/bound studies; and (3) let users reach a standard benchmark scenario (e.g. the LMA-Dark degeneracy) by name instead of re-deriving its ϵ values.

Preset	ϵ values and origin
`sm_no_nsi`	All $\epsilon = 0$. Standard Model limit; equivalent to passing `nsi=None`.
`nsi_diagonal_biggio2009`	$\epsilon_{ee} = 0.30$, $\epsilon_{\tau\tau} = 0.15$. Diagonal-only benchmark within the model-independent 90% CL bounds of Biggio, Blennow, and Fernandez-Martinez [1] ($\lvert\epsilon_{ee}\rvert < 0.50$, $\lvert\epsilon_{\tau\tau}\rvert < 0.33$).
`nsi_lma_dark_esteban2018`	$\epsilon_{ee} = -2.0$. The LMA-Dark degenerate solar solution of Esteban et al. [6]: this large negative ϵ_{ee} flips the sign of the effective MSW potential, mimicking $\theta_{12} \to \pi/2 - \theta_{12} \approx 56.6°$ (paired with the `"_LMA_DARK_NUFIT52_NO"` oscillation preset, Chapter 2).
`nsi_offdiag_icecube2022`	$\epsilon_{\mu\tau} = 0.005$, $\epsilon_{\tau\tau} = 0.015$. Sits near the 90% CL boundary of the IceCube DeepCore atmospheric analysis [15] ($\lvert\epsilon_{\mu\tau}\rvert < 0.0060$, $\lvert\epsilon_{\tau\tau} - \epsilon_{\mu\mu}\rvert < 0.018$).
`nsi_dune_etau`	$\epsilon_{e\tau} = 0.15$ (real). Representative DUNE Near Detector sensitivity benchmark [3]: this flavour-changing coupling produces the strongest degeneracies with δ_{CP}, the mass ordering, and θ_{23}.
`nsi_hyperk_etau`	$\epsilon_{ee} = 0.20$, $\epsilon_{e\tau} = 0.10$ (real). Combines a diagonal MSW correction with a flavour-changing coupling, representative of Hyper-Kamiokande sensitivity studies [13].
`nsi_globalfit_esteban2018`	$\epsilon_{ee} = 0.25$, $\epsilon_{e\tau} = 0.12$, $\epsilon_{\mu\tau} = 0.01$, $\epsilon_{\tau\tau} = 0.08$. Multi-parameter benchmark within the allowed region of the global oscillation analysis of Esteban et al. [6]; not a unique best fit, since significant correlations remain.
`nsi_ee_only`	$\epsilon_{ee} = 0.30$. Simplest possible NSI scenario: only the effective MSW potential is modified, with no flavour-changing coupling [1].
`nsi_mutau_only`	$\epsilon_{\mu\tau} = 0.01$ (real). Atmospheric μ–τ benchmark representative of IceCube/KM3NeT sensitivity to flavour-changing matter interactions [2].

Key References

[1–3, 6, 13, 15].

4.3 bsm_sterile.py — the 3+1 scheme

core/BSM/bsm_sterile.py

PMNS_sterile: *4×4 mixing for one additional light sterile neutrino.*

Theoretical Foundation

A fourth mass eigenstate ν_4 and a fourth sterile flavour eigenstate ν_s are added; ν_s is a gauge singlet and therefore couples to neither W nor Z. The charged-current potential is consequently unaffected by the extension, so, omitting the optional neutral-current term (the default, Chapter 2), the matter potential in the extended basis (e, μ, τ, s) is

$$H_{\text{mat}} = \text{diag}(V_{CC}, 0, 0, 0), \tag{4.4}$$

the same charged-current-only term as the 3-flavour case, simply padded with a zero sterile entry. Since ν_s also does not receive the neutral-current potential V_{NC}, optionally supplying a neutron density activates the relative term derived in Chapter 2,

$$H_{\text{mat}} = \text{diag}(V_{CC}, 0, 0, -V_{NC}), \tag{4.5}$$

which does affect sterile-active oscillation probabilities (see the implementation box below for how this interacts with O_4). The 4×4 mixing matrix is parametrised by adding three active–sterile rotations, R_{14}, R_{24}, R_{34}, to the standard structure, following the parametrisation convention of Kopp et al. [16]:

$$U_4 = R_{23}\,\Delta_4\,R_{24}\,R_{34}\,R_{13}\,\Delta_4^\dagger\,R_{12}\,R_{14}. \tag{4.6}$$

Identifying $O_4 \equiv R_{23}\,\Delta_4\,R_{24}\,R_{34}$ (`PMNS_sterile.outer_block`) leaves a formal reduced factor $U_{\text{red},4} \equiv R_{13}\,\Delta_4^\dagger\,R_{12}\,R_{14}$ satisfying $U_4 = O_4\,U_{\text{red},4}$ exactly, mirroring Chapter 3. As in the SM case, Δ_4 cancels from the reduced *kinetic term* specifically:

$$\Delta_4^\dagger(R_{12}R_{14}\,\text{diag}(k)R_{14}^\dagger R_{12}^\dagger)\Delta_4 = R_{12}R_{14}\,\text{diag}(k)R_{14}^\dagger R_{12}^\dagger\,, \tag{4.7}$$

valid because neither R_{12} (the e–μ plane) nor R_{14} (the e–s plane) mixes with the τ index, which is the only non-trivial index of Δ_4. This is why `PMNS_sterile.reduced` returns the simplified, Δ_4-free

$$U_{\text{red},4}^{\text{code}} = R_{13}R_{12}R_{14} \tag{4.8}$$

rather than the formal $U_{\text{red},4}$ above: the two agree once contracted with the kinetic term (any diagonal unitary conjugating a diagonal matrix leaves it invariant), but $U_{\text{red},4}^{\text{code}}$ is cheaper. Note that, consequently, $O_4\,U_{\text{red},4}^{\text{code}} \neq U_4$ as bare matrices – only $U_{\text{red},4}$ (with $\Delta_4^\dagger$ included) satisfies the exact factorisation; `PMNS_sterile.reduced` returns the code-optimised form, valid specifically for building H_{kin}, not a literal factor of U_4 (see the limitations box below). The same commutation argument as Chapter 3 applies to O_4 for the plain charged-current matter term: $R_{23}, \Delta_4, R_{24}, R_{34}$ all leave the e index fixed, so $O_4\ \text{diag}(V_{CC}, 0, 0, 0)\,O_4^\dagger = \text{diag}(V_{CC}, 0, 0, 0)$ exactly, and the reduced Hamiltonian is assembled and dressed back to the flavour basis exactly as in Chapters 2 and 3, now for $n = 4$. As established in Chapter 2 (`hamiltonian.py` section), this commutation is specific to the charged-current-only term: O_4 mixes the $\{\mu, \tau, s\}$ block and therefore does *not* leave the optional neutral-current term $\text{diag}(0, 0, 0, V_{NC})$ invariant, so the reduction-trick shortcut used above breaks whenever that term is active (see the implementation box below).

SM limit. When $\theta_{14} = \theta_{24} = \theta_{34} = 0$, the 4×4 matrices block-diagonalise exactly into the 3×3 SM matrix *embedded* in the larger space, recovering every result of Chapters 2 and 3 without approximation.

CP-phase counting. For $N = 4$ Dirac generations there are $(N-1)(N-2)/2 = 3$ physical phases: δ_{13} (the standard phase, in the Δ_4 carried through R_{13}), δ_{14} (e–s sector, in R_{14}), and δ_{24} (μ–s sector, in R_{24}). R_{34} is always real: the fourth possible phase can be absorbed into a field redefinition, so sterile presets do not include `delta34_deg`.

Approximations and Limitations

Counting CP phases for the sterile extension requires care: an N-generation Dirac mixing matrix has $(N-1)(N-2)/2$ independent physical phases. For $N = 4$ (the 3+1 case) this

gives exactly three Dirac phases (δ_{13}, δ_{14}, δ_{24}); the rotation R_{34} is always real because the fourth phase can be absorbed into a redefinition of the charged-lepton fields, and hence sterile presets do not include `delta34_deg`.

Theoretical Foundation

Transforming to the reduced basis: H_{kin} and H_{mat}

Exactly as in Chapter 3, the reduced-basis kinetic term uses the code's $U^{\text{code}}_{\text{red},4} = R_{13}R_{12}R_{14}$ together with its *Hermitian conjugate* (never the plain transpose, for the same reason as Chapters 2 and 3):

$$H^{\text{red}}_{\text{kin},4} = U^{\text{code}}_{\text{red},4}\ \text{diag}(k_1, k_2, k_3, k_4)\ \left(U^{\text{code}}_{\text{red},4}\right)^{\dagger}. \tag{4.9}$$

Unlike the SM's $R_{13}R_{12}$, $U^{\text{code}}_{\text{red},4}$ is genuinely complex whenever the active-sterile CP phase $\delta_{14} \neq 0$ (it carries R_{14}, which – unlike R_{12}/R_{13} – is built with a non-trivial phase); the Hermitian-conjugate convention is therefore not merely a safe default here, but the only correct choice.

Depending on whether NSI and/or the neutral-current term are active, the matter term in the reduced basis takes one of three forms:

- **Without NSI and without the neutral-current term** (the default). The matter term needs no basis change at all: $O_4\ \text{diag}(V_{CC}, 0, 0, 0)\, O_4^{\dagger} = \text{diag}(V_{CC}, 0, 0, 0)$ exactly (shown above), so

$$H^{\text{red}}_{\text{mat},4} = H^{\text{flavour}}_{\text{mat},4} = \text{diag}(V_{CC}, 0, 0, 0), \tag{4.10}$$

  and the reduced Hamiltonian is simply $H_{\text{red},4} = H^{\text{red}}_{\text{kin},4} + H^{\text{red}}_{\text{mat},4}$ (`hamiltonian_reduced`, Chapter 2), dressed back to the flavour basis with $O_4(\cdot)O_4^{\dagger}$ (`hamiltonian_flavour`). This case introduces no approximation beyond Chapter 3: the reduced-basis simplification stays exact for any $\theta_{14}, \theta_{24}, \theta_{34}$.

- **Without NSI but with the neutral-current term.** Supplying a neutron density activates $H^{\text{flavour}}_{\text{mat},4} = \text{diag}(V_{CC}, 0, 0, -V_{NC})$ (Chapter 2) instead: since this no longer commutes with O_4, the shortcut above is unavailable and `hamiltonian_matter_reduced` takes the general rotated path,

$$H^{\text{red}}_{\text{mat},4} = O_4^{\dagger}\, H^{\text{flavour}}_{\text{mat},4}\, O_4, \tag{4.11}$$

  even though NSI is off.

- **Simultaneously with NSI (and, optionally, the neutral-current term).** Fully supported by the same code path (Section 4.2): `NSIConfig.epsilon_tensor(n_flavours=4)` embeds the 3×3 ϵ block into the top-left corner of a 4×4 zero matrix (the sterile row/column stay zero, since ν_s is an $\text{SU}(2)_L \times \text{U}(1)_Y$ singlet and carries no NSI coupling), and `hamiltonian_matter_reduced` rotates the resulting $H^{\text{NSI, flavour}}_{\text{mat},4} = V\left(\text{diag}(1,0,0,0) + \epsilon_4\right)$ (plus the $-V_{NC}$ sterile-diagonal term when a neutron density is also supplied) into the reduced basis with the *same* $O_4^{\dagger}(\cdot)O_4$ transformation:

$$H^{\text{NSI, red}}_{\text{mat},4} = O_4^{\dagger}\, H^{\text{NSI, flavour}}_{\text{mat},4}\, O_4, \tag{4.12}$$

again exact for arbitrary ϵ, since it is computed explicitly rather than assumed to commute with O_4 – the same approximation trade-off as the 3-flavour NSI case applies here too (previous sections): the closed-form real-arithmetic shortcut is lost, and diagonalisation falls back to `torch.linalg.eigh`.

Nothing in `hamiltonian_reduced`/`hamiltonian_flavour` branches explicitly on "sterile or NSI or both": the flavour count $n \in \{3, 4\}$ and the presence of `oscillation.nsi` are read directly off `OscillationParameters`, so all four combinations (SM, SM+NSI, sterile, sterile+NSI) are handled by the exact same two functions.

Approximations and Limitations

Limitation: $U = O\,U_{\text{red}}$ holds exactly only for the *formal* U_{red}, not for `reduced`. Both here and in Chapter 3, the abstract contract of Chapter 2 – $U = O\,U_{\text{red}}$ for the three methods `pmns_matrix`, `outer_block`, `reduced` – is, strictly, a simplification of what the code returns: `reduced` always drops a Δ (or Δ_4) factor that is provably irrelevant for building H_{kin} (any diagonal unitary conjugating $\text{diag}(k)$ leaves it invariant) but whose omission means `outer_block()·reduced()` is *not* bit-for-bit equal to `pmns_matrix()`. This is harmless for every use of `reduced` in this project (always as the sandwiching matrix of a kinetic term), but it means Chapter 2's statement of the contract should be read as "U and $O\,U_{\text{red}}$ agree up to a diagonal phase that leaves $\text{diag}(k)$ invariant", not as a literal matrix identity between the three named methods.

Implementation in `tpeanuts`

- `PMNSSterileParams`: parameters for *only* the sterile extension ($\theta_{14}, \theta_{24}, \theta_{34}$ and their phases); the SM sector lives in a separate `PMNSParams` object passed alongside this one to the `PMNS_sterile` constructor. Δm^2_{41} also does not live here: like $\Delta m^2_{21}/\Delta m^2_{3\ell}$, it is stored on `OscillationParameters` because it never enters a mixing rotation.
- `PMNS_sterile`: subclass of `core.common.pmns.PMNS` with `n_flavours=4`; it inherits unchanged every piece that is generic in size (`R12`, `R13`, `R23`, `Delta`, `flavour_basis`, `reduced_basis`, `refresh`, `select_antinu`, ...) and only adds the extra constructors `R14`, `R24`, `R34` and the concrete implementation of `pmns_matrix`, `reduced`, and `outer_block`.
- Direct consequence of this design: **every** piece of `medium.*` code that operates on a generic `PMNS` object works without modification for 4 flavours, simply by passing a `PMNS_sterile` instance instead of `PMNS_SM`; the same holds for NSI (previous sections), which can be combined freely with the sterile extension since `NSIConfig.epsilon_tensor` embeds the 3×3 ϵ block in the top-left corner of the 4×4 matter matrix, leaving the sterile row/column at zero.
- Named presets (e.g. `"sterile_3p1_bestfit_giunti2017"`) are built via `PropagationConfig.oscillation_parameters_from_preset` (Chapter 2), which internally creates both parameter objects and sets Δm^2_{41} on the resulting `OscillationParameters`.

Key References

3+1 mixing parametrisation convention: [16]. Global 3+1 fit: [9]. IceCube atmospheric ν_μ disappearance search: [14].

4.4 Sterile presets

`tpeanuts/config/presets.py`

`OSCILLATION_PRESETS` entries carrying `theta14_deg`: named, literature-justified 3+1 parameter sets, dispatched to `PMNS_sterile` automatically.

Implementation in `tpeanuts`

A preset is a *sterile* preset purely by having a `theta14_deg` key; `PropagationConfig.oscillation_parameters_from_preset` (Chapter 2) dispatches on that key alone to build a `PMNS_sterile` instead of a `PMNS_SM`, with every other field (the SM angles, $\Delta m^2_{21}/\Delta m^2_{3\ell}$) inherited from the shared NuFIT 5.2 baseline (Chapter 2, Default Configuration box). As with the NSI presets (Section 4.2), these are named, traceable benchmark points rather than a single global best fit – current 3+1 data is in tension between different anomalies, so no universally preferred point exists.

Preset	Parameters and origin
`sterile_3p1_null_mixing`	$\theta_{14} = \theta_{24} = \theta_{34} = 0$, $\Delta m^2_{41} = 1.0$ eV^2. Null-hypothesis reference: with every sterile angle at zero, `PMNS_sterile` is numerically identical to the plain 3-flavour SM (the exact SM-limit embedding proved above), run through the 4-flavour machinery – used to validate that embedding in `core/BSM/test/test3_bsm_sterile.py`.
`sterile_3p1_bestfit_giunti2017`	$\theta_{14} = 8.5°$, $\theta_{24} = 7.5°$, $\theta_{34} = 0$, $\Delta m^2_{41} = 1.7$ eV^2 ($\sin^2 2\theta_{14} = 0.085$, $\sin^2 2\theta_{24} = 0.068$). Global 3+1 fit combining the reactor and gallium anomalies with solar data [9]; sterile CP phases fixed to zero (no sensitivity in that data combination).
`sterile_3p1_benchmark_icecube`	$\theta_{14} = 0$, $\theta_{24} \approx 9.22°$ ($\sin^2 2\theta_{24} = 0.10$), $\theta_{34} = 0$, $\Delta m^2_{41} = 0.3$ eV^2. IceCube atmospheric ν_μ disappearance benchmark [14]; θ_{14}, θ_{34} set to zero since IceCube atmospheric neutrinos are primarily sensitive to θ_{24}.
`sterile_3p1_two_flavour_limit`	$\theta_{12} = \theta_{13} = \theta_{23} = 0$, $\theta_{14} = 15°$, $\theta_{24} = 12°$, $\theta_{34} = 0$, $\Delta m^2_{41} = 1.0$ eV^2. A mathematical construction, not a data-driven benchmark: switching off every SM active angle makes the ν_e disappearance channel reduce exactly to the textbook 2-flavour formula $P_{ee} = 1 - \sin^2(2\theta_{14}) \sin^2(1.267\,\Delta m^2_{41} L/E)$ for any θ_{24}, since θ_{24} lives only in O_4, which leaves the electron index untouched. Used to validate the 2-flavour approximation against the full 4-flavour evolutor in isolation, before quantifying the residual active-sector suppression present for realistic presets such as `sterile_3p1_bestfit_giunti2017` (see `sterile2_kinematics.ipynb` in `notebooks/validation/physics/BSM`).

Approximations and Limitations

Approximation exposed by `sterile_3p1_two_flavour_limit`. The familiar 2-flavour disappearance formula is only *exact* once the SM active angles are switched off; for realistic presets (SM angles at their measured values) the ν_μ channel in particular picks up a small residual correction, because R_{14} and R_{24} both act on the sterile index and do not commute in general – $|U_{\mu 4}|^2 = \sin^2\theta_{24}$ holds exactly only when $\theta_{23} = \theta_{14} = 0$. This is a property of the physics (the standard 2-flavour formula is itself an approximation once more than one active-sterile angle is non-zero), not a code limitation; the preset exists specifically to isolate and validate this effect numerically.

4.5 Supported configurations across the codebase

This chapter and Chapter 3 establish the three orthogonal oscillation scenarios exposed by `OscillationParameters`: the plain Standard Model, the Non-Standard Interactions extension, and the 3+1 sterile extension — freely combinable, in any pair or all three at once, via `oscillation.pmns.n_flavours` and `oscillation.nsi` (Chapter 2). Which of these combinations a given *propagation method* actually accepts, however, is not uniform: `hamiltonian_reduced` / `hamiltonian_flavour` themselves, the generic numerical evolutor, and most perturbative and solar paths are fully generic in flavour count and NSI content, while a couple of closed-form solar methods are deliberately restricted to the plain SM because their underlying formula has no BSM generalisation. This section collects that support matrix in one place, so a caller can check at a glance whether a chosen propagation method accepts the oscillation scenario it needs.

Configuration option	Standard Model	NSI	Sterile (3+1)
Numerical propagation (`evolutor_numerical`)	✓ Supported	✓ Supported	✓ Supported
Perturbative propagation — vacuum (`medium.vacuum.evolutor`)	✓ own exact closed form	✓ (no physical effect: $H_{\text{mat}} = 0$ in vacuum)	✓ same closed form, generic in n
Perturbative propagation — earth (`medium.earth.evolutor`)	✓	✓	✓
Perturbative propagation — atmosphere (`medium.atmosphere.evolutor`)	✓	✓	✓
Solar — adiabatic approximated	✓	× raises `ValueError`	× raises `ValueError`
Solar — adiabatic exact	✓	✓	✓ (+ optional V_{NC})
Solar — fully numerical	✓	✓	✓ (+ optional V_{NC})
Solar — Landau-Zener correction	✓ (adiabatic approximated)	×	×
Eigenvalue solver, analytic	Cardano ($n = 3$)	Cardano ($n = 3$) or Ferrari ($n = 4$)	Ferrari ($n = 4$)
Neutral-current term V_{NC} (sterile diagonal)	n/a (unobservable common phase, never computed)	n/a unless sterile also active	✓ optional (off by default)

Reading This Table

- **A ✓ is scenario-exact, not error-free.** Every ✓ entry above is exact within whatever approximation the method itself represents (the adiabatic approximation, a finite numerical step size, . . .) — it is not a claim that the method is free of the usual numerical or physical approximations documented in its own chapter.

- **Vacuum never touches Chapter 6 or Chapter 5.** With $H_{\text{mat}} = 0$ there is nothing to perturb or exponentiate segment by segment, so `medium.vacuum.evolutor` implements the exact three-step mass-basis scheme directly (Chapter 3, the **Coherent propagation scheme** box). That scheme happens to be n-flavour generic, so NSI (a no-op there) and the sterile extension are "supported" in the sense that nothing breaks, not because the perturbative or numerical machinery was invoked.

- **The solar adiabatic-approximated method and Landau-Zener are the only genuinely SM-only propagation paths in the whole project.** Both rely on a closed-form two-level mixing-angle formula (θ_{12}^M, θ_{13}^M, and the MSW resonance condition respectively) that has no NSI or 3+1 generalisation, and the solar dispatcher enforces this by raising rather than silently ignoring the BSM extension.

- **The analytic eigenvalue solver is selected by flavour count alone, never**

by whether NSI is active. Cardano's trigonometric cubic solution applies to any traceless Hermitian 3×3 reduced Hamiltonian (SM or SM+NSI), and Ferrari's quartic solution applies to any traceless Hermitian 4×4 one (sterile, with or without NSI or V_{NC}) — both formulas only require Hermiticity, not a real matrix, so neither NSI's complex ϵ nor a non-zero active-sterile CP phase δ_{14} forces a numerical fallback. That fallback (`torch.linalg.eigvalsh`) is used only when `analytic_eigenvalues=False` (the default) or when the closed-form solver's own conditioning guard detects a near-degenerate spectrum.

Chapter 5

core.numerical — general numerical propagation

This chapter covers the medium-independent numerical evolutor: exact matrix-exponential integration of the propagation Hamiltonian over a piecewise-constant-density discretisation of the path. It makes no assumption on the size of matter effects and works unmodified for the Standard Model, NSI, the 3+1 sterile extension, or any combination (Chapters 2 to 4), since it consumes `hamiltonian_flavour` directly rather than any scenario-specific formula. It is the general-purpose reference method against which the perturbative approximation (Chapter 6) is validated.

5.1 geometry.py — trajectory discretisation

core/numerical/geometry.py

`Trajectory`: the geometry container passed from a medium profile to the numerical evolutor, and `segment_sample_points`, the quadrature-rule helper that builds it.

Theoretical Foundation

Composing segment evolutors (Chapter 2, `evolutor.py` section) requires splitting a propagation path into N segments delimited by $N+1$ boundary points along some geometry coordinate x (a chord length, a radius, a zenith-angle parametrisation – whatever the medium finds natural). For each segment, the numerical core needs exactly two further pieces of information, both computed once by the medium-specific module that builds the `Trajectory` and never revisited by `core.numerical` itself:

- the **dimensionless evolution length** of the segment, $\Delta x_j^{\mathrm{evo}} = \Delta x_j / L_{\mathrm{scale}}$, i.e. the physical segment length divided by the same `evolution_scale_m` used to non-dimensionalise the Hamiltonian (Chapter 2, `potential.py` section) – this is what actually multiplies H in the matrix exponential below, not Δx_j itself;
- a **sample coordinate** $\bar{x}_j$ per segment, at which the medium's electron density (or any other property entering H) is evaluated once and treated as *constant* over the whole segment – the piecewise-constant-density approximation that makes the per-segment problem exactly solvable (next section).

`segment_sample_points` implements three quadrature rules for $\bar{x}_j$ from the $N+1$ boundary coordinates $x_0, \ldots, x_N$:

$$\bar{x}_j = \begin{cases} \frac{1}{2}(x_j + x_{j+1}) & \texttt{"midpoint"} \text{ (default)}, \\ x_j & \texttt{"left"}, \\ x_{j+1} & \texttt{"right"}. \end{cases} \tag{5.1}$$

The midpoint rule is second-order accurate in the segment width for a smoothly varying density (the segment's leading-order mismatch between the true, continuously varying $n_e(x)$ and the constant $n_e(\bar{x}_j)$ used on it cancels to first order when $\bar{x}_j$ is the segment

centre, by the same Taylor-expansion argument as the midpoint rule in quadrature); the endpoint rules (`"left"`/ `"right"`) are only first-order accurate, and exist mainly to match legacy or directionally-biased sampling conventions.

Implementation in tpeanuts

- `Trajectory` (dataclass): `x` (shape $(N{+}1,)$, the geometry-coordinate boundaries), `dx_evolution` (shape $(N,)$, the dimensionless evolution lengths $\Delta x_j^{\mathrm{evo}}$ above), `sample_x` (shape $(N,)$, the $\bar{x}_j$ above), and `meta` (a plain `dict` of medium-specific bookkeeping – layer indices, crossing flags, ... – that `core.numerical` never interprets).
- `segment_sample_points(x, method)`: builds `sample_x` from `x` via the three rules above; broadcasts over any leading batch dimensions, and returns an empty (zero-segment) result for a single-boundary-point input.
- Every medium-specific geometry module (e.g. `medium.earth.geometry`, `medium.atmosphere.profile`) builds its own `x` in whatever coordinate is natural for that medium, converts it to `dx_evolution` using *that medium's* evolution scale, and calls `segment_sample_points` to fill `sample_x` – `Trajectory` is the single, medium-agnostic hand-off point to `core.numerical`.

5.2 evolutor.py — matrix-exponential composition

`core/numerical/evolutor.py`

`evolutor_numerical_segment` and `evolutor_numerical`: exact per-segment matrix exponentials, composed into the total evolutor.

Theoretical Foundation

Exact solution on a piecewise-constant-density segment

The propagation Hamiltonian $H(x)$ varies continuously with x through the local electron density $n_e(x)$. Given the `Trajectory` of the previous section, `core.numerical` approximates $H(x)$ as **piecewise constant**: on segment j (between boundaries x_j and x_{j+1}), $H(x) \approx H_j$, where

$$H_j \equiv \texttt{hamiltonian\_flavour}(\text{oscillation}, E, n_e(\bar{x}_j)) \tag{5.2}$$

is the full flavour-basis Hamiltonian (Chapter 2) evaluated once at the segment's sample point $\bar{x}_j$. **This is the *only* approximation in the method**: given a constant H_j, the position-dependent Schrödinger equation $i\,\mathrm{d}S/\mathrm{d}x = H_j\,S$ (Chapter 2, `evolutor.py` section) integrates *exactly* – not approximately – over the segment, via the matrix exponential:

$$S_j = \exp(-i\,H_j\,\Delta x_j^{\mathrm{evo}})\,. \tag{5.3}$$

No eigendecomposition, diagonalisation, or perturbative expansion is involved: `torch.linalg.matrix_exp` evaluates this directly (via a scaling-and-squaring / Padé algorithm), batched over every segment and every other broadcast dimension (energy, angle, ...) simultaneously, and remains differentiable end-to-end. Because H_j is built

with `hamiltonian_flavour` – the single, fully general Hamiltonian-assembly function (Chapters 2 to 4) – this formula is correct as written for the 3-flavour Standard Model, NSI, the 3+1 sterile extension, or any combination, with H_j simply changing size (3×3 or 4×4) and content accordingly; nothing in this module branches on which scenario is active. In particular, an optional neutron density $n_n(\bar{x}_j)$ sampled alongside $n_e(\bar{x}_j)$ is forwarded straight through to `hamiltonian_flavour` as `n_n_mol_cm3`, enabling the 3+1 sterile extension's neutral-current term (Chapter 2) with no change to this module's own logic: omitting it (the default) reproduces the pre-existing CC-only behaviour exactly.

Composing the total evolutor

The N per-segment evolutors are composed into the total operator via the ordered product already established generically in Chapter 2:

$$S(x_N, x_0) = S_{N-1}\, S_{N-2} \cdots S_1\, S_0, \tag{5.4}$$

using `compose_segment_evolutors` (Chapter 2) with its binary-tree reduction for $O(\log N)$ parallel depth on GPU rather than $O(N)$ sequential steps. Optionally, `evolutor_numerical` can instead return the *history* of partial products $S(x_j, x_0)$ for every $j = 0, \ldots, N$ (an accumulating loop rather than the tree reduction, since every intermediate result is needed): useful for path-resolved quantities, e.g. probabilities evaluated at every crossed layer of a medium rather than only at the final boundary.

Implementation in `tpeanuts`

- `evolutor_numerical_segment(oscillation, E_MeV, n_e_mol_cm3, dx_evolution, *, n_n_mol_cm3=None, ...)`: builds one H_j per segment (the segment dimension is the final axis of `n_e_mol_cm3`/`dx_evolution`), broadcasting `oscillation.mass_spectrum` and `antinu` to the segment shape via `dataclasses.replace` before calling `hamiltonian_flavour` once for the whole segment batch, then exponentiates: `torch.linalg.matrix_exp(-1j * H * dx[...,None,None])`.
- `evolutor_numerical(oscillation, E_MeV, n_e_mol_cm3, dx_evolution, *, n_n_mol_cm3=None, return_history=False, ...)`: calls the above, then either composes the segments into a single operator (`compose_segment_evolutors`) or accumulates and returns the full per-segment history.
- Both functions run under `@torch.no_grad()`: the numerical evolutor is used for forward propagation, not for parameter inference through automatic differentiation.

Approximations and Limitations

Accuracy is entirely a discretisation choice, made outside this module. `core.numerical` performs no adaptive step-size control and no local error estimate: the accuracy of $S(x_N, x_0)$ is fixed entirely by how finely the `Trajectory` passed in discretises the path (how many segments N, and which quadrature rule fixed $\bar{x}_j$, previous section) – both decided by the calling medium-specific module before `core.numerical` ever sees the problem. Refining the trajectory (more, smaller segments) is the only way to improve accuracy; there is no mechanism to detect that a given N is insufficient. This is the computational cost paid for generality: unlike the perturbative approximation (Chapter 6), which is only valid in a restricted regime but is $\mathcal{O}(1)$ in the number of

density evaluations, the numerical method is valid everywhere but its accuracy is $\mathcal{O}(N)$ in the number of `hamiltonian_flavour` evaluations and matrix exponentials.

Chapter 6

`core.perturbative` — perturbative approximation

This Chapter Describes an Approximation

Everything in this chapter is a **first-order perturbative approximation** to the exact position-dependent Schrödinger equation (Chapter 2), offered as a cheaper alternative to the exact numerical method of Chapter 5 *within a restricted validity range* – it is not a second, independently correct propagation method.

The approximation. Within one segment, the Hamiltonian is split into a constant average and a position-dependent residual, $H(x) = H_{\text{avg}} + \delta H(x)$, where H_{avg} uses the segment-*average* density and $\delta H(x)$ captures the fluctuation around it. The evolutor is expanded to **first order** in δH: $S \approx U_0 + U_1$, with $U_0 = \exp(-iH_{\text{avg}}L)$ exact for the constant average (Section 6.2), and U_1 the first Dyson-series correction from δH.

Range and conditions of validity. The truncation is controlled by the phase the *residual* potential accumulates over the segment, $|\delta V(x)| \cdot L$: the approximation is trustworthy while this stays small compared to 1 throughout the segment (equivalently, while the neglected second- and higher-order Dyson terms stay small relative to U_1). In practice this means: (a) the profile model (Section 6.3) must fit the true density well enough, and/or segments must be short enough, that the density does not deviate strongly from its local average within any one segment; (b) accuracy is expected to degrade for segments spanning a large density contrast (e.g. a sharp discontinuity treated as a single segment). **No automatic error estimate or order check is built into this code path** – unlike Chapter 5's explicit accuracy note, there is no signal here that a given (segment, profile-model) choice has left the valid regime; it must be established externally, typically by cross-validating against the exact numerical evolutor of Chapter 5 for representative parameters before trusting a perturbative configuration in production.

This chapter covers `spectral.py` (the closed-form spectral decomposition every perturbative segment evolutor is built from), `evolutor.py` (the zeroth- and first-order evolution operators themselves), and the family of interchangeable density-profile models that supply the analytic inputs the first-order correction needs.

6.1 `spectral.py` — closed-form spectral decomposition

`core/perturbative/spectral.py`

Trace/traceless splitting, characteristic-polynomial invariants, and closed-form spectral projectors for a 3×3 *or* 4×4 *Hermitian Hamiltonian.*

Theoretical Foundation

Trace and traceless parts

Any Hermitian H splits uniquely into a multiple of the identity and a traceless remainder,

$$T = H - \frac{\mathrm{Tr}(H)}{N} I_N, \qquad N = H.\mathrm{shape}[-1] \in \{3, 4\}. \tag{6.1}$$

Since I_N commutes with everything, H and T share the same eigenvectors, and H's eigenvalues are simply T's shifted uniformly by $\mathrm{Tr}(H)/N$. Working with T instead of H removes the sub-leading term of the characteristic polynomial (no λ^{N-1} term, since $\mathrm{Tr}(T) = 0$), which is what makes the closed-form projector formulas below tractable.

Characteristic-polynomial invariants

For $N = 3$, T's characteristic polynomial is $\lambda^3 + c_1\lambda + c_0 = 0$ with

$$c_1 = -\tfrac{1}{2}\,\mathrm{Tr}\big(T^2\big), \qquad c_0 = -\tfrac{1}{3}\,\mathrm{Tr}\big(T^3\big). \tag{6.2}$$

The $N = 4$ projector formula additionally needs $e_3 = \frac{1}{3}\,\mathrm{Tr}(T^3) = -c_0$ (kept as a separate function in the code to avoid a sign-confusion bug, since both invariants are used together for $N = 4$), and its own eigenvalue solver (below) further needs $e_4 = \det(T)$, the fourth elementary symmetric polynomial of the eigenvalues. By default the eigenvalues λ_a are obtained numerically via `torch.linalg.eigvalsh`; a closed-form alternative, evaluating these invariants explicitly, is also available and derived next.

Theoretical Foundation

Closed-form eigenvalues: Cardano ($N = 3$) and Ferrari ($N = 4$)

Since T is Hermitian and traceless, its characteristic polynomial has no quadratic ($N = 3$) or cubic ($N = 4$) term and is guaranteed to be real-rooted; both properties are what make an explicit closed-form root solver possible here, unlike for a generic matrix.

$N = 3$: Cardano's trigonometric solution. The depressed cubic $\lambda^3 + c_1\lambda + c_0 = 0$ has $c_1 = -\frac{1}{2}\,\mathrm{Tr}(T^2) \le 0$ always (equality iff $T = 0$), since $\mathrm{Tr}(T^2) = \sum_a \lambda_a^2 \ge 0$ for Hermitian T; this is exactly the condition under which the classical trigonometric solution applies, giving all three roots directly, with no branch selection or complex intermediate arithmetic:

$$\lambda_k = 2\sqrt{-\tfrac{c_1}{3}}\,\cos\Big[\tfrac{1}{3}\arccos(r) - \tfrac{2\pi k}{3}\Big], \qquad r = \frac{-c_0}{2\big(-c_1/3\big)^{3/2}}, \qquad k = 0, 1, 2. \tag{6.3}$$

$r \in [-1, 1]$ exactly for a genuinely Hermitian T (it is, up to normalisation, the cosine of the phase entering the standard closed-form eigenvalue formula for a real symmetric 3×3 matrix); in code, r is still clamped to $[-1, 1]$ to absorb floating-point overshoot at (near-)degenerate spectra, where this formula loses precision in the same regime as the projector formula above.

$N = 4$: Ferrari's resolvent-cubic solution. The depressed quartic $\lambda^4 + p\lambda^2 + q\lambda + r = 0$, with $p \equiv c_1$, $q \equiv -e_3$, and $r \equiv e_4 = \det(T)$, is solved by first finding a real root m of its *resolvent cubic*,

$$m^3 + p\,m^2 + \Big(\tfrac{p^2}{4} - r\Big)m - \tfrac{q^2}{8} = 0, \tag{6.4}$$

itself solved with the same trigonometric formula as the $N = 3$ case above (after depressing it via $m = z - p/3$): since T is Hermitian the quartic is guaranteed real-rooted, and the resolvent cubic's three roots are exactly the real pairwise sums $m_{ij|kl} = -\frac{1}{2}(\lambda_i+\lambda_j)(\lambda_k+\lambda_l)$ over the three ways of splitting the four (real) roots into two pairs, so it is real-rooted too and the same closed form applies unconditionally. Taking the *largest* resolvent root m (the conditioning-optimal choice, since it keeps the next square root away from zero whenever any pairing is well separated) factors the quartic into two real quadratics:

$$s = \sqrt{2m}, \qquad \lambda \in \left\{ \tfrac{s \pm \sqrt{\Delta_1}}{2}, \tfrac{-s \pm \sqrt{\Delta_2}}{2} \right\}, \qquad \Delta_1 = -2\left(m + p + \tfrac{q}{s}\right), \quad \Delta_2 = -2\left(m + p - \tfrac{q}{s}\right). \tag{6.5}$$

Unlike the $N = 3$ case, this factorisation is not unconditionally well-conditioned: the chosen pairing's m can itself be close to singular (only possible, for a traceless Hermitian T, near $T \approx 0$), or the resolvent cubic's own three roots can nearly coincide even when the *chosen* m is perfectly well separated from zero (e.g. $\lambda = (a, a, -a, -a)$ has a well-conditioned pairing $m = 2a^2$ whose resolvent cubic nonetheless has a repeated root elsewhere, which loses precision through arccos's diverging derivative at ± 1). Batch entries flagged by either check fall back directly to `torch.linalg.eigvalsh` – unlike the projector degeneracy fallback below, which only ever needs to replace the *projectors*, this closed form can lose the *eigenvalues* themselves in these regimes, so the fallback is applied to the eigenvalues here rather than left to the caller.

Theoretical Foundation

Closed-form spectral projectors

The spectral theorem writes $T = \sum_a \lambda_a M_a$ with $M_a = \prod_{b \neq a} \frac{T - \lambda_b I}{\lambda_a - \lambda_b}$ (the Lagrange/-Sylvester interpolation projector). Using Cayley–Hamilton and $\mathrm{Tr}(T) = 0$ to eliminate the "other" eigenvalues $\lambda_{b \neq a}$ from this product, it reduces to an explicit polynomial in T (and T^2, plus T^3 for $N = 4$) with coefficients depending only on λ_a and the invariants above: For $N = 3$:

$$M_a = \frac{(\lambda_a^2 + c_1)\, I + \lambda_a T + T^2}{3\lambda_a^2 + c_1}. \tag{6.6}$$

For $N = 4$:

$$M_a = \frac{T^3 + \lambda_a T^2 + (c_1 + \lambda_a^2) T + (\lambda_a^3 + c_1 \lambda_a - e_3)\, I}{4\lambda_a^3 + 2c_1\lambda_a - e_3}. \tag{6.7}$$

Both denominators equal $p'(\lambda_a)$, the derivative of the characteristic polynomial at that root: they vanish exactly when λ_a is a repeated eigenvalue. Both formulas were verified symbolically and numerically (against direct Lagrange interpolation and against `torch.linalg.eigh` eigenvector outer products) to match to floating-point precision.

Approximations and Limitations

Degeneracy fallback. Near a (near-)degenerate spectrum the closed-form projector formulas lose precision well before their denominator literally reaches zero, since $p'(\lambda_a) \to 0$ continuously. `_spectral_degeneracy_mask` flags batch entries whose minimum pairwise eigenvalue gap is small *relative* to the spectrum scale (or whose scale itself is numerically zero, the $T \approx 0$ case, where the relative test would be $0/0$); flagged entries have their

projectors replaced wholesale by $M_a = v_a v_a^\dagger$ from a direct `torch.linalg.eigh(T)` call – exact and stable by construction, independent of the closed-form formula's conditioning, computed only when at least one batch entry actually needs it. This module runs entirely under `@torch.no_grad()`, so the well-known gradient instability of degenerate eigenvectors is not a concern here (the perturbative path is used for forward propagation, not for differentiating through the spectrum).

Implementation in `tpeanuts`

- `hamiltonian_traceless(H)`: returns $(T, \operatorname{Tr} H)$.
- `hamiltonian_traceless_c0/c1/e3(T)`: the invariants above.
- `hamiltonian_traceless_eigenvalues(T, *, analytic=False, c1=None, c0=None, e3=None, e4=None)`: λ_a via `torch.linalg.eigvalsh` by default. Passing `analytic=True` instead evaluates the closed-form solution derived above: Cardano for T.shape$[-1] = 3$, Ferrari for T.shape$[-1] = 4$ (any other N falls back to `eigvalsh` unconditionally, since no closed form is defined there). The optional `c1`/`c0` (Cardano) and `c1`/`e3`/`e4` (Ferrari) arguments let a caller that has already computed these invariants – chiefly `hamiltonian_spectral_data` – forward them directly, avoiding redundant trace contractions.
- `hamiltonian_spectral_projectors_traceless(T, ...)`: builds M_a (closed form, with the `eigh` fallback above); returns (M, λ, c_1).
- `hamiltonian_spectral_data(H, *, analytic_eigenvalues=False, ...)`: the one-stop entry point used by `evolutor.py` – computes T, $\operatorname{Tr} H$, λ, c_1, and M in one call, reusing intermediate products (T^2, T^3) across every invariant and the projector formula to avoid redundant matrix multiplications. `analytic_eigenvalues` is forwarded unchanged to `hamiltonian_traceless_eigenvalues` above; this is the flag `evolutor_zero_order`/`evolutor_perturbative_segment` (Section 6.2) expose to their own callers.
- `spectral_projector_residuals(M, T, lam)`: diagnostic residual norms (completeness $\sum_a M_a = I$, idempotency $M_a^2 = M_a$, orthogonality $M_a M_b = 0$, unit trace, and reconstruction $\sum_a \lambda_a M_a = T$) – used both by the test suite and by the degeneracy instrumentation itself.

6.2 `evolutor.py` — perturbative segment evolution

`core/perturbative/evolutor.py`

`evolutor_zero_order` (exact, for the segment-average Hamiltonian) and `evolutor_first_order` (the first Dyson-series correction from the density residual), combined into the segment evolutor below.

Theoretical Foundation

Zeroth order: exact evolution under the average Hamiltonian

Given the spectral decomposition $H_{\text{avg}} = T + \text{Tr}(H_{\text{avg}})/N \cdot I$ from the previous section, the spectral theorem gives $\exp(-iH_{\text{avg}}L)$ *exactly* (not perturbatively) as

$$U_0 = \sum_a \exp\Big[-i\Big(\lambda_a + \tfrac{\text{Tr}\, H_{\text{avg}}}{N}\Big)L\Big]\, M_a\,, \tag{6.8}$$

since conjugating by the identity-shift only shifts every eigenvalue by the same constant $\text{Tr}(H_{\text{avg}})/N$, leaving the projectors M_a untouched. U_0 is the entire answer if the segment's density were exactly constant at its average value; every approximation in this chapter lives in the correction below.

First order: the Dyson-series correction from the residual

Writing $H(x) = H_{\text{avg}} + \delta H(x)$ with $\delta H(x) = \delta V(x)\, P$ (a position-independent *direction* matrix P scaled by the position-dependent residual potential $\delta V(x) = V(x) - V_{\text{avg}}$; see below for P), first-order time-dependent perturbation theory in the interaction picture defined by U_0 gives

$$U_1 = -i \int_{x_1}^{x_2} U_0(x_2, x')\, \delta H(x')\, U_0(x', x_1)\, \mathrm{d}x'. \tag{6.9}$$

Expanding U_0 in the spectral basis, $U_0(x_2, x') = \sum_a e^{-i\lambda_a(x_2 - x')} M_a$, and inserting $\delta H(x') = \delta V(x')P$ turns the operator integral into a sum over eigenvalue *pairs* of a scalar oscillatory integral of the residual density weighted by the matrix $M_a P M_b$:

$$U_1 = -i \sum_{a,b} I_{ab}\, M_a\, P\, M_b\,, \qquad I_{ab} = \int_{x_1}^{x_2} \delta V(x')\, e^{-i\lambda_a(x_2 - x') - i\lambda_b(x' - x_1)}\, \mathrm{d}x'. \tag{6.10}$$

I_{ab} (a "spectral oscillatory integral" of the residual profile) is computed analytically by the active profile model (Section 6.3, `residual_integral`); it is the only place the specific density parametrisation enters the perturbative machinery – `evolutor.py` itself never sees a profile's functional form, only $I_{ab}(\lambda_a, \lambda_b)$.

The direction matrix P. For the plain Standard Model or 3+1 sterile case (no NSI), only the electron-flavour entry of H_{mat} varies with density, so $P = \text{diag}(1, 0, \ldots, 0)$ and the sandwich $M_a P M_b$ reduces to the rank-1 outer product of M_a's first column with M_b's first row – the closed-form shortcut used when the caller passes no explicit P. When NSI is active, the coupling ϵ is position-independent (only the electron density magnitude, not the NSI coupling itself, varies along the segment), so the same first-order structure applies with

$$P = O^\dagger(\text{diag}(1, 0, \ldots, 0) + \epsilon)O \tag{6.11}$$

in the reduced basis (Chapter 4) – fixed once per segment (built at $V{=}1$) and reused for both H_{avg}'s matter term and this first-order sandwich, since it no longer reduces to the SM rank-1 shortcut once ϵ breaks the electron-only structure.

The neutral-current direction P_{NC}. When the optional 3+1 sterile neutral-current term is active (Chapter 2), it contributes a second, wholly independent first-order term, since the electron- and neutron-density residuals fluctuate independently along the segment:

$$U_1 = -i \sum_{a,b} I^{(e)}_{ab}\, M_a\, P_{CC}\, M_b \; - i \sum_{a,b} I^{(n)}_{ab}\, M_a\, P_{NC}\, M_b\,, \tag{6.12}$$

where $I_{ab}^{(n)}$ is the same spectral oscillatory integral as $I_{ab}^{(e)}$ but of the *neutron*-density residual (`residual_integral_neutron`, below), and $P_{NC} = O^\dagger \,\mathrm{diag}(0,0,0,-1)\, O$ is the (always rotated) direction matrix derived in Chapter 2. Unlike P_{CC}, P_{NC} never reduces to a rank-1 shortcut regardless of whether NSI is active, since it never commutes with O; the CC term above may still use its own rank-1 shortcut independently, since the two contributions are additive and computed separately. Omitting the neutral-current term (the default) recovers the CC(+NSI)-only expression exactly.

Combining orders, and the zero-length convention

The segment evolutor is $U = U_0 + U_1$ (`evolutor_perturbative_from_H`), with two guards applied by the code around this core formula: (1) if the profile model reports no perturbation for a segment (a genuinely constant density, e.g. `has_perturbation()` is `False`), U_1 is skipped entirely — both because it is identically zero and to avoid computing I_{ab} needlessly; (2) zero-length segments are forced to the identity operator (`enforce_identity_for_zero_length`) regardless of the formula above, since $L = 0$ is a genuine edge case (e.g. a trajectory that grazes but does not cross a layer) rather than a regime where the perturbative expansion itself is being tested.

Approximations and Limitations

Hermitian conjugate, again. `evolutor_perturbative_segment` builds H_{kin} via `hamiltonian_kinetic_reduced` (Chapter 2), which – exactly as flagged there and in Chapter 4 – always uses $U_{\mathrm{red}}\,\mathrm{diag}(k)U_{\mathrm{red}}^\dagger$, never the plain transpose. This matters even more visibly here: for the 3+1 sterile case the code computes the segment's flavour-basis trace directly as $\mathrm{Tr}(H_{\mathrm{kin}} + H_{\mathrm{mat}})$ rather than assuming $\mathrm{Tr}(H_{\mathrm{kin}}) = \sum_i k_i$ (only guaranteed for a *real* U_{red}) or $\mathrm{Tr}(H_{\mathrm{mat}}) = V$ (only true for the plain $\mathrm{diag}(1,0,\ldots,0)$ direction) – both shortcuts used safely in the pure-SM fast path, both invalid once the sterile CP phase δ_{14} or a generic NSI ϵ is active.

Implementation in tpeanuts

- `evolutor_zero_order(H, L, ...)`: the exact U_0 formula above; can also return the spectral data for reuse by the first-order call, avoiding a second diagonalisation.
- `evolutor_first_order(M, lam, trace_H, profile_model, ..., P=None, P_nc=None)`: the U_1 formula above, using the rank-1 shortcut for the CC term when `P` is `None`; when `P_nc` is also supplied, adds the independent neutral-current sandwich term using `profile_model.residual_integral_neutron` and `matter_potential_nc`.
- `evolutor_perturbative_from_H(H, L, profile_model, ..., P=None, P_nc=None)`: combines both orders, with the no-perturbation and zero-length guards; the no-perturbation check also consults `profile_model.has_perturbation_neutron()` when `P_nc` is supplied.
- `evolutor_perturbative_segment(oscillation, E_MeV, profile_model, ..., include_matter_nc=False)`: the user-facing entry point – builds the reduced Hamiltonian (SM, sterile, NSI, or any combination, mirroring Chapters 2 and 4) from the profile model's segment average, then evolves it perturbatively via the function above. The plain Standard Model case takes a dedicated fast path (bit-identical output, avoiding the general-N machinery's over-

head). `include_matter_nc=True` additionally requires `profile_model.potential_n` to be set (i.e. the profile was built with neutron-density coefficients, next section) and is silently ignored for a 3-flavour `oscillation.pmns`, mirroring `hamiltonian_matter_reduced`'s own convention (Chapter 2).

6.3 Comparing the perturbative density-profile models

`core/perturbative/models/*`

Three interchangeable profile models, each supplying `average`, `length`, `zero_mask`, `potential`, `residual_integral`, *and* `has_perturbation` *to the model-agnostic evolutor above.*

Implementation in `tpeanuts`

None of the mathematics in `evolutor.py` or `spectral.py` changes between models: they differ purely in *how* the segment's density profile $n_e(x)$ is parametrised, and hence in the closed-form expression used for I_{ab}.

Model	Density parametrisation and usage
`even_power`	$n_e(x) = a + bx^2 + cx^4 + \delta_1 x^6 + \dots$ (even powers only, arbitrary order) in the raw trajectory coordinate x. Appropriate whenever the segment is symmetric about the trajectory's point of closest approach to a spherically symmetric medium (e.g. an Earth-crossing chord through one shell), which forces $n_e(x) = n_e(-x)$ and eliminates odd powers identically. The general-order model used for Earth propagation.
`prem`	$n_e(x) = a + bx^2$: the 2-coefficient special case of `even_power` truncated at the quadratic term, matching the PREM reference model's own native linear-in-r^2 parametrisation per shell [4]. A lighter-weight specialisation, not an independent formulation – it exists to avoid the general `even_power` machinery's overhead when only two coefficients are ever needed.
`atmosphere`	$n_e(q) = \sum_k c_k q^k$ (every power, even and odd) in a *centred and rescaled* local coordinate $q = (x - \text{centre})/\text{half-width}$. Atmosphere segments are not symmetric about their own centre the way an Earth chord is, so odd terms are required; the local coordinate keeps the fitted coefficients well-conditioned regardless of the segment's absolute position or width.

Every model computes its own `residual_integral` in closed form for ordinary (non-degenerate) eigenvalue gaps, switching to a Taylor expansion in $\lambda_a - \lambda_b$ near degeneracy to avoid the numerical instability of the closed form as the oscillation frequency approaches zero – the same degeneracy-awareness pattern as the spectral projectors themselves (previous section), applied independently at the profile-model level. `perturbative_profile_selection` (`core/perturbative/models/model_selection.py`) maps a public model name (`"even_power"`, `"prem"`, ...) to the concrete layered-profile class, so medium code can request a model without importing model-specific classes directly.

Optional neutron-density coefficients. All three segment classes optionally accept a second, independent set of coefficients for the neutron density n_n – same functional form and coordinate as the electron-density fit above, fitted or sup-

plied separately – exposing `average_n`, `potential_n`, `residual_integral_neutron`, and `has_perturbation_neutron` alongside the electron-only attributes, required by `evolutor_perturbative_segment`'s `include_matter_nc=True` (previous section). They default to `None`, reproducing the pre-existing CC-only behaviour exactly; the layered profile classes that build these segments (`EvenPowerProfileLayered`, `PremTabulatedProfile`) load them from a composition-derived neutron-density table when constructed with `include_neutron=True` (Chapter 9), and `AtmospherePolynomialProfile` fits them from a separate neutron-density sample at the same nodes (Chapter 10).

PART III

Physical Propagation Media — `medium/`

Chapter 7

`medium.vacuum` — coherent propagation in vacuum

`medium.vacuum` is the simplest propagation medium in the project: $n_e \equiv 0$ everywhere, so the matter term of the Hamiltonian (Chapter 2) never enters and every formula reduces to the closed-form vacuum evolutor already derived, in general terms, in the coherent-propagation scheme of Chapter 3. Its role is twofold:

- **Base case and test bed.** With no density profile to model and a fully closed-form evolutor, this is the cheapest possible path through the entire propagation stack (`PMNS` → `kinetic_eigenvalue_vector` → evolutor → probability → flux, Chapter 2), with an exact analytic answer to check every other piece against. It is, in practice, the first thing exercised when validating a new oscillation preset, a new BSM extension, or a refactor of the shared infrastructure: if `medium.vacuum` disagrees with the legacy `peanuts` reference (Section 7.4), nothing built on top of it can be trusted either.
- **Physical use cases.** Vacuum propagation is the correct description whenever matter effects are genuinely negligible along the whole path: reactor and other short-baseline experiments, or any scenario where the accumulated matter phase $V \cdot L$ is small enough to ignore relative to the vacuum kinetic phases. It is also the $n_e \to 0$ reference limit that the matter-propagation media of the following chapters must recover exactly in that limit.

Approximations and Limitations

The four-module pattern. `medium.vacuum` fixes a module layout that every subsequent medium in this part of the document repeats, adapted to that medium's own needs: an `evolutor.py` building the medium-specific S from `OscillationParameters` and geometry inputs; a `probability.py` converting that evolutor into transition and final-state probabilities via the shared `core.common.probability` conventions (Chapter 2); a `flux.py` applying `core.common.flux` normalisations on top; and, where a legacy NumPy reference implementation exists to check against, a `validation.py` comparing the two numerically. Media with additional physical structure add further modules on top of this skeleton (`profile.py` for a density profile, `geometry.py` for path geometry, `exposure_*.py` for detector exposure, ...) without changing this core four-piece contract.

7.1 `evolutor.py` — the vacuum evolutor

`medium/vacuum/evolutor.py`

`vacuum_evolutor` and `vacuum_evolved_state`: the closed-form coherent vacuum operator and its action on a state.

Theoretical Foundation

With $H_{\text{mat}} = 0$, the reduced and flavour Hamiltonians coincide with the pure kinetic term, and the general coherent-propagation scheme of Chapter 3 is exact and complete on its own (no reduction trick is needed, since there is no matter term to factor out):

$$S(L,E) = U\ \mathrm{diag}(e^{-ik_1 x}, e^{-ik_2 x}, e^{-ik_3 x})\, U^\dagger\,, \qquad x = \frac{L\,[\mathrm{m}]}{L_{\text{scale}}}, \tag{7.1}$$

where U is the full PMNS matrix (Chapter 2), k_i the dimensionless kinetic eigenvalues (Chapter 2, `potential.py` section), and the baseline L (given in km) is converted to metres and non-dimensionalised by the same `evolution_scale_m` used throughout the project. This is the same formula the sterile extension recovers automatically for $n = 4$ (Chapter 4), simply by passing a 4×4 `PMNS_sterile` object in place of `PMNS_SM`; nothing in this module is 3-flavour-specific.

Implementation in `tpeanuts`

- `vacuum_evolutor(oscillation, E_MeV, L_km, ...)`: builds S exactly as above; broadcasts energy, baseline, and mass-splitting batch shapes together before the phase multiplication.
- `vacuum_evolved_state(nustate, oscillation, E_MeV, L_km, ...)`: applies `core.common.evolutor.apply_evolutor_to_state` (Chapter 2) to a coherent flavour-basis initial state.
- `legacy_precision` is accepted for call-site symmetry with every matter-propagation evolutor in later chapters but is a genuine no-op here: there is no matter potential for the flag to affect.

7.2 `probability.py` — vacuum probabilities

`medium/vacuum/probability.py`

`vacuum_probability_transition` (the full transition-probability matrix), `vacuum_probability_state` (final-state probabilities for a given initial state), and `vacuum_probability_integrated` (spectrum-weighted energy average).

Implementation in `tpeanuts`

- `vacuum_probability_transition(oscillation, E_MeV, L_km, ...)`: builds S and returns $P_{\beta\alpha} = |S_{\beta\alpha}|^2$ via `core.common.probability.probability_transition` (Chapter 2) – the full 3×3 (or 4×4) channel matrix, no initial state required.
- `vacuum_probability_state(nustate, oscillation, E_MeV, L_km, massbasis, ...)`: dispatches to the coherent or incoherent formula of Chapter 2 (`evolutor.py/probability.py` sections) via `probability_state`, according to `massbasis`: `False` evolves `nustate` as coherent flavour amplitudes ($P_\beta = |(S\,\psi)_\beta|^2$); `True` (the default here) treats it as incoherent mass-basis weights ($P_\beta = \sum_i |(S\,U)_{\beta i}|^2 w_i$). Either way the function returns a final flavour-probability vector, never a flux.

- vacuum_probability_integrated(nustate, oscillation, E_MeV, L_km, spectrum, ...): builds vacuum_probability_state on an energy grid and averages it with core.common.probability.probability_integrated (Chapter 2), weighted by an explicit production spectrum – there is no default weight, since a flat spectrum over an arbitrary grid is a modelling choice.

7.3 flux.py — vacuum flux

medium/vacuum/flux.py

vacuum_flux_state and vacuum_flux_integrated: apply core.common.flux normalisation and energy integration to vacuum_probability_state's output.

Implementation in tpeanuts

- vacuum_flux_state(nustate, oscillation, E_MeV, L_km, flux, spectrum, massbasis, ...): adds no new physics – calls vacuum_probability_state above, then core.common.flux.flux_state (Chapter 2) to apply the flux normalisation and optional spectral weight. This is the minimal instance of the flux pattern that every later medium's flux.py extends.
- vacuum_flux_integrated(nustate, oscillation, E_MeV, L_km, flux, massbasis, ..., energy_dim=-2): builds vacuum_flux_state on an energy grid and integrates it with core.common.flux.flux_integrated, returning an unnormalised physical rate rather than the normalised average of vacuum_probability_integrated.

7.4 validation.py — cross-check against legacy peanuts

medium/vacuum/validation.py

Numerical comparison against the bundled legacy, NumPy-based peanuts package.

Implementation in tpeanuts

The legacy vacuum implementation is scalar (one energy, one baseline, one initial state at a time); this module evaluates both implementations on matching scalar inputs and compares the results directly, establishing the pattern reused by medium.solar.validation and medium.earth.validation (Chapters 8 and 9).

- ensure_legacy_importable() / legacy_modules(): locate and import the bundled peanuts package (kept in the repository specifically as this validation reference, not as a runtime dependency of tpeanuts itself).
- legacy_pmns_from_torch(...): rebuilds a legacy peanuts.pmns.PMNS object from a torch PMNS instance's parameters, so both implementations start from identical inputs.
- compare_vacuum_probability_state_with_legacy(...) and compare_vacuum_evolved_state_with_legacy(...): run both implementations and return their outputs (and/or a residual) for the test suite to assert

against.

Chapter 8

`medium.solar` — adiabatic MSW propagation in the Sun

Solar neutrinos are produced deep in the high-density solar core and exit through a smoothly, monotonically decreasing electron density to vacuum. This chapter's physics is dominated by one fact exploited throughout: the density gradient is slow enough, on the scale of the relevant MSW resonances, that the **adiabatic approximation** applies almost everywhere in the standard oscillation parameter space – a neutrino produced in a given local matter eigenstate stays in it as the density drops, so propagation reduces to an incoherent mixture of eigenstates rather than a coherent evolutor (see the note in Section 8.3).

8.1 `profile.py` — the tabulated solar model

`medium/solar/profile.py`

***`SolarProfile`**: tabulated solar radius, electron density, per-source production fractions, and per-source total fluxes.*

Implementation in `tpeanuts`

`SolarProfile` stores four tabulated quantities on a common radius grid $r/R_\odot \in [0, 1]$: the electron density $n_e(r)$ (mol/cm^3), a production-fraction profile $f_{\text{source}}(r)$ per neutrino-producing reaction (pp, ^{8}B, ^{7}Be, ...), and a total integrated flux Φ_{source} per source. `SolarProfile.default` loads these from the package's bundled CSV tables (`tpeanuts/data/solar`), whose paths are fixed by `tpeanuts.config.default.solar_model_filename` and `solar_fluxes_filename` – currently the **zenodo SF3-AGSS09** tables, a standard-solar-model dataset combining helioseismic constraints with the AGSS09 heavy-element abundances [23], tabulated over the full solar radius (2001 rows). `SolarParameters` allows overriding both file paths explicitly, but the defaults are the project's validated reference composition and should not be changed casually.

- `electron_density(r_query)` / `production_fraction(source, r_query)`: linear interpolation on the tabulated grid (`util.math.interp1d_linear`), clamped to the boundary values outside $[0, 1]$.
- `source_fractions(sources)`: stacks one or several tabulated (not interpolated) production-fraction profiles.
- `normalized_fraction(source)`: normalises a production profile to unit trapezoidal integral over r (`util.math.normalize_trapz`).
- `flux(source)`: the total (radius-integrated) flux normalisation for one source, as tabulated.
- `use_LZ`: boolean flag (default `False`) selecting whether `probability.py` (Section 8.4) applies the Landau-Zener correction of Section 8.3.

Approximations and Limitations

Validity of the tabulated model. The bundled solar model is a *standard* solar model: a hydrostatic, helioseismically calibrated stellar-structure calculation, not a direct measurement. Its accuracy is therefore tied to the input microphysics (opacities, nuclear reaction rates, heavy-element abundances) and is typically quoted at the few-percent level for the core density and composition profiles that set $n_e(r)$ and the production-fraction shapes; solar-neutrino and helioseismic data provide independent, largely consistent cross-checks of this precision. Because $n_e(r)$ only ever enters through the dimensionless ratio V_k (Section 8.2), a uniform rescaling error in the model's overall density normalisation would bias the resonance radius, not the qualitative adiabatic/non-adiabatic character of the propagation.

8.2 `matter_mixing.py` — the adiabatic matter-mixing angles

`medium/solar/matter_mixing.py`

Closed-form matter-modified mixing angles $\theta_{12}^M(r)$, $\theta_{13}^M(r)$ used by the adiabatic solar-probability approximation.

Theoretical Foundation

Because $\Delta m^2_{3\ell} \gg \Delta m^2_{21}$ and θ_{13} is small, the solar MSW problem separates hierarchically into two sequential effective 2-flavour resonances (the 1–3 sector, then the 1–2 sector), each with the standard closed-form matter-mixing-angle solution [10] – no numerical diagonalisation of the full 3×3 Hamiltonian is needed in the Standard Model case. Define the dimensionless matter-potential ratio and the effective atmospheric splitting

$$V_k(\Delta m^2, E, n_e) = \frac{V(n_e)}{k(\Delta m^2, E)} = \frac{2E\hbar c\, V_{CC}}{\Delta m^2}, \tag{8.1}$$

$$\Delta m^2_{ee} = \cos^2\theta_{12}\,\Delta m^2_{31} + \sin^2\theta_{12}\,\Delta m^2_{32}, \tag{8.2}$$

using V and k exactly as defined in Chapter 2 (`potential.py` section) – V_k is simply their ratio, evaluated here for a given splitting rather than embedded in the Hamiltonian. The matter-modified reactor angle is

$$\theta_{13}^M = \tfrac{1}{2}\arccos\left(\frac{\cos 2\theta_{13} - V_k(\Delta m^2_{ee}, E, n_e)}{\sqrt{\left(\cos 2\theta_{13} - V_k(\Delta m^2_{ee}, E, n_e)\right)^2 + \sin^2 2\theta_{13}}}\right), \tag{8.3}$$

and, using θ_{13}^M as an input, the matter-modified solar angle is

$$\theta_{12}^M = \tfrac{1}{2}\arccos\left(\frac{\cos 2\theta_{12} - V'_k}{\sqrt{(\cos 2\theta_{12} - V'_k)^2 + \sin^2 2\theta_{12}\cos^2(\theta_{13}^M - \theta_{13})}}\right), \tag{8.4}$$

$$V'_k = V_k(\Delta m^2_{21}, E, n_e)\cos^2\theta_{13}^M + \frac{\Delta m^2_{ee}}{\Delta m^2_{21}}\sin^2(\theta_{13}^M - \theta_{13}). \tag{8.5}$$

Each resonance occurs where the corresponding $V_k = \cos 2\theta$, i.e. where the vacuum mixing angle is driven to $\pi/4$ by the matter potential; away from resonance $\theta^M \to \theta$ (low density) or $\theta^M \to \pi/2$ (high density), recovering the vacuum and fully-suppressed limits respectively.

Approximations and Limitations

`legacy_precision=True` reproduces the legacy `peanuts` combined prefactor for V_k ($3.868\times 10^{-7}/2.533$, both factors independently rounded to 4 significant digits in the original implementation) rather than deriving the matter and kinetic prefactors separately from full-precision constants (Chapter 2); the two choices differ by a relative $\sim 3.6\times10^{-4}$ – an order of magnitude larger than the matter-potential-only rounding error of Chapter 2, since both legacy prefactors compound. Intended purely for bit-comparable validation against legacy `peanuts`.

Implementation in `tpeanuts`

`Vk(Deltam2, E, ne, antinu, legacy_precision)`, `DeltamSqee(oscillation)`, `th13_M(oscillation, E, ne, ...)`, `th12_M(oscillation, E, ne, ..., th13m=None)` implement the four formulas above directly; `th12_M` accepts a pre-computed `th13m` to avoid recomputing it when both angles are needed together (as `Tei` below does).

8.3 `landau_zener.py` — non-adiabatic corrections

`medium/solar/landau_zener.py`

Landau-Zener transition probability $P_{LZ}(E)$ quantifying the breakdown of the adiabatic approximation near the MSW resonance.

Theoretical Foundation

At *finite* adiabaticity, a neutrino crossing the MSW resonance can jump between the two local matter eigenstates instead of adiabatically following one of them. The exponential Landau-Zener approximation [20] gives the jump probability as

$$P_{LZ} = \exp\left(-\frac{\pi}{2}\gamma_{\rm res}\right), \tag{8.6}$$

with the adiabaticity parameter evaluated at the resonance [10]

$$\gamma_{\rm res} = \frac{\Delta m^2_{21}\sin^2 2\theta_{12}}{2E\hbar c\cos 2\theta_{12}}\,L_n(r_{\rm res}), \qquad L_n = \left|\frac{n_e}{{\rm d}n_e/{\rm d}l}\right|_{\rm res}, \tag{8.7}$$

where L_n is the electron-density scale height (in metres) at the resonance radius $r_{\rm res}(E)$, defined by $V_k(\Delta m^2_{21}, E, n_e(r_{\rm res})) = \cos 2\theta_{12}$ (i.e. where $\theta^M_{12} = \pi/4$). Large $\gamma_{\rm res}$ means a slowly varying density on the scale of the oscillation length at resonance, hence a vanishing jump probability (adiabatic); small $\gamma_{\rm res}$ means a comparatively abrupt density change, hence a sizeable jump probability.

Approximations and Limitations

Scope and limitations, as documented in the module itself. Only the θ_{12}-sector resonance is tracked (θ_{13}'s resonance sits deep in the solar core at very high density and is negligible for every standard solar source). $P_{LZ} = 0$ is returned wherever no

resonance exists in the solar volume: below-threshold energies, antineutrinos (no MSW resonance for standard parameters), and LMA-Dark parameters ($\theta_{12} > \pi/4$, resonance at unphysical negative density). For standard LMA parameters and pp/^{8}B solar neutrinos, $\gamma_{\rm res} \sim 10^3$–$10^4$, so $P_{LZ} \sim e^{-1500} \approx 0$ and the pure adiabatic approximation is excellent; this module becomes relevant mainly for non-standard parameter explorations or precision studies in the 1–3 MeV range. Solar NSI is **not** included here: the resonance condition uses the standard V_k, and `Tei` (Section 8.4) silently ignores `p_lz` once `oscillation.nsi` is set, since both the resonance condition and the adiabaticity parameter would need to be rederived for off-diagonal ϵ.

Implementation in `tpeanuts`

`density_gradient(solar_profile)`: $\mathrm{d}n_e/\mathrm{d}\hat{r}$ on the tabulated grid via central differences (one-sided at the boundaries). `resonance_radius(oscillation, E, solar_profile)`: locates $r_{\rm res}(E)$ by linear interpolation at the first sign-change of $V_k - \cos 2\theta_{12}$ along the (monotonically decreasing) density profile; returns `NaN` where no crossing exists. `plz(oscillation, E, solar_profile)`: the $P_{LZ}(E)$ formula above, masked to exactly zero wherever `resonance_radius` returns `NaN`.

No Coherent Medium Evolutor

Unlike every other medium in this document, `medium.solar` has **no `evolutor.py`**. This is a direct consequence of the adiabatic approximation, not an omission: once a neutrino is assigned to a local matter eigenstate at its production radius (`Tei`, next section), the adiabatic theorem guarantees it stays in the *corresponding* eigenstate all the way to the solar surface, where the density has dropped to (effectively) zero and the local matter eigenstates have smoothly become the *vacuum* mass eigenstates. No further coherent phase evolution needs computing: the production-point projection already *is* the final incoherent mass-basis composition, and only the final flavour projection (`solar_probability_state`) remains. Consequently `medium.solar` exposes no `solar_probability_transition`: unlike the other media (Chapters 7, 9 and 10), there is no coherent evolutor S to build a transition matrix from, so `solar_probability_state` is the lowest-tier public probability function for this medium.

8.4 probability.py — production weights and flavour probabilities

`medium/solar/probability.py`

`Tei` (production-point matter-eigenstate weights), `solar_probability_mass` (integrated over the production profile), `solar_probability_state` (final flavour probabilities), and `solar_probability_integrated` (spectrum-weighted energy average).

Theoretical Foundation

`Tei`: flavour-to-matter-basis production weights

T_{ei} answers: given an electron neutrino produced at electron density n_e and energy E, what is the probability it is projected onto local matter eigenstate i? In the Standard

Model (adiabatic, no LZ) case this is read directly off the matter-mixing angles of Section 8.2:

$$(w_1, w_2, w_3) = \left(\cos^2\theta_{13}^M \cos^2\theta_{12}^M,\ \ \cos^2\theta_{13}^M \sin^2\theta_{12}^M,\ \ \sin^2\theta_{13}^M\right), \tag{8.8}$$

i.e. $|\langle\nu_e|\nu_i^M\rangle|^2$ for the standard PMNS-like parametrisation applied to the matter-modified angles. When a Landau-Zener probability P_{LZ} is supplied (Section 8.3), the 1–2 sector weights mix according to the LZ jump probability,

$$w_1 = (1-P_{LZ})\cos^2\theta_{12}^M + P_{LZ}\sin^2\theta_{12}^M, \qquad w_2 = (1-P_{LZ})\sin^2\theta_{12}^M + P_{LZ}\cos^2\theta_{12}^M, \tag{8.9}$$

while $w_3 = \sin^2\theta_{13}^M$ is untouched (the θ_{13} resonance is negligible, Section 8.3). **When NSI is active** (`oscillation.nsi` set), this entire analytic construction is replaced by direct numerical diagonalisation: the full flavour-basis Hamiltonian $H = H_{\rm kin} + H_{\rm mat}^{\rm NSI}$ (`hamiltonian_flavour`, Chapters 2 and 4) is built at each (E, n_e) point and diagonalised with `torch.linalg.eigh`; the production weights are then simply the squared electron-flavour components of each matter eigenvector,

$$w_i(E, n_e) = \left|\langle\nu_e|\nu_i^M(E, n_e)\rangle\right|^2 = |\mathrm{eigvec}[\ldots, 0, i]|^2, \tag{8.10}$$

which supports arbitrary off-diagonal ϵ (including LMA-Dark-type degenerate scenarios) at the cost of losing the closed form; `p_lz` is silently ignored in this path.

From production weights to a final flavour probability

`solar_probability_mass` integrates $T_{ei}(E, n_e(r))$ over the production-radius distribution of one or more sources, weighted by each source's normalised production-fraction profile $f_{\rm source}(r)$ (Section 8.1):

$$P_i(E) = \frac{\int_0^1 T_{ei}(E, n_e(r))\, f_{\rm source}(r)\, \mathrm{d}r}{\int_0^1 f_{\rm source}(r)\, \mathrm{d}r}, \tag{8.11}$$

evaluated by the trapezoidal rule, giving an energy-dependent (not radius-resolved) incoherent *vacuum* mass-basis probability vector $P_i(E)$ for that source – by the argument of the note above, this average over production radius already *is* the mass-basis composition reaching the solar surface, with no further coherent step required. `solar_probability_state` performs the last, purely kinematic conversion to flavour, using the ordinary incoherent-mixture formula of Chapter 2 (`probability.py` section) with the *identity* evolutor (there is nothing left to evolve coherently):

$$P_\alpha(E) = \sum_i |U_{\alpha i}|^2\, P_i(E), \tag{8.12}$$

where U is the ordinary *vacuum* PMNS matrix (Chapters 2 and 3) – the matter-modified angles only ever appear inside T_{ei}, at the production point; every subsequent step in this chapter works with the vacuum mixing matrix.

Implementation in tpeanuts

- `Tei(oscillation, E, ne, p_lz=None, legacy_precision=False)`: the weight formulas above.
- `solar_probability_mass(oscillation, E, profile, sources, ...)`: builds the spatially resolved LZ mask (production radii above the resonance density only, Section 8.3) when `profile.use_LZ` and no NSI is active, then integrates T_{ei} against the source's production-fraction profile as above.
- `solar_probability_state(oscillation, E, profile, sources, ...)`: the final flavour-probability formula above, via `core.common.probability.probability_incoherent` (Chapter 2) with the identity operator standing in for "no further coherent evolution".
- `solar_probability_integrated(oscillation, E, profile, sources, spectrum, ...)`: builds `solar_probability_state` on an energy grid and averages it with `core.common.probability.probability_integrated` (Chapter 2), weighted by an explicit production `spectrum`.

8.5 `flux.py` — solar flux

`medium/solar/flux.py`

`solar_flux_state` and `solar_flux_integrated`: apply per-source flux normalisations, optional spectra, and energy integration to `solar_probability_state`'s output.

Implementation in tpeanuts

- `solar_flux_state(sources, profile, oscillation, E_MeV, source_spectrum=None, ..., date=None)`: adds no new physics – calls `solar_probability_state` above for the requested source(s), then `core.common.flux.flux_state` (Chapter 2) to apply each source's total flux normalisation (`SolarProfile.flux`) and optional energy spectrum – the same minimal pattern established in Chapter 7. The optional `date` rescales the result by the instantaneous Sun-Earth distance modulation $(1\,\mathrm{AU}/R(\mathrm{date}))^2$ (`medium.solar.geometry.sun_earth_distance_factor`); omitted (the default), the flux stays at its 1 AU reference normalisation.
- `solar_flux_integrated(sources, profile, oscillation, E_MeV, source_spectrum=None, ..., energy_dim=-2, date=None)`: builds `solar_flux_state` on an energy grid and integrates it with `core.common.flux.flux_integrated`, returning an unnormalised physical rate. `SolarProfile.flux` is each source's already energy-*integrated* total, not a spectral density, so a normalised production spectrum – `profile.spectrum` by default, or an explicit `source_spectrum` override – is what turns it into the genuine $\mathrm{d}\Phi/\mathrm{d}E$ that `flux_integrated` expects; without it, the energy integral would silently pick up a spurious factor of the grid's own units and spacing.

Chapter 9

`medium.earth` — Earth-crossing matter propagation

Earth-crossing neutrinos (atmospheric, astrophysical, or terrestrial-source) see a matter density that varies by four orders of magnitude between the crust and the inner core, along a chord whose length and depth of penetration depend entirely on the detector's nadir angle. This chapter combines the perturbative machinery of Chapter 6 (dense, shell-crossing trajectories) with the exact numerical evolutor of Chapter 5 (used as the reference and as an alternative pipeline), plus a dedicated geometric layer converting detector angles into trajectory coordinates, and a time-exposure layer averaging over a year of Earth rotation and orbital motion.

9.1 `profile.py` — the Earth density model

`medium/earth/profile.py`

***`EarthProfile`**: shell radii and a delegated perturbative density model, transformed into trajectory coordinates on demand.*

Implementation in `tpeanuts`

Sources for the density profile. Every option ultimately traces back to the Preliminary Reference Earth Model, PREM [4] – a seismologically constrained, spherically symmetric radial density model, tabulated in ∼500 layers from the inner core to the surface. `tpeanuts` offers three ways to consume it, selected by `EarthParameters.profile_perturbative_name` (Chapter 6, `model_selection.py`):

- `"even_power"` (**the default**): the legacy `peanuts` even-power polynomial fit per shell, $n_e(r) = \alpha + \beta r^2 + \gamma r^4 + \ldots$, read from `data/density/earth_density.csv`. This is what every perturbative Earth evolutor formula in this chapter uses unless overridden.
- `"prem"`: the canonical PREM table's piecewise-linear-in-r^2 parametrisation per shell, consumed directly (converting tabulated mass density to electron density via layer-appropriate $\langle Z/A \rangle$ ratios) rather than through a polynomial fit (Chapter 6).
- **Tabulated** (`earth_tabulated_density=True`): bypasses the perturbative polynomial representation entirely and interpolates the raw tabulated density directly – at the cost of losing the closed-form residual integral that the perturbative pipeline needs, so this mode is only usable together with the numerical evolutor (Chapter 5), not the analytical one below.

Implementation in `tpeanuts`

- `EarthParameters`: bundles the profile-model selection above, `profile_scale_m`/`evolution_scale_m` (normally both $R_\oplus$, kept independent for testing), detector `depth_m`, the `method` choice (`"analytical"`/`"numerical"`,

Section 9.4), and the numerical-mode step count `nsteps` and sampling rule `ode_method` (Chapter 5).

- `EarthProfile.shells_x(eta)`: for a surface-equivalent nadir angle, returns each shell's chord-coordinate crossing point via $x_j = \sqrt{r_j^2 - \sin^2\eta}$ (clamped at 0), and which shells are physically crossed ($r_j > \sin\eta$).
- `EarthProfile.trajectory_profile(eta)`: re-expresses the selected radial model in the trajectory coordinate x via the chord identity $r^2 = x^2 + \sin^2\eta$ (Section 9.2).
- `EarthProfile.density_x_eta(x, eta)`: pointwise electron density at a trajectory coordinate, used directly by the numerical pipeline (Section 9.4).

Implementation in `tpeanuts`

Optional neutron density, for the 3+1 sterile neutral-current term. Both the `"even_power"` and `"prem"` layered profile models optionally expose a neutron-density companion table alongside their default electron-density one, built with `include_neutron=True`:

- `"prem"`: the canonical PREM loader computes a layer-appropriate $\langle Z/A\rangle$ to convert tabulated mass density to n_e (implbox above); the complementary neutron fraction $1 - \langle Z/A\rangle$ gives n_n from the very same tabulated mass density, at no extra physical assumption – `load_prem500_neutron_profile` fits it with the same piecewise-linear-in-r^2 shells (identical `rj` boundaries) as the electron-density loader.
- `"even_power"`: has no composition information of its own (it is a bare polynomial fit to tabulated $n_e(r)$, next section), so its neutron companion is instead read from a separate, independently fitted table, `data/density/earth_density_nn.csv`, matching the default electron table's shell boundaries exactly.

`EarthProfile.density_n_x_eta(x, eta)` / `call_neutron(r, eta)` evaluate this companion table pointwise, mirroring `density_x_eta`/`call` above; they raise a clear `ValueError` if the selected profile was not built with `include_neutron=True` (e.g. the default `"even_power"` construction). See Chapter 2 (`potential.py` section) for the physics n_n enables, and the `evolutor.py`/`probability.py` sections below for how it is threaded through the analytical and numerical Earth pipelines via `include_matter_nc`.

9.2 geometry.py — nadir angle and trajectory coordinates

`medium/earth/geometry.py`

Pure geometric conversions: detector depth and nadir/zenith angle to dimensionless trajectory coordinates. Computes no Hamiltonians, evolutors, or probabilities.

Nadir Angle and Its Relation to the Zenith Angle

Two angle conventions coexist in this chapter, both measured at the detector. The **zenith angle** θ is measured from the local upward vertical: $\theta = 0$ is a neutrino arriving from directly overhead (shortest possible path, no Earth crossing), $\theta = 180°$ is a neutrino arriving from directly below (having crossed the full Earth diameter). The **nadir angle** η used internally throughout this chapter is measured from the opposite direction (straight

down, toward the Earth's centre), so the two are exact complements:

$$\eta = \pi - \theta \,. \tag{9.1}$$

$\eta = 0$ is therefore the longest possible path (straight through the centre) and $\eta = \pi/2$ is the horizon (grazing incidence, zero Earth material for a surface detector). `earth_evolutor_from_zenith` (Section 9.3) applies exactly this conversion for callers who prefer to work with θ in degrees.

Theoretical Foundation

For a detector displaced below the surface by depth d, define the dimensionless **detector radius**

$$r_d = 1 - \frac{d}{R_\oplus} \quad (\leq 1), \tag{9.2}$$

and the **surface-equivalent nadir angle** η' – the nadir angle at which a ray from the Earth's centre through the detector would exit the (undisplaced) surface, obtained from the sine rule applied to the chord geometry:

$$\eta' = \arcsin(r_d \sin\eta) \,. \tag{9.3}$$

η' is what `EarthProfile.shells_x/trajectory_profile` (Section 9.1) actually consume, so that a buried detector sees exactly the shells a surface detector at the corresponding η' would. The detector's own position along the trajectory chord is

$$x_d = r_d \cos\eta, \tag{9.4}$$

and, for shallow trajectories that never reach a tabulated shell boundary (Case B, Section 9.3), the total matter path length from the surface to the detector is

$$\Delta x_B = r_d \cos\eta + \sqrt{1 - r_d^2 \sin^2\eta} \,. \tag{9.5}$$

`classify_eta_regions` partitions any $\eta \in [0, \pi]$ into three disjoint outcomes: **above horizon** ($\eta \in [\pi/2, \pi]$ for a surface detector only – no Earth material at all, only relevant when $d = 0$), **Case A** ($\eta \in [0, \pi/2)$, shell-crossing), and **Case B** ($\eta \in [\pi/2, \pi]$ for $d > 0$, shallow detector-near paths) – see Section 9.3.

9.3 `evolutor.py` — Case A and Case B evolutors

`medium/earth/evolutor.py`

`earth_evolutor`: the full flavour-basis Earth evolution operator, dispatching every trajectory to one of two geometrically distinct constructions.

Case A versus Case B

	Case A ($0 \le \eta < \pi/2$)	**Case B** ($\eta \ge \pi/2$, $d > 0$)
Path	Deep, crosses one or more tabulated Earth shells	Shallow, surface to a buried detector, no shell crossed
Density	Piecewise, one perturbative segment per crossed shell	Single constant value, sampled at the mid-depth radius
Segments	Many (near half + mirrored far half)	One
Evolutor	Composed product of many perturbative segment evolutors (Chapter 6)	A single perturbative segment evolutor

Theoretical Foundation

Case A: shell-crossing trajectories

The chord identity $r^2 = x^2 + \sin^2\eta$ makes the physical density profile *even* in x: $n_e(x) = n_e(-x)$, since both $\pm x$ map to the same r. This lets the chord be built from two mirror-symmetric halves without recomputing the shell geometry twice:

1. **Near half** (point of closest approach, $x = 0$, to the detector, $x = x_d$): the crossed shells between 0 and x_d are ordered outermost-first; each is turned into a perturbative segment evolutor (Chapter 6) via the selected density model (Section 9.1), and non-crossed shells are masked to the identity. These compose (Chapter 2) into the detector-side half-evolutor $U_{\text{half,det}}$.
2. **Far half** (mirrored, $x \in [-x_d^{\max}, 0]$): recomputed *directly* from the mirrored segment endpoints $(x_1, x_2) \to (-x_2, -x_1)$, **not** derived from the near half by transpose or reordering. Although the density profile is even, the first-order perturbative correction's oscillatory residual integral (Chapter 6) depends on the segment endpoints through a complex phase that is *not* itself even, so the mirrored segment's evolutor genuinely differs and must be built through the same segment-evolutor pipeline independently. A transpose-based shortcut would only be valid when the reduced Hamiltonian is real (i.e. $\delta_{14} = 0$ for the 3+1 sterile extension, Chapter 4); this direct recomputation is correct unconditionally, at the cost of one extra pass.

The two halves combine into the reduced-basis Earth evolutor,

$$U_{\text{red}}^{\text{Earth}} = U_{\text{half,det}}\, U_{\text{half,far}}\,, \tag{9.6}$$

which is then dressed into the flavour basis via `pmns.flavour_basis` (Chapters 2 to 4), correct for the Standard Model, NSI, the 3+1 sterile extension, or any combination.

Case B: shallow detector-near trajectories

For $\eta \ge \pi/2$ with a buried detector, the path from the surface to the detector never reaches a tabulated shell boundary. The density is approximated by its value at the mid-depth radius $r_{\text{mid}} = 1 - d/(2R_\oplus)$, treated as a single constant-density perturbative segment of length Δx_B (Section 9.2):

$$U_{\text{red}}^{\text{Earth}} = \texttt{evolutor\_perturbative\_segment}\big(n_e(r_{\text{mid}}),\ \Delta x_B\big)\,, \tag{9.7}$$

again dressed to the flavour basis exactly as in Case A. Since the profile is exactly constant on this single segment, the perturbative first-order correction (Chapter 6) is identically zero and U_0 alone is exact for this segment.
Entries with $\eta \in [\pi/2, \pi]$ *and* $d = 0$ (above the horizon for a surface detector) never enter either construction: they are left as the identity operator directly by `earth_evolutor`, since no Earth material is crossed at all.

Implementation in `tpeanuts`

`earth_evolutor(profile_earth, oscillation, E, eta, depth_m, ...)`: classifies every (E, η) entry (`classify_eta_regions`), routes Case A entries through `_earth_evolutor_case_a_batched` and Case B entries through `_earth_evolutor_case_b_batched`, and optionally re-projects each result onto the nearest unitary matrix (`reunitarize`, via `util.math.project_to_unitary`) to absorb the small numerical drift the perturbative expansion can introduce. `earth_evolutor_from_zenith` is the zenith-angle convenience wrapper (Section 9.2). Both support the 3-flavour Standard Model and the 3+1 sterile extension transparently, sizing the identity operator and every intermediate tensor by `oscillation.pmns.n_flavours`. Both also accept `include_matter_nc=False`, forwarded down to every `evolutor_perturbative_segment` call in both Case A and Case B (Chapter 6): when `True`, the Case B constant segment additionally samples `EarthProfile.call_neutron` at the mid-depth radius (mirroring the electron-density sample above it), enabling the 3+1 sterile neutral-current term end-to-end. `False` (the default) reproduces the pre-existing CC-only behaviour exactly, and requires no neutron-density support from the selected profile model.

9.4 `probability.py` — analytical versus numerical

`medium/earth/probability.py`

`earth_probability_state`: dispatches between the perturbative-analytical and the exact-numerical Earth probability pipeline. *`earth_probability_transition`* *and* *`earth_probability_integrated`* *build on it (full transition matrix and spectrum-weighted energy average, respectively).*

Analytical versus Numerical

	`method="analytical"`	`method="numerical"`
Evolutor	`earth_evolutor` (Section 9.3): a handful of perturbative segments per shell, closed-form residual integrals	`evolutor_numerical` (Chapter 5): `nsteps` matrix exponentials sampled along a fine `Trajectory`
Accuracy driver	First-order truncation per segment (Chapter 6); exact only if the density is well fit by the chosen model	Exact per segment; accuracy set purely by `nsteps` (Chapter 5)
Density model	Perturbative even-power/PREM shell coefficients (Section 9.1)	Any `EarthProfile.density_x_eta`, including the tabulated (interpolated) option
Batching	Fully vectorised over (E, η)	Currently scalar η per call
Extras	`reunitarize` drift correction	`full_oscillation` returns the whole path, not just the endpoint

The numerical pipeline is the reference: it makes no perturbative assumption, so it is what the analytical pipeline is validated against (matching the general relationship between Chapters 5 and 6).

Implementation in `tpeanuts`

- `earth_probability_transition(profile_earth, oscillation, E_MeV, eta, depth_m, ...)`: builds U_{Earth} via `earth_evolutor` and returns the full $|U_{\text{Earth}}[\beta, \alpha]|^2$ matrix via `core.common.probability.probability_transition` (Chapter 2); uses the analytical evolutor only, since the numerical segment pipeline below only ever evolves an explicit initial state and has no matrix-level counterpart.
- `earth_probability_state_analytical(nustate, profile_earth, oscillation, E_MeV, eta, depth_m, massbasis, ...)`: builds U_{Earth} via `earth_evolutor` and applies the coherent or incoherent probability formula of Chapter 2 depending on `massbasis`.
- `earth_probability_state_numerical(nustate, profile_earth, oscillation, E_MeV, eta, depth_m, ...)`: builds a scalar `Trajectory` (`build_earth_trajectory`, Chapter 5) – Case A samples `EarthProfile.density_x_eta` along the chord, Case B repeats the constant mid-depth density – then calls `evolutor_numerical` and the same probability dispatch; `full_oscillation=True` additionally returns the probability at every sampled point, not only the final one.
- `earth_probability_state(...)`: the public dispatcher selecting one of the two pipelines by the string `method`.
- `earth_probability_integrated(nustate, profile_earth, oscillation, E_MeV, eta, depth_m, spectrum, ...)`: builds `earth_probability_state`

on an energy grid at a fixed nadir angle and averages it with core.common.probability.probability_integrated (Chapter 2), weighted by an explicit production spectrum. This time-averages over energy, not over nadir angle, and is therefore distinct from earth_probability_exposure below (Section 9.6).

Implementation in tpeanuts

Every function above (and earth_probability_transition) accepts include_matter_nc=False, forwarded to earth_evolutor for the analytical pipeline or sampled directly from profile_earth.call_neutron alongside the electron-density sample for the numerical one (both Case A and the Case B constant-density fallback). Requesting it with a profile model that lacks neutron-density coefficients (Section 9.1) raises a clear ValueError rather than silently falling back to CC-only; requesting it for a 3-flavour oscillation.pmns is silently ignored (Chapter 2).

9.5 flux.py — Earth flux

medium/earth/flux.py

earth_flux_state, earth_flux_integrated, and earth_flux_exposure: apply core.common.flux normalisations, energy integration, and exposure integration (respectively) on top of earth_probability_state and earth_probability_exposure (Section 9.6).

Implementation in tpeanuts

- earth_flux_state(nustate, profile_earth, oscillation, E_MeV, eta, depth_m, flux, spectrum, method, ...): calls earth_probability_state then core.common.flux.flux_state (Chapter 2), propagating the (probabilities_along_path, x) pair through unchanged when method="numerical" and full_oscillation=True.
- earth_flux_integrated(nustate, profile_earth, oscillation, E_MeV, eta, depth_m, flux, spectrum, ..., energy_dim=-2): builds earth_flux_state on an energy grid at a fixed nadir angle and integrates it with core.common.flux.flux_integrated, returning an unnormalised physical rate. Distinct from earth_flux_exposure below, which integrates over the nadir angle instead of energy.
- earth_flux_exposure(nustate, profile_earth, oscillation, E_MeV, depth_m, flux, spectrum, exposure, ...): instead starts from earth_probability_exposure (Section 9.6): the angular exposure average happens at the probability level, before the flux normalisation is applied.

All three accept include_matter_nc=False, forwarded unchanged to the underlying probability function.

9.6 Nadir-angle exposure: `exposure_math.py`, `exposure_table.py`, `exposure_integration.py`, `exposure_io.py`

`medium/earth/exposure_*.py`

Together, these four modules build a time-averaged nadir-angle exposure weight $W(\eta)$ for a detector at fixed geographic latitude, and integrate Earth probabilities against it.

Theoretical Foundation

The physical exposure problem

A detector at fixed geographic latitude λ observes a given nadir angle η only during specific windows of each day, because the visible nadir angle towards a fixed astrophysical/atmospheric source direction sweeps as the Earth rotates and as the Sun's apparent declination $\delta_\odot(t)$ (approximated as varying sinusoidally with the orbital inclination $\iota \approx 23.44°$ over the year) changes with the season. The exposure weight

$$W(\eta) = \int_{d_1}^{d_2} [\text{fraction of day } d \text{ spent at nadir angle } \eta]\, \mathrm{d}d \tag{9.8}$$

integrates that daily observability window over a requested day-of-year range $[d_1, d_2] \subseteq [0, 365]$ (the full year by default), giving a weight function used to time-average Earth-crossing probabilities:

$$P_{\text{exp}}(E) = \int_0^\pi W(\eta)\, P_{\text{Earth}}(E, \eta)\, \mathrm{d}\eta\,. \tag{9.9}$$

From an elementary rate to $W(\eta)$: the elliptic-integral machinery

The instantaneous rate at which nadir angle η is observed has no elementary closed-form antiderivative in the day-phase variable; the project evaluates it via **incomplete elliptic integrals of the first kind**, $F(\varphi, m)$ (`util.math.ellipf_incomplete`), through three nested closed-form functions, entirely in complex arithmetic to remain well-defined across the geometry's branch cuts (the physical result is always the real part of a difference):

- `IndefiniteIntegralDay(T, eta, lam)`: the antiderivative, in the day-phase variable $T \in [-1, 1]$, of the instantaneous nadir-observation rate, expressed through $F(\varphi, m)$ with an elliptic amplitude φ and parameter m built from $\sin\iota$, $\sin(\eta \mp \lambda)$, and T (the exact closed form is a several-term rational combination of square roots of these quantities, verified numerically against direct integration in the test suite rather than reproduced symbol-by-symbol here).
- `IntegralAngle(eta, lam, a1, a2)`: evaluates `IndefiniteIntegralDay` at the two endpoints of the sub-interval of T consistent with *both* the day/night geometry at nadir angle η *and* a single year-angle window $[a_1, a_2]$ (found via `util.math.intersection` of the relevant validity intervals), returning zero when no valid sub-interval exists.
- `IntegralDay(eta, lam, d1, d2)`: converts the requested day-of-year window to year-angle radians ($a = 2\pi d/365$), splits it at the half-year boundaries (the day/night geometry is evaluated separately on each half via a sign reflection), and sums the `IntegralAngle` contributions – this *is* $W(\eta)$ for one nadir angle.

`make_eta_grid(ns, daynight=...)` builds the uniform grid on which $W(\eta)$ is tabulated, optionally restricted to the day half (larger η, closer to π) or night half (smaller η, closer to 0, where core-crossing trajectories concentrate) of the full $[0, \pi]$ range.

Implementation in `tpeanuts`

- **`ExposureParameters`**: selects the exposure backend (`exposure_source`: `"math"` direct integration, `"cache"` a precomputed torch file, `"csv"` a tabulated file in any of the Nadir/Zenith/CosZenith conventions, or `"legacy"` delegating to legacy `peanuts` for validation), the detector latitude, the day-of-year window, the day/night filter, and grid size.
- `build_nadir_exposure(exposure, context, normalized)` (`exposure_table.py`): dispatches to the selected backend and returns a **`NadirExposureTable`** (an (η, W) pair), optionally normalised to unit trapezoidal integral so it reads as a probability density over nadir angle.
- `integrate_exposure(probabilities_eta, eta, exposure)`: the $P_{\text{exp}}(E)$ formula above, evaluated by the trapezoidal rule (`torch.trapezoid`).
- `earth_probability_exposure(...)` (`exposure_integration.py`): the end-to-end entry point – builds the exposure table, evaluates `earth_probability_state` (Section 9.4) over the full η grid (optionally chunked via `chunk_eta` to bound memory), and integrates against $W(\eta)$. Accepts `include_matter_nc=False`, forwarded to every `earth_probability_state` call.
- `exposure_io.py`: CSV and torch-cache read/write helpers for the `"csv"`/`"cache"` backends above – pure I/O, no additional physics.

Chapter 10

`medium.atmosphere` — atmospheric neutrino propagation

Atmospheric neutrinos are produced by cosmic-ray air showers at a range of altitudes above the Earth's surface and propagate down through the atmosphere – a comparatively low-density medium, but one whose geometry (production height, zenith angle, detector depth) is considerably richer than the single-scale exponential often assumed. This chapter follows the same profile/geometry/evolutor/probability/flux structure established in Chapter 7, adapted to that geometry, plus a production-altitude density layer (`density.py`) and an atmospheric slant-depth layer (`depth.py`) with no direct analogue in the previous media chapters.

10.1 `profile.py` and `density.py` — the atmosphere density model

`medium/atmosphere/density.py, profile.py`

Pluggable production-to-surface electron-density backends, and ***`AtmosphereProfile`****, the trajectory container consumed by the numerical evolutor (Chapter 5).*

Implementation in `tpeanuts`

Unlike the Earth chapter's single PREM-derived source (Section 9.1), `atmosphere_density` offers a genuine choice of density backends, selected by `atmosphere_density_source`:

- `"exponential"` (**the default**): the barometric formula $\rho(h) = \rho_0 e^{-h/H}$, with $\rho_0 = 1.225 \times 10^{-3}$ g/cm^3 (U.S. Standard Atmosphere sea-level density) and scale height $H = 8.4$ km – a single-parameter approximation, adequate when only the coarse exponential fall-off matters.
- `"nusquids"`/`"earthatm"`: the external nuSQuIDS `EarthAtm` density formula (`external.nusquids`), for bit-comparable cross-checks against that reference implementation.
- `"file"`: linear interpolation of a user-supplied two-column altitude/density table.
- `"mceq"`: densities consistent with an initialised MCEq atmosphere object [8], for internal consistency with an MCEq-generated production flux.
- `"msis"`/`"pymsis"`: the NRLMSIS 2.0 empirical atmosphere model [5], the most physically detailed option (season-, location-, and space-weather-dependent), via the optional `pymsis` backend.

Every backend returns mass density $\rho(h)$ in g/cm^3; electron density is obtained via $n_e = Y_e\,\rho \times$ (mol/g conversion), with the electron fraction Y_e read from the backend's own configuration (PyMSIS) or from the shared default $Y_e = 0.5$ (air, elsewhere) – exactly the same mass-to-electron-density conversion pattern as Chapter 2/Section 9.1. `atmosphere_density` also accepts `density_type="neutron_density"`, returning $n_n = (1 - Y_e)\,\rho \times$ (mol/g conversion) from the very same Y_e/ρ already computed for n_e – no

new backend logic, since air's composition is fully captured by the single scalar Y_e already in use. This enables the 3+1 sterile neutral-current term (Chapter 2) for every density backend uniformly.

Implementation in tpeanuts

`AtmosphereParameters` bundles the density-backend selection above with the numerical trajectory settings (`nsteps`, sampling `method`) and the analytical-mode fit settings (`perturbative_segments`, `perturbative_degree`, next section); `matter=False` forces zero density everywhere (a pure-vacuum check). `AtmosphereProfile` builds the production-to-surface `Trajectory` (Chapter 5) from (`h_km`,`theta_deg`,`depth_km`) via the geometry of the next section, and samples the selected density backend once per segment. `AtmosphereParameters.include_matter_nc=False` additionally makes `AtmosphereProfile` sample `density_type="neutron_density"` alongside n_e, exposed as `n_n_molcm3` (`None` when not requested); since `atmosphere_density` supports every backend uniformly (implbox above), unlike Chapter 9 there is no profile model for which this can fail to be available.

10.2 geometry.py and depth.py — angles, altitude, and atmospheric depth

`medium/atmosphere/geometry.py, depth.py`

Zenith/nadir angle conversions, surface/detector angle mapping, path-length geometry, and the altitude ↔ atmospheric-depth relation.

Two Angles and Two "Depths"

This chapter uses **three** angle labels and **two** unrelated quantities both called "depth" – keeping them apart is essential:

- θ (**detector zenith angle**, the MCEq convention): measured at the detector from the local upward vertical; $\theta = 0°$ is straight down (shortest path), $\theta = 90°$ is horizontal.
- η (**nadir angle**, Chapter 9's convention): $\eta = \pi - \theta$, identical relation and identical meaning to the one used throughout the Earth chapter.
- α (**surface zenith angle**): the zenith angle the same ray would have if measured by an observer *at the Earth's surface* directly above/along the trajectory, rather than at the (possibly buried) detector – related to θ by impact-parameter conservation (below), not simple equality.
- `depth_km` or `depth_m` (**detector depth**): how far below the Earth's surface the detector sits – a *geometric* length, identical in meaning to Earth's d (Chapter 9).
- X (**atmospheric depth**, `depth.py`): a *column density* in g/cm^2 (mass per unit area integrated along the path), **not** a length – the physical quantity most directly

tied to how many interaction lengths of atmosphere a cosmic-ray shower (and hence its neutrino production point) has traversed. It is unrelated to detector depth beyond sharing the English word.

Theoretical Foundation

Angle conversions

The nadir/zenith relation is exactly the one used throughout Chapter 9:

$$\eta = \pi - \theta \,. \tag{10.1}$$

The detector zenith angle θ and the surface zenith angle α of the *same* straight-line ray are related by conservation of the impact parameter (the perpendicular distance from Earth's centre to the ray is the same whether measured from the detector's radius $r_d = R_\oplus - d$ or the surface radius $R_\oplus$):

$$r_d \sin\theta = R_\oplus \sin\alpha \qquad \Longleftrightarrow \qquad \theta = \arcsin\!\Big(\frac{R_\oplus}{r_d}\sin\alpha\Big),\ \ \alpha = \arcsin\!\Big(\frac{r_d}{R_\oplus}\sin\theta\Big). \tag{10.2}$$

Because $R_\oplus/r_d > 1$ for a buried detector, not every surface angle maps to a valid detector ray: α must stay below $\alpha_{\max} = \arcsin(r_d/R_\oplus)$ for the first arcsine to remain defined (`alpha_max_for_detector_depth`). For a surface detector ($d = 0$), $r_d = R_\oplus$ and $\alpha \equiv \theta$ trivially.

Path lengths

For a neutrino produced at altitude h and detected at zenith angle θ with detector depth d (so $r_d = R_\oplus - d$), the straight-line law of cosines applied to the triangle (Earth centre, detector, production point) gives the total detector-to-production distance,

$$L_{\text{total}} = -r_d\cos\theta + \sqrt{(R_\oplus + h)^2 - r_d^2\sin^2\theta}\,, \tag{10.3}$$

and, setting $h = 0$, the purely **underground** part of the same ray (detector to surface),

$$L_{\text{underground}} = -r_d\cos\theta + \sqrt{R_\oplus^2 - r_d^2\sin^2\theta}. \tag{10.4}$$

The **atmospheric** path length actually carrying matter effects is the difference, $L_{\text{atm}} = L_{\text{total}} - L_{\text{underground}}$ (`atmosphere_path_length`); `atmosphere_path_grid` samples altitude along it via the same law-of-cosines relation evaluated at distance s from the detector,

$$r(s) = \sqrt{r_d^2 + s^2 + 2r_d s\cos\theta}\,, \qquad h(s) = r(s) - R_\oplus. \tag{10.5}$$

Vertical and slant atmospheric depth

Independently of the path-length geometry above, the atmospheric depth X (the column density integral, used e.g. to tabulate cosmic-ray shower development or MCEq/Honda fluxes rather than oscillation propagation directly) is

$$X_{\text{vertical}}(h) = \int_h^\infty \rho(h')\,\mathrm{d}h'\,, \qquad X_{\text{slant}}(h,\alpha) = \frac{X_{\text{vertical}}(h)}{\cos\alpha}, \tag{10.6}$$

the second equation being the plane-parallel approximation (valid for α not too close to the horizon, where the Earth's curvature would need to be accounted for); α here is the *surface* zenith angle defined above, since the plane-parallel picture is anchored to a flat patch of atmosphere.

Implementation in `tpeanuts`

- `theta_to_eta` / `eta_to_theta`: $\eta = \pi - \theta$ and its inverse.
- `alpha_surface_to_theta_detector` / `theta_detector_to_alpha_surface` / `alpha_max_for_detector_depth`: the impact-parameter mapping above and its domain limit.
- `total_path_length` / `underground_path_length` / `atmosphere_path_length`: the three path-length formulas above.
- `altitude_along_detector_path` / `atmosphere_path_grid`: the $r(s)$ sampling formula above, and the resulting distance/altitude grid used to build an `AtmosphereProfile` trajectory.
- `atmosphere_vertical_depth(h_km, rho_gcm3)` / `atmosphere_slant_depth(X_vertical, alpha_deg)` (`depth.py`): the $X_{\text{vertical}}/X_{\text{slant}}$ formulas above, the first via trapezoidal integration from the top of the supplied grid downward.
- `compute_dXdh` / `interpolate_flux_at_Xobs` (`depth.py`): the depth-profile derivative and a depth-tabulated-flux interpolator, both backend-neutral utilities for working with $\Phi(E, X, \alpha)$ tables from any external source.

10.3 `evolutor.py` — atmosphere evolution operators

`medium/atmosphere/evolutor.py`

`atmosphere_evolutor`: dispatches between a numerical (default) and an analytical perturbative atmosphere evolutor.

Implementation in `tpeanuts`

The same numerical/analytical distinction of Chapter 9 applies here, with the **default reversed** (`method="numerical"` by default, versus Earth's `"analytical"` default): the atmosphere's low density and short path make the numerical matrix-exponential pipeline (Chapter 5) cheap enough to use by default.

- `atmosphere_evolutor_numerical(...)`: builds an `AtmosphereProfile` trajectory (Section 10.1) and calls `evolutor_numerical` (Chapter 5) directly on its sampled density; entries with $L_{\text{atm}} \leq 0$ (grazing/no-path geometries) are masked to the identity.
- `atmosphere_evolutor_analytical(...)`: *fits* a local polynomial density model on the fly – splits the atmospheric path into `perturbative_segments` equal sub-intervals, samples the density backend at `perturbative_degree`+1 Chebyshev-like nodes per sub-interval (`q`$\in [-1, 1]$, centred/rescaled), and builds an `AtmospherePolynomialProfile` (Chapter 6, `atmosphere` model) from those samples, then evolves it with `evolutor_perturbative_segment` (Chapter 6) exactly as the Earth analytical pipeline does. Unlike Earth's tabulated shell coefficients, the polynomial here is fitted *per call*, from whichever density backend is selected (Section 10.1) – the price of supporting arbitrary density sources with the same closed-form perturbative machinery.

- atmosphere_evolutor(...): the public dispatcher, mirroring earth_probability_state (Section 9.4) in spirit.

Implementation in tpeanuts

Both evolutors accept atmosphere.include_matter_nc=False (Section 10.1), enabling the 3+1 sterile neutral-current term (Chapter 2). The numerical evolutor forwards AtmosphereProfile's sampled n_n_molcm3 straight through to evolutor_numerical as n_n_mol_cm3. The analytical evolutor fits a *second*, independent AtmospherePolynomialProfile to a neutron-density sample at the same nodes as the electron-density fit, and passes its coefficients as coefficients_n into the same segment model (Chapter 6), then calls evolutor_perturbative_segment with include_matter_nc=True. Both default to False, reproducing the pre-existing CC-only behaviour exactly.

10.4 probability.py — atmosphere-surface probabilities

medium/atmosphere/probability.py

atmosphere_probability_transition and atmosphere_probability_state: final flavour probabilities at the Earth surface from an atmosphere evolution operator. atmosphere_probability_integrated, atmosphere_probability_integrated_angular, and atmosphere_probability_integrated_height build on atmosphere_probability_state to average over energy, the full zenith range, and production height, respectively.

Implementation in tpeanuts

Structurally identical to Chapters 7 and 9: atmosphere_probability_transition(oscillation, E_MeV, h_km, theta_deg, depth_km, method, ...) returns the full transition-probability matrix $P = |S_{\mathrm{atm}}|^2$ (Chapter 2); atmosphere_probability_state(nustate, ..., massbasis, ...) dispatches between the coherent formula ($\psi_{\mathrm{surface}} = S_{\mathrm{atm}}\psi_{\mathrm{initial}}$, $P_\alpha = |\psi_{\mathrm{surface},\alpha}|^2$) and the incoherent formula ($P_\alpha = \sum_i |(S_{\mathrm{atm}}U_{\mathrm{PMNS}})_{\alpha i}|^2 w_i$) of Chapter 2, exactly as vacuum_probability_state and earth_probability_state do, using whichever evolutor atmosphere_evolutor (Section 10.3) returns. Because every function in this section takes the whole AtmosphereParameters bundle as its atmosphere argument rather than exploding it into individual keywords, include_matter_nc (previous section) is already available at every level here with no further plumbing.

Approximations and Limitations

A real, previously uncaught 3-flavour-only bug. atmosphere_probability_state used to crash outright for a 4-flavour (3+1 sterile) initial state, even though atmosphere_evolutor underneath it already built the correct 4×4 operator: a shared broadcasting helper (util.type.broadcast_last3, used identically in Chapter 7) hard-coded its input's final dimension to exactly 3. The fix generalises the helper to accept 3 *or* 4 (renamed broadcast_flavour_vector to match, since it is no longer 3-specific) and is shared by medium.vacuum.probability, medium.vacuum.evolutor, and this module. No test exercised a sterile initial state through this function before the fix.

Implementation in `tpeanuts`

Atmospheric probabilities depend on three axes at once (E, production height h, zenith angle θ), so this medium is the only one with three dedicated `*_integrated` reductions, each delegating to the corresponding Chapter 2 primitive:

- `atmosphere_probability_integrated(nustate, ..., spectrum, integrate_angular, integrate_height, ...)`: **always** averages over energy (spectrum-weighted, via `core.common.probability.probability_integrated`), then optionally chains a height average and/or an angular average by hand, each a further, independent reduction rather than one fused operation.
- `atmosphere_probability_integrated_angular(nustate, ...)`: averages over the full zenith range via `core.common.probability.probability_integrated_angular` (geometric $\sin\theta$ weight).
- `atmosphere_probability_integrated_height(nustate, ..., production_flux, ...)`: averages over production height, weighted by the height-resolved production flux, again via `core.common.probability.probability_integrated` – height is not a universal Chapter 2 concept, so this wrapper lives here, even though it reuses that module's generic weighted-average primitive.

Every reduction's `*_dim` parameter defaults to -2 ("the axis immediately before flavour"), matching Chapter 2's own convention; for the composite function's fixed energy-then-height-then-angle chaining order to line up with that default at every step, the input grid should be laid out as $(\ldots, \theta, h, E, \text{flavour})$.

10.5 `flux.py` — atmosphere flux

`medium/atmosphere/flux.py`

`atmosphere_flux_state` (surface flux from `atmosphere_probability_state`) and three `*_integrated` variants mirroring the probability-layer reductions above.

Implementation in `tpeanuts`

Like `probability.py` above, every function here takes the whole `atmosphere` (`AtmosphereParameters`) bundle, so `include_matter_nc` is available with no further plumbing.

- `atmosphere_flux_state(nustate, oscillation, E_MeV, h_km, theta_deg, flux, spectrum, depth_km, ...)`: the standard pattern – `atmosphere_probability_state` then `core.common.flux.flux_state` (Chapter 2).
- `atmosphere_flux_integrated(..., integrate_angular, integrate_height, ...)`: always integrates over energy via `core.common.flux.flux_integrated` (an unnormalised rate, unlike the probability-layer average), then optionally chains a plain trapezoidal height integral and/or an angular integral (`core.common.flux.flux_integrated_angular`) – no separate height weight is needed here, since `flux` already carries whatever height dependence it has.

- `atmosphere_flux_integrated_angular(...)`: integrates over the full zenith range via `core.common.flux.flux_integrated_angular`.
- `atmosphere_flux_integrated_height(..., production_flux, ...)`: integrates over production height using the tabulated production-flux data. Atmosphere-specific and, unlike the three functions above, calls no `core.common` function directly: it combines `atmosphere_probability_integrated_height` (the production-flux-weighted average probability over height, $\langle P \rangle_h$) with the plain trapezoidal integral of `production_flux` itself over height, $\Phi_h = \int \text{production\_flux}(h)\,\mathrm{d}h$; the product $\langle P \rangle_h \, \Phi_h$ equals $\int P(h)\,\text{production\_flux}(h)\,\mathrm{d}h$ exactly, by the definition of the weighted average.

Detector Composition Lives in the Pipeline Layer

Unlike Chapter 9, `medium.atmosphere` only ever propagates production → surface: it has no notion of a detector, and no `validation.py`. Weighting a surface or detector probability by production-flux data, integrating a detector flux over production height, and summing the incoherent contributions from several produced flavours are all pipeline-level concerns, composed in `pipeline.atmosphere_earth` (`detector_flux_from_production`, `integrate_detector_flux_over_height`, `sum_detected_flavours`) on top of `pipeline.atmosphere_earth.propagate_surface_to_detector`, which applies the Earth evolutor of Chapter 9 to this medium's coherent surface state via ordinary matrix-vector composition, not the matrix-matrix composition used internally by Section 9.3.

PART IV

High-Level Pipelines — `pipeline/`

Chapter 11

pipeline.solar — solar production to surface

This part covers the five `pipeline.*` workflows: user-facing orchestration layers sitting above `core` and `medium` (Chapters 2 and 4 to 10). None of the five chapters that follow introduce new physics: every pipeline function's job is to prepare inputs, call the physics blocks already derived in the previous parts of this document, integrate or aggregate the resulting observables, and return them bundled in a single result object – so, unlike earlier chapters, none of these sections carry a blue **Theoretical Foundation** box except where a pipeline composes two physics stages in a way that is not already written down elsewhere. Each chapter instead closes with a teal **Usage Example** box: a complete, runnable Python snippet showing how that pipeline's entry points are actually called together.

11.1 config.propagation.PropagationConfig — shared pipeline configuration

`config/propagation.py`

`PropagationConfig`: the single configuration object every pipeline function in this part accepts.

Implementation in `tpeanuts`

`PropagationConfig` replaces what used to be 20–38 separate keyword arguments threaded into each pipeline entry point with one dataclass. There is exactly one `PropagationConfig` for the whole project, not one per pipeline: fields a given pipeline does not need (e.g. `exposure/solar` for the atmosphere pipeline, Chapter 14) are simply left at their defaults and unused. Pipeline functions destructure it into the smaller pieces `core/medium` functions actually need (a `RuntimeContext`, an `OscillationParameters`, a built profile object, ...) rather than passing the whole object down, so those lower layers stay decoupled from this pipeline-level container.

Field	Type	Default	Role
runtime	RuntimeContext	required	Torch device/d throughout.
oscillation	OscillationParameters	required	Built pmns ob mass splittings selection (Chapt oscillation_par below.
exposure	ExposureParameters	ExposureParameters()	Nadir-exposure tings (Section 9.6 pipeline.earth/ when averaging ov exposure window atmosphere.
earth	EarthParameters	EarthParameters()	Earth density-prof tion settings (S Used by every pipe Earth-crossing sta
solar	SolarParameters	SolarParameters()	Solar-model const tings (Chapter pipelines only.
atmosphere	AtmosphereParameters	AtmosphereParameters()	Atmosphere de construction setti ter 10). Atmosph only.
production_mode	"point"/"coherent"/"incoherent"	"point"	Solar production- ment (solar pipeli
rho0	Optional[TensorLike]	None	Production (fraction of production_mode
source	Optional[str]	None	Solar source key "pp") for produ in { "incoherent"}.
detector_depth_m	float	0.0	Detector depth Earth's surface, shifts the Earth-c length wherever regeneration stage
reunitarize_earth	bool	False	Project the Eart operator back on est unitary matrix small numerical d
reunitarize_atmosphere	bool	False	Same, for t sphere evolution (atmosphere_evo Independent reunitarize_ear two stages are separately.
analytic_eigenvalues	bool	False	Use the closed dano/Ferrari solution (Chapter of torch.linal for method="a Earth/atmosphere tion; raises if con method="numeric never diagonalises

`PropagationConfig.oscillation_parameters_from_preset(preset_name="_SM_NUFIT52_NO", *, antinu=False, NSI_extension=None, context=None)` is the static factory that builds the `oscillation` field (Chapter 2, Default Configuration box): it looks up `preset_name` in `tpeanuts.config.presets.OSCILLATION_PRESETS`, dispatches to a sterile or plain-SM `PMNS` exactly as described in Chapter 4, and optionally attaches an `NSIConfig` built from `NSI_extension` (Section 4.2). This is the one place in the pipeline layer allowed to depend on concrete SM/BSM/NSI/preset implementations, keeping `OscillationParameters` itself a passive container (Chapter 2).

11.2 pipeline.solar — solar production to surface

pipeline/solar.py

propagate_solar_to_surface: the incoherent solar mass mixture produced inside the Sun, at the solar surface.

Implementation in tpeanuts

- `SolarSurfaceResult` (frozen dataclass): `source`, `E_MeV`, the built `profile` (`medium.solar.profile.SolarProfile`), the source's `production_distribution` (production-radius weights), the incoherent `mass_weights` $w_i(E)$, and the corresponding `flavour_probabilities` – the same quantities `solar_probability_mass`/`probability_incoherent` (Chapters 2 and 8) already compute, simply bundled together with the profile and inputs that produced them.
- `propagate_solar_to_surface(*, E_MeV, config, source, solar_profile=None, method="adiabatic_approximated", legacy_precision=False, include_matter_nc=None)`: builds (or reuses `solar_profile`), reads off the source's production-radius distribution, calls `solar_probability_mass` for the mass weights, then `probability_incoherent` with the identity operator standing in for "no further coherent evolution" to obtain `flavour_probabilities` at the solar surface directly (before any Earth regeneration). `method` and `include_matter_nc` are exactly the ones documented in Chapter 8's `probability.py` section; this function adds no behaviour of its own beyond wiring `PropagationConfig` into that call.

Solar mass mixture at two energies, boron-8 source

```
import torch

from tpeanuts.config.propagation import PropagationConfig
from tpeanuts.pipeline.solar import propagate_solar_to_surface
from tpeanuts.util.context import RuntimeContext

context = RuntimeContext.resolve("cpu", torch.float64)
oscillation = PropagationConfig.oscillation_parameters_from_preset(
    "_SM_NUFIT52_NO", context=context,
)
config = PropagationConfig(runtime=context, oscillation=oscillation)

result = propagate_solar_to_surface(
    E_MeV=[2.0, 8.0],
```

```
    config=config,
    source="8B",
)

print(result.mass_weights.shape)           # torch.Size([2, 3])
print(result.mass_weights.sum(dim=-1))     # tensor([1., 1.], dtype=torch.float64
    )
print(result.flavour_probabilities)        # P(nu_e), P(nu_mu), P(nu_tau) per
    energy
```

The default `method="adiabatic_approximated"` uses the closed-form two-level mixing-angle reduction (Chapter 8); passing `method="adiabatic_exact"` or `method="numerical"` instead – required for NSI or a sterile `oscillation` preset, per the support matrix in Section 4.5 – needs no other change to the call above.

Chapter 12

`pipeline.earth` — Earth surface to detector

`pipeline.earth` propagates an already-known incident state – flavour-basis coherent amplitudes, or incoherent mass-basis weights such as the ones Chapter 11 produces – through the Earth to a detector, over either an explicit nadir-angle grid or the configured day/night exposure window (Section 9.6). It is the Earth leg reused, unchanged, by `pipeline.solar_earth` (Chapter 13); used on its own it also covers any externally-produced incident flux (reactor, accelerator, atmospheric production already reduced to a single flavour, . . .) that only needs Earth regeneration, not a solar production stage.

12.1 `pipeline.earth` — pure Earth workflow

`pipeline/earth.py`

`propagate_earth_to_detector` and `propagate_earth_to_detector_exposure`: Earth-crossing propagation over an explicit nadir-angle grid, or averaged over the configured exposure window.

Implementation in `tpeanuts`

- `EarthDetectorResult` (frozen dataclass): `incident_state`, `incident_basis` (`"flavour"` or `"mass"`), `E_MeV`, the built `profile` (`medium.earth.profile.EarthProfile`), and *either* the explicit-grid pair (`eta`, `exposure`, `probabilities_eta`) *or* the exposure-averaged `probabilities_exposure` – exactly one of the two pairs is populated, matching which of the two functions below built the result.
- `propagate_earth_to_detector(incident_state, *, E_MeV, config, incident_basis="flavour", earth_profile=None, eta=None)`: resolves the nadir-angle grid via `medium.earth.exposure_table.prepare_nadir_exposure` (an explicit `eta` if given, otherwise the one implied by `config.exposure`), then calls `earth_probability_state` (Chapter 9) on the Cartesian product of the energy and eta grids. `config.earth.chunk_eta` splits the eta axis into sub-batches evaluated one at a time (a memory control for large energy–angle grids), concatenated back together before being returned.
- `propagate_earth_to_detector_exposure(incident_state, *, E_MeV, config, incident_basis="flavour", earth_profile=None)`: calls `earth_probability_exposure` (Section 9.6) directly, i.e. propagates and averages over the nadir-angle exposure weight $W(\eta)$ in one step rather than returning a per-angle grid.

Both functions accept `incident_basis="mass"` for an incoherent mass mixture (e.g. Chapter 11's `mass_weights`) and `"flavour"` (the default) for coherent flavour amplitudes, dispatching to `earth_probability_state`/`earth_probability_exposure`'s own `massbasis` flag accordingly.

A pure ν_μ beam crossing the Earth on a nadir-angle grid

```python
import torch

from tpeanuts.config.propagation import PropagationConfig
from tpeanuts.core.common.neutrino import flavour_state
from tpeanuts.pipeline.earth import propagate_earth_to_detector
from tpeanuts.util.context import RuntimeContext

context = RuntimeContext.resolve("cpu", torch.float64)
oscillation = PropagationConfig.oscillation_parameters_from_preset(
    "_SM_NUFIT52_NO", context=context,
)
config = PropagationConfig(runtime=context, oscillation=oscillation)

nu_mu = flavour_state("mu", device=context.device, dtype=context.dtype)

result = propagate_earth_to_detector(
    nu_mu,
    E_MeV=[500.0, 2000.0],
    config=config,
    eta=[0.2, 0.8, 1.4],    # nadir angle in radians, 0 = through the core
)

print(result.probabilities_eta.shape)    # torch.Size([2, 3, 3]): (E, eta,
    flavour)
```

For a detector-exposure average instead of a fixed grid, call `propagate_earth_to_detector_exposure` with the same `nu_mu`/`E_MeV`/`config` – no `eta` argument, since the angular average is taken internally over `config.exposure` (Section 9.6):

The same beam, averaged over the configured detector exposure

```python
from tpeanuts.pipeline.earth import propagate_earth_to_detector_exposure

exposure_result = propagate_earth_to_detector_exposure(
    nu_mu,
    E_MeV=[500.0, 2000.0],
    config=config,
)
print(exposure_result.probabilities_exposure.shape)    # torch.Size([2, 3]): (E,
    flavour)
```

Chapter 13

`pipeline.solar_earth` — solar production composed with Earth regeneration

`pipeline.solar_earth` composes Chapters 11 and 12 into the single workflow most solar-neutrino analyses actually need: production inside the Sun, incoherent propagation to the Earth (the mass mixture decoheres over the Sun–Earth distance, Chapter 2), then Earth-crossing regeneration to a detector. It introduces exactly one piece of genuine composition logic beyond simply calling the two pipelines in sequence: combining the Sun–Earth distance modulation with the detector-exposure choice, derived below.

Theoretical Foundation

Combining the Sun–Earth distance modulation with detector exposure

Solar-model flux tables (Chapter 8) are normalised to 1 AU, but Earth's elliptical orbit makes the physically received flux vary by about ±3.4% over the year, via the instantaneous factor $(1\,\mathrm{AU}/R(\mathrm{date}))^2$ (`medium.solar.geometry.sun_earth_distance_factor`). Exactly one of two ways to apply it is meaningful for a given call, matching whether the detector-exposure average (Section 9.6) is active:

- **No exposure averaging** (`integrate_exposure` resolves to `False`): pass an explicit ISO `date="YYYY-MM-DD"`: the instantaneous factor for that single calendar date is applied to `detector_flux`. Passing `date` together with an exposure-averaged call raises, since a single date is not meaningful once already averaging over a day-of-year window.
- **Exposure averaging active** (`integrate_exposure` resolves to `True`): pass `average_sun_earth_distance=True` instead, which averages the factor uniformly over the *same* day-of-year window already used for the nadir-angle exposure average (`config.exposure.exposure_d1/exposure_d2`, via `sun_earth_distance_factor_averaged`), so one exposure window consistently accounts for both the detector's day/night geometry and the Sun–Earth distance over the same period. Passing it without exposure averaging active raises, for the symmetric reason: there is no day-of-year window to average over.

Omitting both (the default) leaves the result at the solar model's native 1 AU normalisation, exactly as Chapter 11 alone would.

13.1 `pipeline.solar_earth` — composed workflow

`pipeline/solar_earth.py`

`propagate_solar_to_earth_detector`: solar production and incoherent Earth regeneration, without duplicating the intermediate mass-mixture state.

Implementation in tpeanuts

- **SolarEarthDetectorResult** (frozen dataclass): the full solar_surface (**SolarSurfaceResult**, Chapter 11) and earth_detector (**EarthDetectorResult**, Chapter 12) sub-results, plus the composed detector_flux, detector_probabilities (an alias into whichever of earth_detector.probabilities_eta/probabilities_exposure was populated – **the same tensor object**, not a copy), and the optional energy-integrated probabilities_energy_averaged/detector_flux_energy_integrated.
- propagate_solar_to_earth_detector(*, E_MeV, config, source, solar_profile=None, earth_profile=None, eta=None, source_spectrum=None, integrate_exposure=None, integrate_energy=False, solar_method="adiabatic_approximated", legacy_precision=False, include_matter_nc=None, date=None, average_sun_earth_distance=False): calls propagate_solar_to_surface (forwarding solar_method/include_matter_nc for the solar leg only – the Earth leg has no analogous parameter exposed here yet), feeds solar.mass_weights *directly* into propagate_earth_to_detector (incident_basis="mass") or its _exposure variant according to integrate_exposure (None resolves from config.exposure.integrate_exposure), then applies core.common.flux.flux_state with the source's total flux and spectrum (source_spectrum, or profile.spectrum(source, ...) when omitted) to obtain detector_flux. Setting integrate_energy=True additionally reduces both the probabilities and the flux over the energy grid via probability_integrated/flux_integrated (Chapter 2).

Boron-8 solar neutrinos detected on a nadir-angle grid

```
import torch

from tpeanuts.config.propagation import PropagationConfig
from tpeanuts.pipeline.solar_earth import propagate_solar_to_earth_detector
from tpeanuts.util.context import RuntimeContext

context = RuntimeContext.resolve("cpu", torch.float64)
oscillation = PropagationConfig.oscillation_parameters_from_preset(
    "_SM_NUFIT52_NO", context=context,
)
config = PropagationConfig(runtime=context, oscillation=oscillation)

result = propagate_solar_to_earth_detector(
    E_MeV=[5.0, 8.0],
    config=config,
    source="8B",
    eta=[0.4, 1.0],
    integrate_exposure=False,
)

# The Earth leg's incident state IS the solar mass-mixture, not a copy:
assert result.earth_detector.incident_state is result.solar_surface.mass_weights
print(result.detector_flux.shape)   # torch.Size([2, 2, 3]): (E, eta, flavour)
```

Requesting the exposure-averaged detector rate instead only changes two arguments – eta is dropped and integrate_exposure=True takes its place, resolving to propagate_earth_to_detector_exposure internally – and integrate_energy=True additionally collapses the energy axis:

Exposure- and energy-integrated event rate

```
spectrum = torch.tensor([0.4, 0.6], dtype=torch.float64)

rate_result = propagate_solar_to_earth_detector(
    E_MeV=[4.0, 8.0],
    config=config,
    source="8B",
    source_spectrum=spectrum,
    integrate_exposure=True,
    integrate_energy=True,
    average_sun_earth_distance=True,
)
print(rate_result.detector_flux_energy_integrated.shape)    # torch.Size([3])
```

Chapter 14

pipeline.atmosphere — atmospheric production to Earth surface

pipeline.atmosphere is the production-to-surface leg for atmospheric neutrinos: given a tabulated production flux for one produced particle (e.g. Honda/HKKM-style tables, binned in energy, production height, and zenith/off-axis angle), it selects the requested angular bin and coherently propagates that produced flavour through the atmosphere to the Earth surface (Chapter 10). Continuing the resulting surface state through the Earth to a detector is a separate step, covered by Chapter 15.

14.1 pipeline.atmosphere — production selection and surface propagation

pipeline/atmosphere.py

select_production_flux and propagate_atmosphere_to_surface: pick one produced particle/angle from a flux dataset, then propagate it coherently to the Earth surface.

Implementation in tpeanuts

- select_production_flux(flux_data, particle, *, alpha_deg=None, theta_deg=None, angle_index=None, angle_mode="theta", angle_tolerance_deg=None, context=...): given a {particle: {grids, tables, ...}} mapping (as produced by the project's flux-table loaders), locates the angular bin closest to the requested alpha_deg/theta_deg (or an explicit angle_index), and returns a single production dictionary – the energy/height grids, the selected slice of the height-resolved production flux phi_Eh and the angle-integrated phi_E_theta, the resolved theta_deg, and the core.common.neutrino.flavour_index of particle – that every function below consumes. angle_tolerance_deg raises if no angular bin lies close enough to the requested angle, instead of silently returning a distant one.
- AtmosphereSurfaceResult (frozen dataclass): the production dictionary above, the coherent initial_state and surface_states amplitudes, the full evolutor S_atmosphere, surface_probabilities, the built trajectories (Chapter 10, geometry.py section), and – only when production["phi_Eh"] is present – the production-flux-weighted surface_probability_height_averaged, surface_flux_Eh, and their optional height/energy-integrated reductions.
- propagate_atmosphere_to_surface(production, config, *, initial_flavour=None, trajectory_steps=200, integrate_energy=False): builds the coherent initial state (defaulting to production["particle"] itself, i.e. no flavour change at production), evaluates atmosphere_evolutor (Chapter 10) on the full energy–height grid at the production angle, and applies it to obtain surface_states and surface_probabilities. When

the production dictionary carries a height-resolved flux, it is additionally combined with the probabilities via `core.common.flux.flux_state` and `probability_weighted_average`/`flux_integrated_coordinate` (Chapter 2) to give the height-averaged probability and height-integrated flux directly, so a caller that only needs the energy-resolved surface flux does not have to redo that reduction by hand.

Muon-neutrino production at 60° zenith, propagated to the surface

```
import torch

from tpeanuts.config.propagation import PropagationConfig
from tpeanuts.pipeline.atmosphere import (
    propagate_atmosphere_to_surface,
    select_production_flux,
)
from tpeanuts.util.context import RuntimeContext

context = RuntimeContext.resolve("cpu", torch.float64)
oscillation = PropagationConfig.oscillation_parameters_from_preset(
    "_SM_NUFIT52_NO", context=context,
)
config = PropagationConfig(runtime=context, oscillation=oscillation)

# flux_data: {"numu": {...}, "nue": {...}, ...}, as returned by this
# project's Honda/HKKM-style flux-table loader (see medium.atmosphere).
production = select_production_flux(
    flux_data,
    particle="numu",
    theta_deg=60.0,
    angle_tolerance_deg=1.0,
    context=context,
)

surface = propagate_atmosphere_to_surface(
    production,
    config,
    trajectory_steps=200,
)

print(surface.surface_probabilities.shape)          # (E, h, flavour)
print(surface.surface_flux_Eh.shape)                # (E, h, flavour)
print(surface.surface_flux_height_integrated_E.shape)  # (E, flavour)
```

Chapter 15

`pipeline.atmosphere_earth` — atmospheric surface to detector

`pipeline.atmosphere_earth` continues a coherent atmospheric surface state (Chapter 14) through the Earth to a detector. It also hosts the detector-composition helpers that `medium.atmosphere` deliberately does not carry itself (Chapter 10): the atmosphere medium only propagates production → surface, while combining per-flavour detector probabilities with production-flux tables, integrating over production height, and summing incoherent produced-flavour contributions are all pipeline-level composition concerns tied to this surface-to-detector step.

Theoretical Foundation

The production-flux-weighted height average, computed directly

`propagate_atmosphere_grid_to_detector` (below) needs the same production-flux-weighted probability average over production height that Chapter 10 already derives for a single flavour (`atmosphere_probability_integrated_height`), but here for the *full* transition matrix $P_{\beta\alpha}(\theta, E, h)$ across every zenith angle and produced flavour α at once. Rather than calling that single-flavour function once per (θ, α) pair, it evaluates the same weighted-average identity directly on the stacked tensors,

$$\langle P\rangle_h(\theta, E)_{\beta\alpha} = \frac{\int P_{\beta\alpha}(\theta, E, h)\, \phi_\alpha^{\mathrm{prod}}(\theta, E, h)\, \mathrm{d}h}{\int \phi_\alpha^{\mathrm{prod}}(\theta, E, h)\, \mathrm{d}h}, \tag{15.1}$$

with the produced-flavour axis α broadcast against the transition matrix's own two flavour indices via an extra singleton dimension on the weight (`production_flux.unsqueeze(-2)`), so a single pair of trapezoidal integrals (numerator and denominator) produces the height-averaged transition matrix for every (θ, α, β) combination in one call. The corresponding height-*integrated* (not averaged) detector flux, $\Phi_\beta(\theta, E) = \sum_\alpha \int P_{\beta\alpha}\, \phi_\alpha^{\mathrm{prod}}\, \mathrm{d}h$, is obtained from the transition matrix directly via `core.common.flux.flux_transition/flux_integrated_coordinate` (Chapter 2), mirroring the single-flavour identity $\langle P\rangle_h\, \Phi_h = \int P(h)\, \phi^{\mathrm{prod}}(h)\, \mathrm{d}h$ already established in Chapter 10.

15.1 `pipeline.atmosphere_earth` — surface-to-detector and composition helpers

`pipeline/atmosphere_earth.py`

`propagate_surface_to_detector` (one produced flavour, one zenith angle) and `propagate_atmosphere_grid_` (every produced flavour, every zenith angle, full transition matrix), plus the detector-composition helpers `detector_flux_from_production`, `integrate_detector_flux_over_height`, and `sum_detected_flu`

Implementation in tpeanuts

- **AtmosphereEarthDetectorResult** (frozen dataclass): the input `surface` (**AtmosphereSurfaceResult**, Chapter 14), the Earth evolutor `S_earth`, the composed `S_total` ($S_{\text{earth}}\,S_{\text{atmosphere}}$), `detector_states`/`detector_probabilities` for the single produced flavour carried by `surface`, the full `transition_probabilities` matrix, and – when `surface.production` carries a height-resolved flux – the same height-averaged/-integrated fields as **AtmosphereSurfaceResult**, now evaluated at the detector.
- `propagate_surface_to_detector(surface, config, *, profile_earth=None, integrate_energy=False)`: builds (or reuses) the Earth profile, evaluates `earth_evolutor_from_zenith` (Chapter 9) at the production zenith angle, composes it with `surface.S_atmosphere`, and applies the result to `surface.initial_state`.
- **AtmosphereDetectorGridResult** (frozen dataclass): the stacked, multi-angle, full-transition observables returned by the function below – `transition_probability_theta_Eh_beta_alpha`, `probability_height_theta_E_beta_alpha`, `production_flux_theta_Eh_alpha`, `detector_flux_theta_Eh_beta`, the height-, and optionally angle- and energy-integrated fluxes, plus the individual `detector_results` (one **AtmosphereEarthDetectorResult** per zenith angle).
- `propagate_atmosphere_grid_to_detector(production_by_flavour, config, *, flavour_order=None, profile_earth=None, trajectory_steps=200, integrate_angular=False, integrate_energy=False)`: given `{flavour: [production dict per zenith angle]}` (aligned energy/height/angle grids across flavours, missing flavours treated as zero production flux), propagates each produced flavour at each zenith angle in turn, stacks the resulting transition matrices and production fluxes, and evaluates the weighted-average identity derived above.
- `detector_flux_from_production(production_flux, detector_probabilities)` / `integrate_detector_flux_over_height(h_grid_km, detector_flux)` / `sum_detected_flavours(integrated_flux_by_production_flavour)`: the three composition primitives `propagate_atmosphere_grid_to_detector` is built from, also usable directly by a caller assembling a custom multi-source composition – respectively `core.common.flux.flux_state`, a plain trapezoidal height integral, and an incoherent sum over a `{flavour: flux}` mapping (requiring at least one entry and matching shapes across all of them).

Continuing the atmosphere surface state through the Earth

```python
from tpeanuts.pipeline.atmosphere_earth import propagate_surface_to_detector

# `surface` from the pipeline.atmosphere example (Chapter 13): a coherent
# nu_mu state produced at theta_deg=60, already propagated to the surface.
detector = propagate_surface_to_detector(surface, config)

print(detector.detector_probabilities.shape)      # (E, h, flavour)
print(detector.transition_probabilities.shape)    # (E, h, flavour_out,
    flavour_in)
```

Summing several produced-flavour contributions at the detector, once each has already been propagated and integrated over production height, is a plain incoherent sum over a name → tensor mapping:

Combining ν_μ- and ν_e-produced contributions at the detector

```
from tpeanuts.pipeline.atmosphere_earth import (
    integrate_detector_flux_over_height,
    sum_detected_flavours,
)

h_km = surface.production["h_grid_km"]
numu_integrated = integrate_detector_flux_over_height(h_km, detector.
    detector_flux_Eh)
nue_integrated = integrate_detector_flux_over_height(h_km, nue_detector.
    detector_flux_Eh)

total_detector_flux = sum_detected_flavours({
    "numu": numu_integrated,
    "nue": nue_integrated,
})
```

References

[1] C. Biggio, M. Blennow, and E. Fernandez-Martinez. "General bounds on non-standard neutrino interactions". In: *JHEP* 08 (2009), p. 090. eprint: 0907.0097.

[2] P. Coloma, M. C. Gonzalez-Garcia, M. Maltoni, and T. Schwetz. "COHERENT enlightenment of the neutrino dark side". In: *Phys. Rev. D* 96 (2017), p. 115007. eprint: 1708.02899.

[3] DUNE Collaboration, B. Abi, et al. "Prospects for beyond the Standard Model physics searches at the Deep Underground Neutrino Experiment". In: *Eur. Phys. J. C* 81 (2021), p. 322. eprint: 2008.12769.

[4] A. M. Dziewonski and D. L. Anderson. "Preliminary reference Earth model". In: *Phys. Earth Planet. Inter.* 25 (1981), pp. 297–356.

[5] J. T. Emmert et al. "NRLMSIS 2.0: A Whole-Atmosphere Empirical Model of Temperature and Neutral Species Densities". In: *Earth and Space Science* 8 (2021), e2020EA001321.

[6] I. Esteban, M. C. Gonzalez-Garcia, M. Maltoni, I. Martinez-Soler, and T. Schwetz. "Updated constraints on non-standard interactions from global analysis of oscillation data". In: *JHEP* 08 (2018), p. 180. eprint: 1805.04530.

[7] I. Esteban, M. C. Gonzalez-Garcia, M. Maltoni, T. Schwetz, and A. Zhou. "The fate of hints: updated global analysis of three-flavor neutrino oscillations". In: *JHEP* 09 (2020), p. 178. eprint: 2007.14792.

[8] A. Fedynitch, R. Engel, T. K. Gaisser, F. Riehn, and T. Stanev. "Calculation of conventional and prompt lepton fluxes at very high energy". In: *EPJ Web of Conferences*. Vol. 99. 2015, p. 08001. eprint: 1503.00544.

[9] S. Gariazzo, C. Giunti, M. Laveder, and Y. F. Li. "Updated Global 3+1 Analysis of Short-BaseLine Neutrino Oscillations". In: *JHEP* 06 (2017), p. 135. eprint: 1703.00860.

[10] C. Giunti and C. W. Kim. *Fundamentals of Neutrino Physics and Astrophysics*. Oxford University Press, 2007.

[11] T. E. Gonzalo and M. Lucente. "PEANUTS: a software for the automatic computation of solar neutrino flux and its propagation within Earth". In: *Eur. Phys. J. C* 84 (2024), p. 119. eprint: 2303.15527.

[12] Y. Grossman. "Nonstandard neutrino interactions and neutrino oscillation experiments". In: *Phys. Lett. B* 359 (1995), pp. 141–147. eprint: hep-ph/9507344.

[13] Hyper-Kamiokande Proto-Collaboration, K. Abe, et al. "Physics potential of a long-baseline neutrino oscillation experiment using a J-PARC neutrino beam and Hyper-Kamiokande". In: *Prog. Theor. Exp. Phys.* 2015 (2015), p. 053C02. eprint: 1502.05199.

[14] IceCube Collaboration. "Searching for eV-scale sterile neutrinos with eight years of atmospheric neutrinos at the IceCube neutrino telescope". In: *Phys. Rev. Lett.* 125 (2020), p. 141801. eprint: 2005.12943.

[15] IceCube Collaboration. "Measurement of atmospheric neutrino oscillation parameters using convolutional neural networks with 9.3 years of data in IceCube DeepCore". In: *Phys. Rev. D* 106 (2022), p. 032009. eprint: 2112.09122.

[16] J. Kopp, P. A. N. Machado, M. Maltoni, and T. Schwetz. "Sterile Neutrino Oscillations: The Global Picture". In: *JHEP* 05 (2013), p. 050. eprint: 1303.3011.

[17] Z. Maki, M. Nakagawa, and S. Sakata. “Remarks on the unified model of elementary particles”. In: *Prog. Theor. Phys.* 28 (1962), pp. 870–880.

[18] S. P. Mikheyev and A. Y. Smirnov. “Resonance Amplification of Oscillations in Matter and Spectroscopy of Solar Neutrinos”. In: *Sov. J. Nucl. Phys.* 42 (1985), pp. 913–917.

[19] NuFIT Collaboration. *NuFIT 5.2 (2022)*. http://www.nu-fit.org. 2022.

[20] S. J. Parke. “Nonadiabatic Level Crossing in Resonant Neutrino Oscillations”. In: *Phys. Rev. Lett.* 57 (1986), pp. 1275–1278.

[21] Particle Data Group, S. Navas, et al. “Review of Particle Physics”. In: *Phys. Rev. D* 110 (2024), p. 030001.

[22] B. Pontecorvo. “Mesonium and antimesonium”. In: *Sov. Phys. JETP* 6 (1958), p. 429.

[23] N. Vinyoles et al. “A New Generation of Standard Solar Models”. In: *Astrophys. J.* 835 (2017), p. 202. eprint: 1611.09867.

[24] L. Wolfenstein. “Neutrino oscillations in matter”. In: *Phys. Rev. D* 17 (1978), pp. 2369–2374.